

Машинное обучение на графах и рекомендательные системы (конспект лекций)

Бернгардт О.И.

ИГУ, ИСЗФ СО РАН

Книга представляет собой конспект лекций по графовым нейронным сетям и рекомендательным системам, читаемым автором в магистратуре и бакалавриате на факультете бизнес-коммуникаций и информатики Иркутского государственного университета. Материалы предназначены для студентов и специалистов, уже знакомых с базовыми методами машинного обучения с учителем и без учителя.

Работа выполнена при поддержке Фонда Потанина.

Содержание

1. Введение.....	6
2. Классическое машинное обучение на графах.....	8
2.1. Классические детерминированные эмбединги.....	8
2.1.1. Классические признаки узлов.....	8
2.1.1.1. Основные признаки.....	8
2.1.1.2. Графлеты и графлет-эмбединг для ноды.....	10
2.1.1.3. Кластеризация нод.....	11
2.1.2. Классические эмбединги ребра.....	12
2.1.2.1. Признаки, основанные на концах ребра.....	12
2.1.2.2. Признаки, основанные на расстоянии.....	13
2.1.2.3. Признаки, основанные на локальном соседстве.....	14
2.1.2.4. Глобальные признаки.....	15
2.1.2.5. Кластеризация ребер.....	16
2.1.3. Классические эмбединги для графа.....	17
2.1.3.1. Комбинация признаков нод.....	17
2.1.3.2. Алгоритм Вейсфейлера-Лемана.....	17
2.1.3.3. Кластеризация графов.....	18
3. Топологические эмбединги на основе близости.....	20
3.1. Случайные блуждания.....	20
3.1.1. Генерация случайных блужданий.....	20
3.1.2. Контролируемые блуждания и алгоритм Node2Vec.....	22
3.1.2.1. Контролируемые случайные блуждания.....	23
3.1.2.2. Реализация Node2vec.....	25
3.1.3. Блуждания с телепортацией и PageRank.....	30
3.1.4. Блуждания с перезапуском и рекомендательные системы.....	32
3.1.5. Обобщение PageRank.....	33
3.2. Эмбединги как факторизация матрицы смежности.....	34
3.2.1. Разложение Холецкого.....	34
3.2.2. Неотрицательная матричная факторизация.....	35
3.2.3. Поиск разложением по собственным векторам.....	36
3.2.4. Проблемы факторизации.....	37
3.3. Эмбединг для графа и анонимные блуждания.....	37
4. Учет внутренних признаков нод: передача сообщений в графах.....	40
4.1. Бинарная классификация нод.....	40
4.2. Передача сообщений и агрегация.....	40
4.3. Коллективный классификатор.....	41
4.3.1. Локальный классификатор.....	42
4.3.2. Классификатор отношений (связный классификатор).....	43
4.3.3. Коллективные вычисления.....	45
4.3.4. Обобщение на взвешенные графы.....	46
4.4. Распространение доверия.....	47
4.5. Корректировка и сглаживание.....	47
5. Графовые нейронные сети.....	49
5.1. Введение.....	49
5.2. Простейшее решение задачи прогноза на графе.....	49
5.2.1. Простейший подход и его недостатки.....	49
5.2.2. Граф вычислений графовой нейронной сети.....	50

5.5.3.Графовая свертка.....	52
5.3.Обучение графовых нейронных сетей.....	54
5.3.1.Типы задач и обучение графовых нейронных сетей.....	54
5.3.1.1. Задача уровня узла.....	55
5.3.1.2. Задача уровня ребра.....	55
5.3.1.3. Задача уровня графа.....	55
5.3.2.Методы обучения на графах.....	56
5.3.3.Типы задач на графах и выходные головы.....	56
5.3.4.Деление данных на обучающую, валидационную, и тестовую выборки.....	57
5.3.4.1.Деление при задачах уровня узла.....	57
5.3.4.2.Деление при задачах уровня ребра.....	58
5.3.4.3.Деление при задачах уровня графа.....	59
5.3.5.Функции потерь.....	59
5.3.6.Метрики качества.....	60
5.3.7. Пример прогноза класса узла сверточной сетью.....	60
6. Повышение качества графовых нейронных сетей.....	62
6.1.Аугментация графа.....	63
6.1.1.Узлы с малым числом признаков.....	63
6.1.2.Разреженные графы.....	64
6.1.3.Плотные графы.....	64
6.2.Усложнение агрегации слоя.....	65
6.2.1.Модернизация признаков.....	65
6.2.2.Проблема циклов в графах.....	65
6.3.Улучшенные архитектуры сетей.....	66
6.3.1.Skip-connections и Residual blocks.....	66
6.3.2.Сеть графового внимания (GAT, Graph Attention Network).....	67
6.3.3.Сэмплирование и агрегирование (GraphSAGE).....	68
6.3.4.Вариационный графовый автоэнкодер (VGAE).....	70
7.Решение задач уровня графа нейронными сетями.....	73
7.1.Простые способы кодирования графа.....	73
7.2.Сеть графового изоморфизма(GIN).....	74
8.Неоднородные графы.....	77
8.1. Графовая сверточная нейронная сеть отношений (R-GCN).....	77
8.2.Обучение R-GCN.....	78
8.3.Особенности разделения датасетов.....	79
8.4.Проблема масштабируемости R-GCN.....	80
9.Графы знаний.....	81
9.1.Особенности задач на графах знаний.....	81
9.2.Типы отношений в графах знаний.....	81
9.3.Задача прогноза в графе знаний.....	83
9.3.Translating Embeddings (TransE).....	83
9.3.1.Идея метода.....	83
9.3.2.Обучение модели TransE.....	84
9.3.3.Ограничения TransE.....	84
9.4.Transition in relations space(TransR).....	85
9.4.1.Идея алгоритма.....	85
9.4.2.Обучение.....	86
9.4.3.Оценка итогового качества.....	86
10.Рассуждения в графах знаний.....	88
10.1.Типы сложных запросов.....	88

10.2.Коробочные эмбединги.....	88
10.3.Ограничения коробочного эмбединга.....	89
11.Задачи уровня подграфа.....	91
11.1.Выделение подграфов.....	91
11.2.Проверка изоморфизма графов - классические подходы.....	91
11.3.Проверка вхождения графов методами машинного обучения.....	92
11.3.1.Неотрицательные упорядоченные эмбединги.....	92
11.3.2.Функция потерь для поиска неотрицательных эмбедингов.....	93
11.4.Поиск значимых подграфов.....	95
11.4.1.Оценка частоты подграфов.....	95
11.4.2.Генерация случайных графов для сравнения.....	97
11.4.3.Поиск значимых подграфов методами проверки гипотез.....	98
11.4.4.Извлечение наиболее частых подграфов заданного размера.....	99
12.Рекомендательные системы.....	101
12.1.Модели ранжирования на основе эмбедингов.....	102
12.2.Дифференцируемые потери.....	102
12.2.1. Бинарная кросс-энтропия.....	103
12.2.2. Байесовские персонализированные ранговые потери.....	103
12.3.Матричная факторизация.....	103
12.4.Учёт более широких окрестностей.....	105
12.4.1.Архитектура модели NGCF.....	105
12.4.1.1.Инициализация эмбедингов.....	105
12.4.1.2.Распространение эмбедингов и обучение.....	105
12.4.2.Архитектура LightGCN.....	107
13.Сообщества.....	108
13.1.Поиск сообществ в графе.....	108
13.2.Поиск сообществ через слабые ребра. Алгоритм Лёвена.....	110
13.3.Поиск сообществ через общие ноды.....	111
13.3.1.Модель графа принадлежности (AGM).....	111
13.3.2.Нейронное обнаружение перекрывающихся сообществ (NOCD).....	112
13.3.3.Перенос обучения: использование классификатора.....	114
Рекомендуемая литература и источники к курсу.....	115

1. Введение

Граф - это объект, состоящий из нод (узлов, вершин) и ребер (связей, линков).

Одним из способов описания графа является матрица смежности A_{ij} . Матрица смежности в простейшем варианте - это матрица N на N , где N - число узлов в графе. Если два узла u и v соединены ребром, то в этой матрице ставится какое-то число (вес ребра, $d(u, v)$). Если не соединены - ставится ноль.

Классификация графов по взвешанности:

- Невзвешенный граф: матрица смежности $A_{i,j}$ содержит только 0 и 1.
- Взвешенный граф: элементы $A_{i,j}$ принимают произвольные числовые значения (веса рёбер).

Классификация графов по направленности:

- Ненаправленный граф: $A = A^T$.
- Направленный граф: $A \neq A^T$.

Если граф ненаправленный, это означает, что путь открыт в оба направления. То есть, как из узла A можно попасть в узел B , так и из узла B можно попасть в узел A по той же траектории. Для направленного графа матрица смежности A не симметрична. Если существует элемент, у которого нет симметричного элемента относительно главной диагонали, то граф направленный.

Путь в графе - это последовательность нод, которые соединены ребрами.

Классификация графов по связности: если в графе существуют две точки (узлы), которые нельзя соединить путём, проходящим по рёбрам графа, то граф называется несвязным. Граф, в котором из любого узла можно попасть в любой другой узел по рёбрам, называется связным. Для направленных графов есть понятия слабой и сильной связности.

Базовые параметры графа:

- Плотность $\rho = \frac{(2|E|)}{|V|(|V|-1)}$, где E, V - число ребер и нод соответственно.

Плотность графа определяется как число ребер в графе деленое на максимально возможное число ребер в графе с таким числом нод.

- Расстояние между узлами u и v - это длина кратчайшего пути $dist(u, v) = d(u, v)$.
- Диаметр графа: $diam(G) = \max_{u, v \in V} dist(u, v)$ - максимальное из всех кратчайших расстояний.
- Центр графа $center(G)$ - узел с минимальным эксцентриситетом (центр - это нода в графе, от которой максимальное расстояние до любой ноды минимально): $center(G) : argmin_{v \in G} (\max_{u \in G} dist(v, u))$. Центров у графа может быть много.
- Радиус графа: максимальное расстояние от центра до нод графа: $rad(G) : (\max_{u \in G} dist(center(G), u))$.

Классификация графов по плотности:

- Плотный граф: плотность $\rho \approx 1$ - почти все возможные рёбра присутствуют.
- Разреженный граф: $\rho \ll 1$ - большинство возможных рёбер отсутствуют.

Плотные графы - это графы, где из любого узла можно попасть почти в любой другой узел за один шаг (через одно ребро). В реальном мире большинство графов разреженные. Это означает, что в матрице смежности таких графов большинство элементов - нули. Поэтому при работе с матрицами смежности часто используют разреженные форматы хранения, указывая только ненулевые элементы.

Граф, узлы которого можно разбить на два непересекающихся множества U и V , так что все рёбра идут только между U и V - это двудольный граф. Двудольные графы часто используются в рекомендательных системах.

Основные обозначения используемые в тексте

- G - граф;
- V - множество узлов (нодов);
- E - множество ребер (связей);
- A - матрица смежности: $A_{uv} = 1$ (или вес ребра), если есть ребро $u \rightarrow v$, иначе 0;
- X - матрица исходных признаков узлов: строка X_u - внутренние признаки узла u ;
- H - матрица представлений (признаков) узлов в графовой сети: строка h_u - представление для узла u (обычно промежуточное);
- Z_α - аналогично, представление, признак, эмбединг объекта α (обычно окончательное);
- $N(v)$ - множество соседей узла v : $N(v) = \{u \in G | A_{vu} > 0\}$.

Алгоритм 1. Построение графа Karate Club

```
import networkx as nx
G = nx.karate_club_graph()
nx.draw(G, with_labels=True)
```

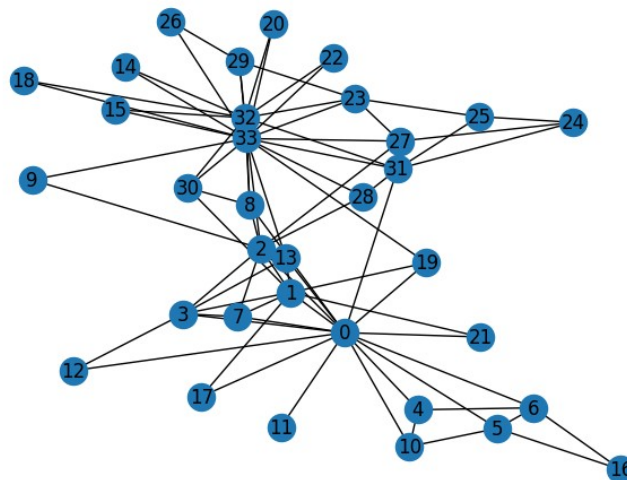


Рис.1. Граф Karate Club. Число внутри ноды - номер ноды.

2. Классическое машинное обучение на графах

На графах существуют задачи как с учителем, так и без учителя. К задачам с учителем обычно относятся классификация и регрессия (предсказание). К задачам без учителя - кластеризация, поиск аномалий, генеративные задачи.

В машинном обучении на обычных (неструктурированных) данных модель получает данные и часть из них кодируется (обычно категориальные переменные заменяются на их векторные представления - эмбединги). Всё это подаётся на модель машинного обучения для получения результата обработки. Если обучение с учителем, модель тренируется на размеченных данных. Если обучение без учителя, подбирается подходящая модель для выявления необходимых закономерностей.

Так как модели машинного обучения обычно работают с числами, категориальные переменные преобразуются в числовые векторы (эмбединги). Оптимальный эмбединг подбирается под конкретную задачу.

С графами ситуация сложнее. Первичные данные сильно структурированы и представляют собой сеть связей (граф), где каждый узел также обладает своими характеристиками (например, для актёра - рост, вес, возраст и т.д.) и между узлами существуют различные ребра (они могут иметь разный вес, тип, направленность). Чтобы подать эти данные на модели машинного обучения, необходимо создать подходящее векторное представление для таких данных - эмбединги.

2.1. Классические детерминированные эмбединги

2.1.1. Классические признаки узлов

2.1.1.1. Основные признаки

Классические признаки нод делятся на локальные и глобальные и дают способ оценить важность ноды в графе, основываясь только на ее связях с остальными нодами. Локальные учитывают небольшое число нод в окрестности исследуемой ноды, глобальные учитывают все ноды графа.

К классическим локальным признакам для ноды относят:

1. Степень вершины (degree) $deg(v) = \sum_{u \in G} A_{v,u}$.
2. Локальный коэффициент кластеризации - $Clustering(v) = \frac{\sum_{s,t \in N(v), s \in N(t)} 1}{\sum_{s,t \in N(v)} 1}$
3. Центральность по степени (degree centrality), обычно определяется по степени ноды: $C_d(v) = deg(v)/(N-1)$.
4. Центральность близости (closeness centrality) - мера близости узла ко всем остальным узлам в графе: $C_c(v) = \frac{N}{\sum_{u \in G} d(u,v)}$
5. Промежуточная центральность (betweenness centrality) - мера того, как часто узел лежит на кратчайших путях между другими узлами. $C_b(v) = \frac{\sum_{s,t \in G} \sigma(s,t|v)}{\sum_{s,t \in G} \sigma(s,t)}$ здесь

$\sigma(s,t), \sigma(s,t|v)$ - число кратчайших путей между узлами s и t , и число таких кратчайших путей проходящих через узел v соответственно.

6. Центральность по собственному вектору (eigenvector centrality) - мера влияния узла в сети. Узел имеет высокую центральность, если он связан с узлами, которые сами имеют высокую центральность. Вектор центральностей $C_{eig}(v) = x_v$ для узлов удовлетворяет уравнению $Ax = \lambda x$, где λ - наибольшее собственное значение для матрицы смежности A .
7. Число графлетов заданного типа, в которых участвует узел в графе. Этот признак будет рассмотрен позже.

К классическим глобальным признакам для узла относят PageRank (PR), рекурсивно определяемую величину оценивающую важность узла в направленном графе с учетом важности остальных узлов и связей между ними:

$$PR(v) = (1 - \beta) / N + \beta \cdot \sum_{u \in N(v)} PR(u) / \deg_{out}(u)$$

где β - множитель затухания (обычно 0.85), N - число узлов в графе, $\deg_{out}(u)$ - число ребер выходящих из u , $N(u)$ - соседи узла u . Детально также этот признак будет рассматриваться позже.

Алгоритм 2. Классические признаки узла графа - степени вершин

```
import networkx as nx
G=nx.karate_club_graph()
d = dict(G.degree)
nx.draw(G,labels=d)
```

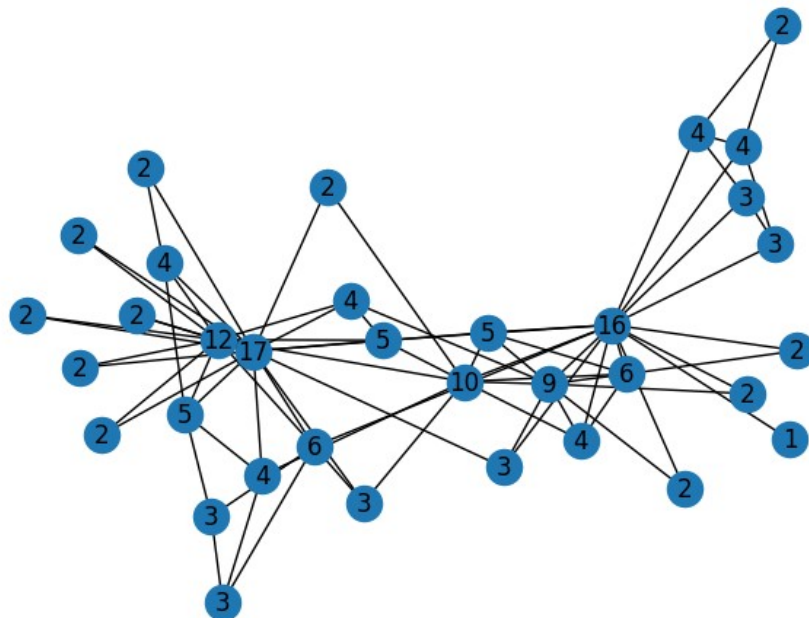


Рис.2. Степени узла графа Karate Club. Число внутри узла - степень узла.

2.1.1.2.Графлеты и графлет-эмбединг для ноды

Каждую вершину можно описать (закодировать) тем, как относительно неё расположены её соседи. Закодировать структуру вокруг заданной вершины можно, рассматривая небольшие подсети (подграфы, графлеты), в которые она входит. В графлетах часто не различают конкретные вершины, а только структуру соединений - различаются лишь исследуемая нода и остальные, что соответствует бинарной разметке.

Графлет - это шаблон (элементарный объект, небольшой связный подграф), который описывает способ соединения небольшой группы узлов относительно изучаемой (анкерной) ноды (Рис.3).

Число графлетов разного размера:

- Двухузловые графлеты: 1 вариант;
- Трёхузловые графлеты: 3 варианта;
- Четырёхузловые графлеты: 11 вариантов;
- Пятиузловые графлеты: 58 вариантов.

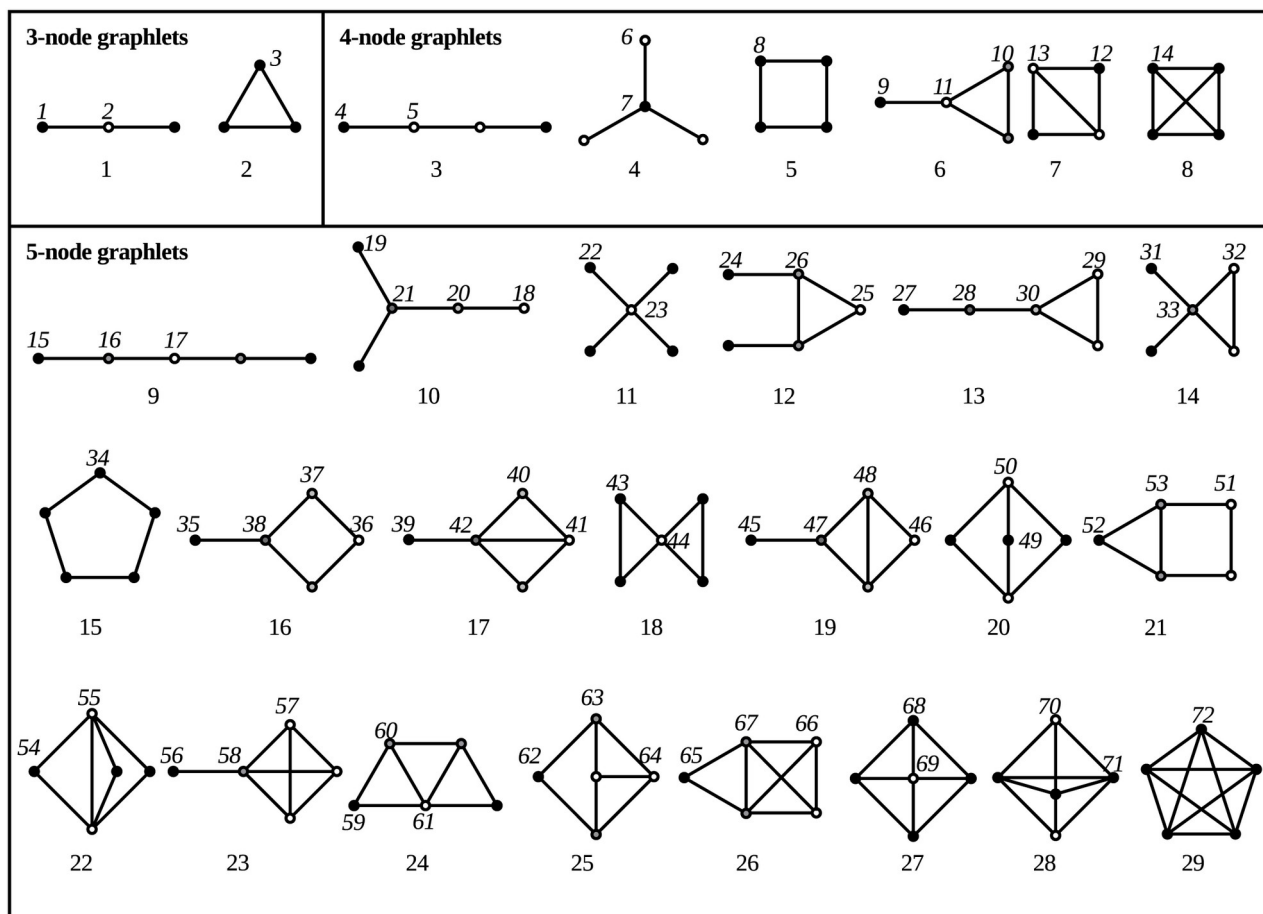


Рис.3. Виды графлетов с учетом положения анкерной ноды, из [Melckenbeeck et al (2016)//PLoS ONE 11(1): e0147078. <https://doi.org/10.1371/journal.pone.0147078>]. Точки отмеченные номерами соответствуют положению анкерной ноды.

Таким образом, когда мы смотрим на окружение какой-то вершины, мы можем описать его вектором, где каждый элемент - это количество раз, которое данная вершина

участвует в конкретном типе графлета, содержащих эту вершину и некоторых её соседей. Этот вектор фактически описывает локальную топологию графа вблизи этой вершины.

Количество возможных типов графлетов (а значит, и размерность вектора признаков) растёт очень быстро, экспоненциально с увеличением числа узлов в графлете. Поэтому на практике достаточно ограничиться небольшим количеством графлетов (например, до размера 4 или 5).

Алгоритм 3. Определение гистограммы графлетов для нод графа

```
import pyfglt.fglt as fg
F = fg.compute(G)
F.head()
```

Node id (0-based)	[0] vertex (==1)	[1] degree	[2] 2- path	[3] bifork	[4] 3- cycle	[5] 3- path, end	[6] 3- path, interior	[7] claw, leaf	[8] claw, root	[9] paw, handle	[10] paw, base	[11] paw, center	[12] 4- cycle	[13] diamond, off-cord	[14] diamond, on-cord	[15] 4- clique
0	1	16	17	102	18	81	197	13	352	6	34	171	10	2	30	7
1	1	9	19	24	12	73	56	33	32	8	80	27	6	2	18	7
2	1	10	34	34	11	72	179	84	54	17	75	51	20	6	8	7
3	1	6	20	5	10	49	11	56	1	5	81	5	0	4	7	7
4	1	3	16	1	2	17	1	64	0	15	25	0	1	2	1	0

2.1.1.3. Кластеризация нод

Одним из известных способов кластеризации является метод "мешка слов" (bag of words) - построение гистограммы частот появления различных частей в объектах.

Каждый узел (ноду) в графе можно описать гистограммой (вектором встречаемости) его классических признаков, и весь граф будет описываться как набор таких гистограмм (векторов). Соответственно можно проводить кластеризацию нод в графе известными методами кластеризации, что позволит кластеризовать граф, основываясь на топологических признаках его нод.

Пример кластеризации нод графа Karate Club по графлетам приведен в Алгоритме 4. Для кластеризации используется графлет-представление каждой ноды в качестве вектора ее признаков. Поскольку графлет-представление велико по сравнению с количеством нод (16 признаков и 35 нод), чтобы избежать проклятия размерности используется уменьшение количества признаков главных компонент (с качеством приближения не хуже 99%, до 5 признаков). Кластеризация на три кластера проводится методом смеси гауссовых распределений (Gaussian Mixture).

Из рис.4 видно, что кластеризация выделила в одном классе (зеленый цвет) — центральные ноды, имеющие максимальное число связей с соседями, в другой класс (желтый цвет) — ноды, с которыми они близко связаны, имеющие небольшое число соседей; а в третий класс (фиолетовый цвет) — ноды либо сильно удаленные от центральных нод, либо имеющие существенное (но не большое и не малое) число соседей.

Алгоритм 4. Пример кластеризации нод

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import pyfglt.fglt as fg
```

```

from sklearn.decomposition import PCA
from sklearn.mixture import GaussianMixture

G = nx.karate_club_graph()
F = fg.compute(G)
F=np.array(F)
pca=PCA(0.99)
X=pca.fit_transform(F)
gm=GaussianMixture(n_components=3)
c=gm.fit_predict(X)
pos = nx.kamada_kawai_layout(G)
nx.draw(G, pos, node_color=c, with_labels=True)
plt.savefig('cluster.png')

```

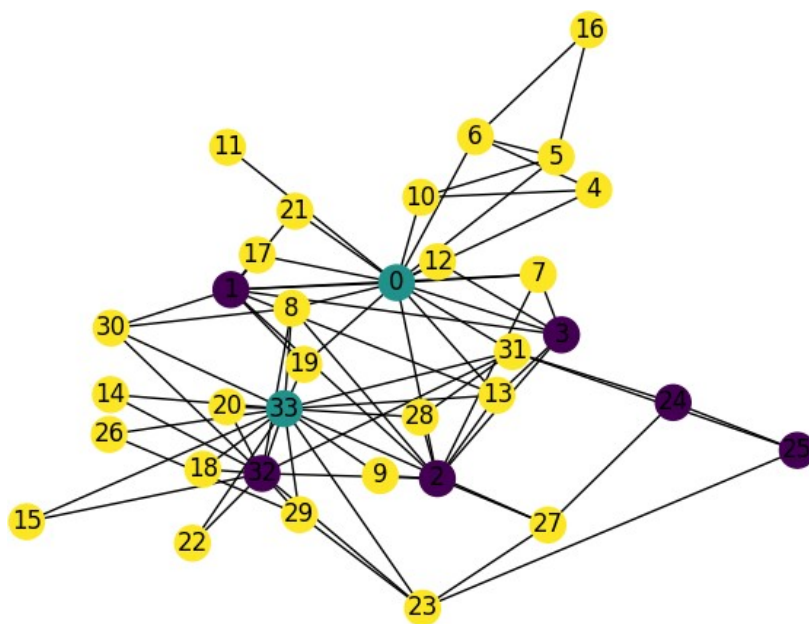


Рис.4. Результат кластеризации нод. Разным цветам соответствуют разные кластера.

2.1.2.Классические эмбединги ребра

2.1.2.1.Признаки, основанные на концах ребра

Часто возникает задача, когда необходимо предсказать, появится ли ребро между двумя узлами. Такая задача может возникать, например, в рекомендательных системах, когда нужно предсказать связь (интерес, покупку) между пользователем и товаром. Эта задача называется задачей прогноза (предсказания) ребра.

Соответственно, возникает проблема: как кодировать (представлять) рёбра? У ребра, в отличие от узла, есть два конца. Самый простой способ закодировать ребро - это взять вектор признаков (эмбединг) каждого из двух соединяемых узлов и сделать из них комбинированный признак для ребра. Например, если ребро соединяет узел U и узел V , у которых эмбединги (векторы признаков) Z_u и Z_v , то самый простой способ - это их сконкатенировать (объединить в один длинный вектор): $H_{uv}=[Z_u||Z_v]$.

В чём проблема такого подхода? Это представление работает только для направленных графов. Если граф ненаправленный, это означает, что ребро из U в V и ребро

из V в U - это одно и то же. Значит, и $H_{uv} = H_{vu}$: эмбединг ребра должен быть инвариантным (симметричным) относительно перестановки концов, а конкатенация несимметрична.

Чтобы сделать признак симметричным, можно использовать не конкатенацию, а, например, поэлементную сумму или произведение векторов: $H_{uv} = Z_u + Z_v$ или $H_{uv} = Z_u \odot Z_v$

Алгоритм 5. Определение эмбедингов для ребер и их распределение

```
import matplotlib.pyplot as plt
import numpy as np
d = dict(G.degree)
Huv=[]
for u,v in G.edges():
    Huv.append([u,v,d[u]+d[v]])
Huv=np.array(Huv)
plt.xlabel('$H_{uv}$')
plt.ylabel('# of cases')
plt.hist(Huv[:,2],bins=50);
```

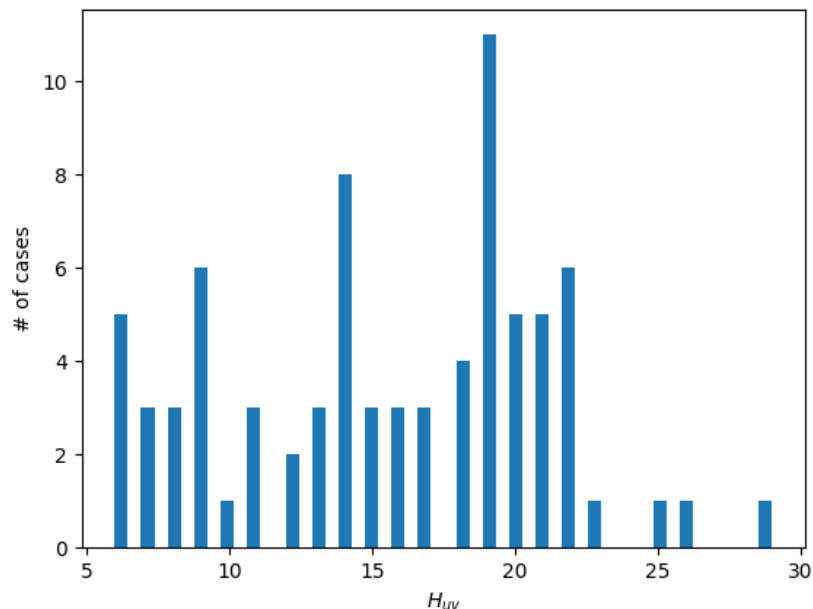


Рис.5 Распределение ребер по эмбедингам.

А если ребро соединяет две части графа, далёкие друг от друга? Проблема эмбедингов основанных только на признаках двух соединяемых узлов в том, что они не учитывают глобальную структуру графа, остальные рёбра и нода.

Существует несколько категорий признаков для пары узлов (потенциального ребра):

1. Признаки, основанные на расстоянии
2. Признаки, основанные на локальном соседстве (пересечении окрестностей)
3. Признаки, основанные на глобальном соседстве (или путях)

2.1.2.2. Признаки, основанные на расстоянии

Как определить, насколько близки два узла? Самый простой способ - вычислить длину кратчайшего пути между ними. Если мы хотим закодировать потенциальное ребро между узлом A и узлом Z, то в качестве одного из признаков этого ребра можно взять длину кратчайшего пути между A и Z в исходном графе. Например, кратчайший путь $A \rightarrow C \rightarrow D \rightarrow E \rightarrow Z$ имеет длину 4 (количество рёбер). Кратчайшее расстояние (количество рёбер в кратчайшем пути) можно посчитать с помощью алгоритмов Дейкстры, A* и т.д.

Этот метод позволяет кодировать не только существующие рёбра (для них длина равна 1), но и любые потенциальные пары узлов, между которыми ребра пока нет.

Для вычисления кратчайшего пути между двумя узлами в библиотеке NetworkX используется функция, например, `nx.shortest_path_length(G, node1, node2)`, которая возвращает длину пути.

2.1.2.3. Признаки, основанные на локальном соседстве

Для неориентированного ребра (u,v) нужны симметричные признаки.

Количество общих соседей (common neighbors). Количество общих соседей у двух нод u,v:

$$C(u, v) = |N(u) \cap N(v)|$$

Коэффициент Адамика-Адара (Adamic-Adar index). Это взвешенная версия подсчёта общих соседей, где каждый общий сосед вносит вклад, обратно пропорциональный логарифму его степени. Идея: общие соседи с маленькой степенью (редкие связи) являются более значимыми индикаторами сильной связи между нодами u и v, чем соседи с высокой степенью (популярные узлы):

$$AA(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{\log(deg(w))}$$

Коэффициент Жаккарда (Jaccard coefficient). Практически это известная метрика IoU (intersection over union) но в терминах множеств соседей двух нод:

$$J(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$$

где $N(u)$ - множество соседей ноды u.

Методы, основанные на пересечении окрестностей не работают, когда у двух узлов нет общих соседей (пересечение пусто). В этом случае все эти коэффициенты будут равны нулю. Это плохо, и связано с тем, что эти признаки не улавливают более дальние связи.

Алгоритм 6. Определение коэффициента Жаккарда для ребер и их отображение

```
edge_f = {}
G1=nx.Graph()
for u in G.nodes:
    for v in G.nodes:
        pred=nx.jaccard_coefficient(G,[(u,v)])
        l=list(pred)[0][2]
        edge_f[tuple((u,v))]=l
    G1.add_edge(u,v,weight=l)
pos=nx.circular_layout(G1)
edge_labels = nx.get_edge_attributes(G1, 'weight')
c=[]
for u,v in G1.edges:
    c.append(edge_labels[u,v])
```



```
nx.draw(G1, pos, edge_color=c,with_labels=True)
```

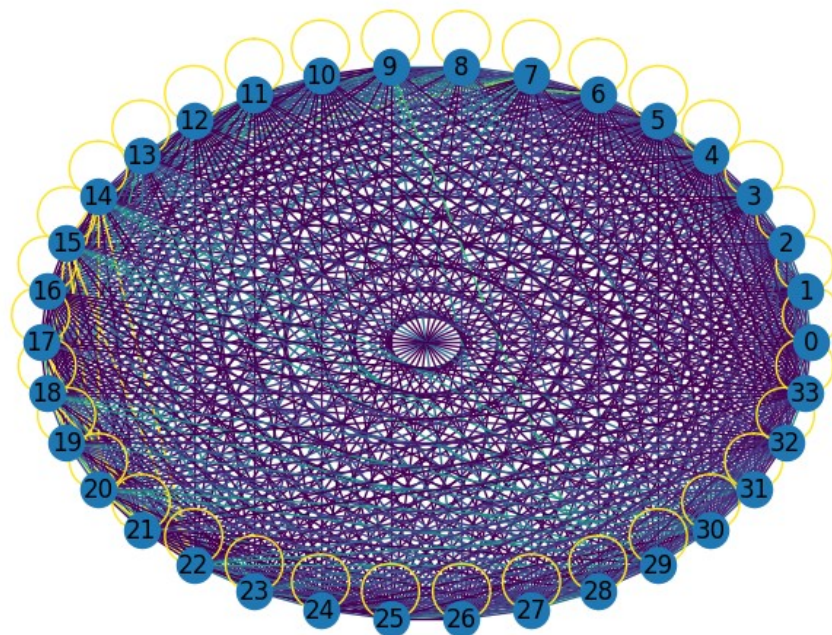


Рис.6. Распределение всех возможных ребер в графе Karate Club по коэффициенту Жаккарда. Разные цвета ребер соответствуют разным значениям коэффициента.

2.1.2.4. Глобальные признаки

Для учета дальних связей используются глобальные признаки. Глобальные признаки обычно считаются путём подсчёта числа путей между двумя узлами. Если рёбра графа взвешены, то можно считать сумму весов этих путей.

Чтобы посчитать количество путей длины 2 от A до B, нужно просуммировать количество путей от A ко всем промежуточным узлам, умноженное на количество путей от этих промежуточных узлов до B. Это математически сводится к возведению матрицы смежности A в квадрат.

Более того, количество путей длины L между любыми двумя узлами равно соответствующему элементу матрицы смежности, возведённой в степень L: A^L .

Идея глобального учёта связей заключается в том, чтобы просуммировать вклады путей всех возможных длин (от 1 до бесконечности), вычислив сумму бесконечного ряда:

$$S = A + A^2 + A^3 + \dots + A^n + \dots$$

Каждый элемент результирующей матрицы S_{uv} будет общим количеством всех путей (любой длины) от узла u к узлу v.

Однако проблема в том, что для связного графа такая сумма будет расходиться: с ростом длины количество путей будет расти.

Решение - ввести дисконт-фактор β , где $0 < \beta < 1$ и суммировать взвешенный ряд:

$$S = \beta A + (\beta A)^2 + (\beta A)^3 + \dots + (\beta A)^n + \dots$$

Такой ряд сходится, если β достаточно мало. Фактически, это геометрическая прогрессия для матриц.

Сумму этого бесконечного ряда можно вычислить по формуле, аналогичной формуле для суммы геометрической прогрессии. Для матриц формула принимает вид:

$$S = (I - \beta A)^{-1} - I,$$

где I - это единичная матрица (матрица, у которой на главной диагонали единицы, а все остальные элементы - нули). Таким образом, задача подсчёта всех путей свелась к относительно простой матричной операции - взятию обратной матрицы.

Этот индекс называется **индексом Каца** (Katz index).

Требование к β : он должен быть меньше, чем величина, обратная максимальному собственному значению матрицы смежности A :

$$\beta < 1/\max(\lambda): Ax = \lambda x$$

Каждый элемент матрицы S_{uv} является мерой связанности узлов u и v , учитывающей все возможные пути с экспоненциальным затуханием для более длинных путей.

2.1.2.5. Кластеризация ребер

Распределение значений коэффициента Жаккарда (или другого признака ребра) часто имеет несколько выраженных мод. Это позволяет провести кластеризацию самих рёбер. Например, можно взять массив значений коэффициента Жаккарда (или всех признаков ребер) для всех рёбер и применить классический алгоритм кластеризации. В результате каждое ребро получит метку кластера. Это позволяет выделить группы рёбер: например, рёбра с высоким коэффициентом (много общих соседей), рёбра с нулевым коэффициентом (нет общих соседей) и рёбра с промежуточными значениями.

Алгоритм 7. Кластеризация ребер по сумме графлет-представлений нод

```
import pyfglt.fglt as fg
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
F = fg.compute(G)
F=np.array(F)
edge_f = []
for u, v in G.edges: edge_f.append(F[u]+F[v])
edge_f=np.array(edge_f)
pca=PCA(n_components=2)
X=pca.fit_transform(edge_f)
clf=DBSCAN(eps=50)
c=clf.fit_predict(X)
pos=nx.kamada_kawai_layout(G)
nx.draw(G, pos, with_labels=True, edge_color=c)
```

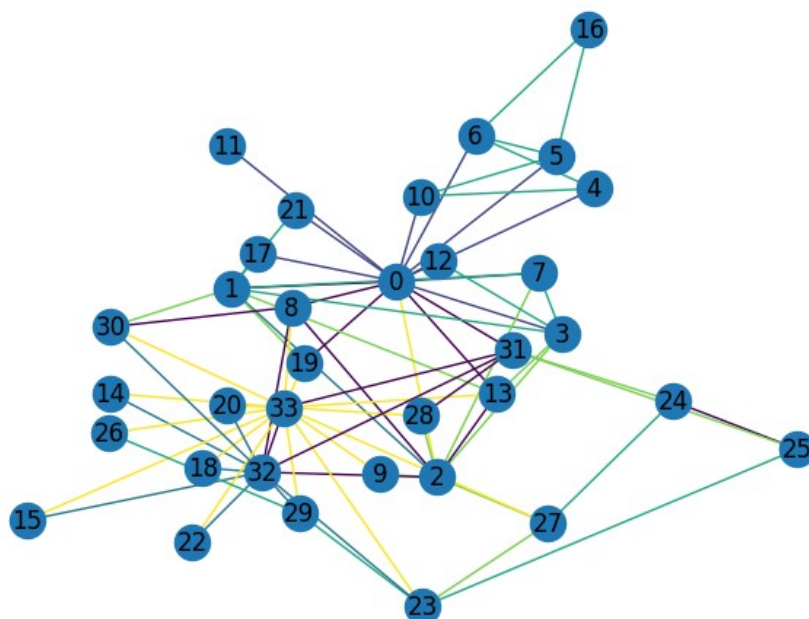



Рис 7. Результат кластеризации ребер по сумме графлет-представлений нод.

2.1.3. Классические эмбединги для графа

2.1.3.1. Комбинация признаков нод

Самым простым способом получить классические признаки для графа является комбинация признаков (нод или ребер). Чаще всего используют сумму или усреднение классических признаков нод.

Графлетное ядро (graphlet kernel) - один из вариантов такого подхода. По графлет представлению нод графа суммированием строится гистограмма вхождений малых подграфов (графлетов, например, 3-5 узлов) в граф.

Недостаток: вычислительная сложность растет экспоненциально, поэтому метод быстро становится медленным и непрактичным.

2.1.3.2. Алгоритм Вейсфейлера-Лемана

Для кодирования не отдельных узлов или пар, а всей структуры графа целиком, используются методы, учитывающие иерархию соседств. Один из таких классических алгоритмов - алгоритм Вейсфейлера-Лемана (Weisfeiler-Lehman, WL).

Идея алгоритма - итеративно присваивать узлам новые "метки" (хэши), которые агрегируют информацию о метках их соседей.

Алгоритм состоит из нескольких этапов (Рис.7):

1. Инициализация: Каждому узлу графа присваивается начальная метка. Например, всем узлам присваивается одинаковая метка "1" (или метка, основанная на степени вершины).

$$h^0(v) =_{v \in G} 1$$

2. Агрегация: На каждом шаге для каждого узла собирается мультимножество (упорядоченный список) меток всех его соседей.

$$h_v^t = [h_v^{t-1}, \{h_u^{t-1} | u \in N(v)\}]$$

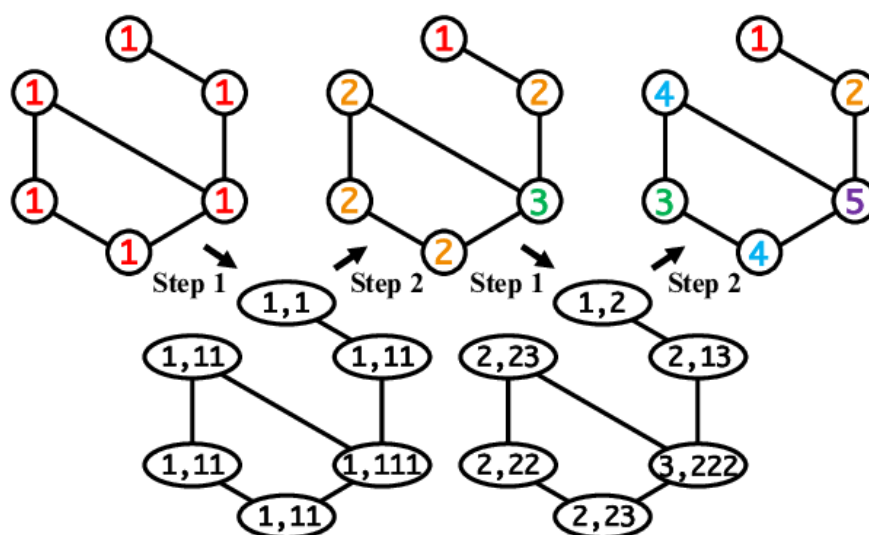
3. Хэширование (обновление меток): Новая метка узла вычисляется как хэш-функция от конкатенации его старой метки и отсортированного списка меток соседей. Этот хэш присваивается как новая уникальная метка узла. Для двух узлов с одинаковым окружением на этом шаге будет сгенерирована одинаковая новая метка.

$$h_v^t = \text{HASH}(h_v^t)$$

4. Повторение: Шаги 2 и 3 повторяются заданное количество раз (K итераций). Чем больше итераций, тем больше "радиус" окрестности, информацию о которой агрегирует метка узла.

5. Получение признаков графа: После K итераций подсчитывается гистограмма (вектор) частот встречаемости всех сгенерированных меток (на всех итерациях, включая начальную) для данного графа. Этот вектор и является эмбедингом (векторным представлением) всего графа.

$$EMB = p_{v \in G}(h_v^t) = Z_G$$



Step 1: generate signature strings. Step 2: sort signature strings and recolor.

Рис.8 Две итерации алгоритма Вейсфейлера-Лемана, из [Zhang et al. 2017/<https://doi.org/10.1145/3097983.3097996>]

Преимуществом алгоритма является то, что он работает за время, пропорциональное диаметру графа (число достаточных итераций алгоритма WL обычно не превышает диаметр).

2.1.3.3. Кластеризация графов

Если у нас есть много графов (например, множество различных молекул), мы получаем для каждого графа вектор-эмбединг. После этого мы можем кластеризовать эти векторы, чтобы найти похожие графы (например, молекулы со сходной структурой), или использовать их для классификации (например, "лекарство" vs "яд").

Таким образом, эмбединг для целого графа, необходим для задач решения кластеризации или классификации графов.

Алгоритм 8 представляет пример кластеризации графов по сумме графлет-представлений их нод. Таким образом каждый граф имеет 16-мерное векторное представление. В качестве набора графов используются случайные графы с одинаковым числом нод — Эрдеша-Реньи, Барбаши-Альберта (модель предпочтительного соединения) и Ватса-Строгаца (модель тесного мира), обладающие существенно различными топологиями (Рис.8), что и позволило их кластеризовать с качеством 100% (по индексу Рэнда). Кластеризация на три класса проводится методом смеси гауссовых распределений.

Алгоритм 8. Кластеризация графов по сумме графлет-представлений их нод

```
import networkx as nx
import numpy as np
import pyfglt.fglt as fg
from sklearn.mixture import GaussianMixture
from sklearn.metrics import confusion_matrix, rand_score

def make_emb(G):
    F = np.array(fg.compute(G))
    return F.sum(axis=0)
N=30
P=0.3
m0=2
GS,YS,XS=[],[],[]
for i in range(100):
    GS.append(nx.erdos_renyi_graph(N, P))
    YS.append(0)
    GS.append(nx.barabasi_albert_graph(N, m0))
    YS.append(1)
    GS.append(nx.watts_strogatz_graph(N, m0, P))
    YS.append(2)
for G in GS:
    XS.append(make_emb(G))
Yp=GaussianMixture(n_components=3).fit_predict(XS)
print(confusion_matrix(YS, Yp))
print('Rand score:', rand_score(YS, Yp))
```

```
[[ 0 100  0]
 [ 0  0 100]
 [100  0  0]]
Rand score: 1.0
```

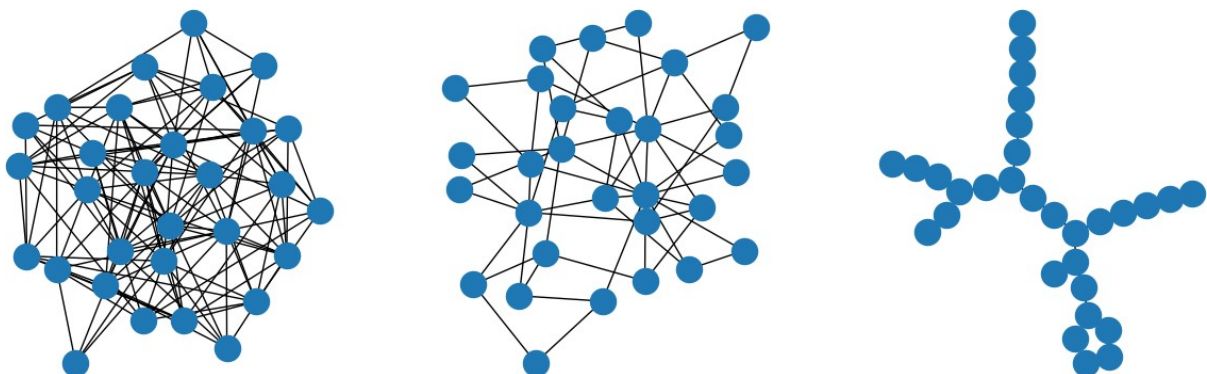


Рис.9 Примеры случайных графов различных типов, использованных для иллюстрации кластеризации, слева направо: Эрдеша-Реньи, Барбаши-Альберта, и Ватса-Строгаца.

3. Топологические эмбединги на основе близости

3.1. Случайные блуждания

3.1.1. Генерация случайных блужданий

Чтобы предсказать свойства ноды, зависящие от ее положения в графе, желательно иметь топологический эмбединг - представление, отражающее только структуру связей этой ноды с другими нодами - структуру графа с точки зрения этой ноды.

Какими свойствами должен обладать топологический эмбединг?

1. Не должен зависеть от номеров нод: нода видит только, то что у нее есть соседи и их свойства, но не идентифицирует их номера;
2. Не должен включать нетопологические атрибуты (признаки) нод;
3. Не должен зависеть от конечной задачи.

Поэтому такие эмбединги обучают на какой-нибудь универсальной задаче, например восстановления топологических характеристик удалённой ноды или вероятности того, что две ноды соединены ребром.

Для определения близости двух нод в таком подходе используется условная вероятность того, что нода u является близкой к v при их известных эмбедингах Z_u, Z_v .

В основе обучения эмбедингов лежит случайное блуждание по графу (random walks). Случайное блуждание по графу - это процесс, при котором агент движется от вершины к вершине графа, на каждом шаге равновероятно выбирая одного из соседей (по рёбрам графа; длина каждого ребра = 1). Одно случайное блуждание это следующий процесс:

1. Выбираем стартовую ноду v .
2. Смотрим на всех соседей текущей ноды,
3. Равновероятно выбираем одного из них,
4. Переходим туда.
5. Повторяем шаги 2-4 пока не набрано блуждание заданной длины

Это равномерное случайное блуждание (simple random walk), на каждом шаге вероятность перехода в соседа u равна $p_{u \in N(v)}(u) = 1/\deg(v)$, где $\deg(v)$ - степень вершины v .

Алгоритм 9. Генерация случайных блужданий

```
import networkx as nx
import random
import matplotlib.pyplot as plt

def drawGraphWithWalk(G, walk):
    pos=nx.kamada_kawai_layout(G)
    c,k=[],0
    for u,v in G.edges:
        found=False
        for k in range(len(walk)-1):
            if (walk[k]==u and walk[k+1]==v) or (walk[k]==v and walk[k+1]==u):
                found=True
                break
        if found: c.append('red')
```

```

    else: c.append('blue')
nx.draw(G,pos,with_labels=True,edge_color=c)

def generate_random_walks(G, num_walks=10, walk_length=80):
    walks = []
    nodes = list(G.nodes())
    for _ in range(num_walks):
        random.shuffle(nodes)
        node=nodes[0]
        walk = [node]
        for _ in range(walk_length - 1):
            curr = walk[-1]
            neighbors = list(G.neighbors(curr))
            if neighbors: walk.append(random.choice(neighbors))
            else: walk.append(curr)
        walks.append(walk)
    return walks

G = nx.karate_club_graph()
plt.figure(figsize=(13,13))
for i in range(4):
    walks = generate_random_walks(G, num_walks=1, walk_length=7)
    plt.subplot(2,2,i+1)
    drawGraphWithWalk(G,walks[0])

```

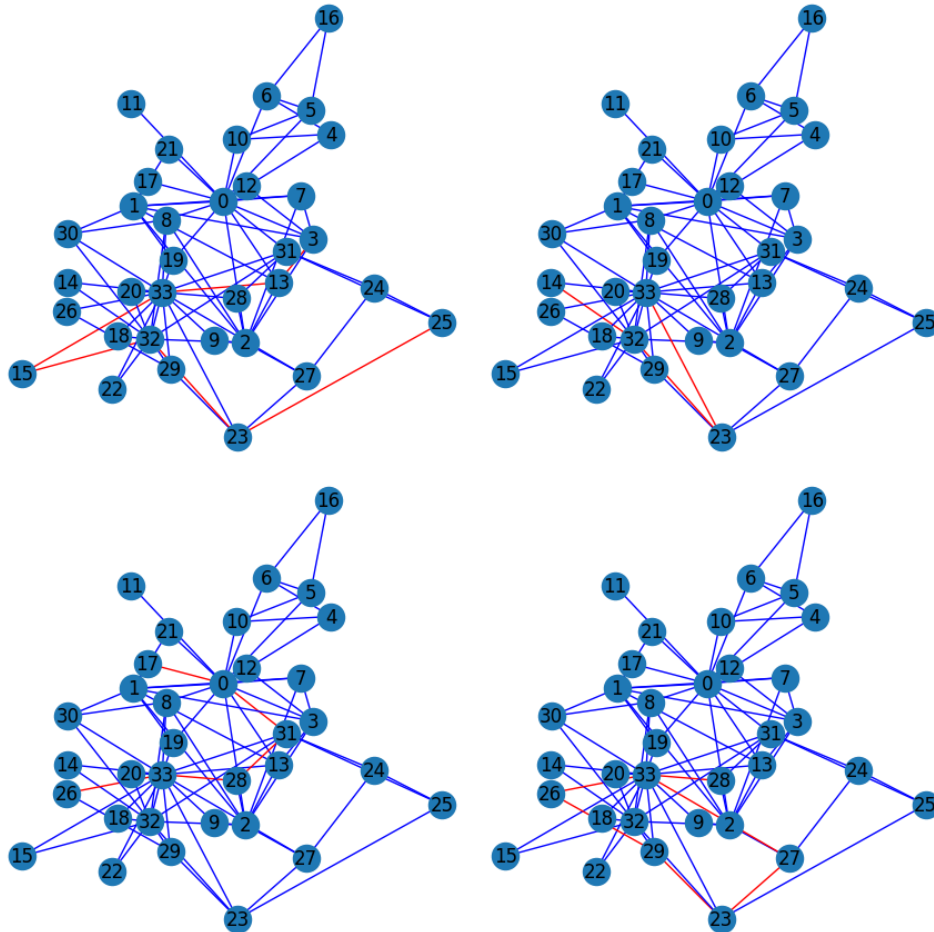


Рис.10. Случайные блуждания по графу Karate Club. Красным цветом выделена траектория блуждания.

При этом близость нод $score(u, v) = P(u|v)$ определяется как вероятность попадания из v в u при случайном блуждании. Для длинных путей эта вероятность вычисляется эмпирически через частоту появления u в блужданиях, стартующих из v .

Для решения задачи обычно генерируется большой набор случайных блужданий (воков) и проводится оценка вероятности попадания из v в u .

Чтобы ее оценить нужно:

1. Сгенерировать много случайных блужданий из v ;
2. В каждом блуждании длины L проверить: встречается ли u хотя бы один раз?
3. Оценить классическую вероятность перехода:

$$P(u|v) = (\text{число воков, где } u \text{ встречался}) / (\text{общее число воков})$$

3.1.2. Контролируемые блуждания и алгоритм Node2Vec

Набор сгенерированных случайных блужданий можно использовать для обучения векторных представлений (эмбедингов) узлов. Идея заключается в следующем: можно использовать случайное блуждание как ряд эмбедингов, и пытаться предсказывать эмбединг одного из нод по эмбедингам его соседей в этом ряду.

Цель обучения - настроить эмбединги нод так, чтобы скалярное произведение векторов-эмбедингов связанных узлов (встречающихся рядом в блужданиях) было

большим, а векторов несвязанных узлов (не встречающихся рядом в блужданиях) - малым. Фактически, мы подбираем эмбединги так, чтобы вероятность перехода от одного узла к другому была пропорциональна косинусному сходству (экспоненте от скалярного произведения) их эмбедингов.

3.1.2.1. Контролируемые случайные блуждания

В методе Node2Vec [https://arxiv.org/abs/1607.00653] используются случайные блуждания, отличающиеся от простого случайного блуждания. Если вы стартуете из узла и сразу идёте случайно (классическим методом случайных блужданий) - агент быстро уйдёт далеко от исходной узла и блуждание потеряет учет локальной (близкой к стартовому узлу) структуры. Поэтому обычно учитывают три типа перемещений при блуждании:

1. Возврат назад - к предыдущей узле;
2. Обход в ширину (BFS-style) - посещение соседей предыдущей узлы;
3. Обход в глубину (DFS-style) - продвижение вперёд, дальше от предыдущей узлы.

Такое блуждание называется контролируемым, а выбор баланса между этими тремя вариантами называется политикой случайного блуждания. В алгоритме Node2Vec для определения политики задаются два параметра: p - параметр для возврата назад, q - параметр для обхода вглубь. Эти два параметра, p и q , управляют стратегией обхода графа, позволяя балансировать между исследованием локальной окрестности начальной узлы (поиск в ширину, BFS) и уходом от начальной узлы в дальние части графа (поиск в глубину, DFS).

Выбор следующего узла при контролируемых случайных блужданиях в Node2Vec происходит следующим образом:

1. Пусть блуждание перешло из узла t (предыдущего) в узел v (текущего) и нужно выбрать следующий узел x среди соседей v . Всем соседям x назначаются веса, которые затем нормируются в вероятности.
2. Вес для возврата к предыдущему узлу t : $W(t|(t,v))=1/p$. Малое значение p ($p \ll 1$) увеличивает вероятность вернуться назад, заставляя блуждание "зацикливаться" на небольшой области. Большое p ($p \gg 1$) уменьшает эту вероятность, поощряя уход от t .
3. Вес для перехода к узлу x , который также является соседом предыдущего узла t ($x \in N(t) \wedge x \in N(v)$) равен 1: $W(x \in N(t)|(t,v))=1$. Это переход внутри локальной структуры вокруг узлы t .
4. Вес для перехода к узлу x , который не является соседом предыдущего узла t ($x \notin N(t) \wedge x \in N(v)$) равен $1/q$: $W(x \notin N(t) \wedge x \in N(v)|(t,v))=1/q$. Малое значение q ($q \ll 1$) увеличивает вероятность такого перехода, поощряя блуждание уходить дальше от исходной точки (режим "глубины", DFS). Большое q ($q \gg 1$) уменьшает эту вероятность, удерживая блуждание ближе к исходной окрестности (режим "ширины", BFS).
5. Нормируем веса по L1-норме, чтобы получить вероятности перехода к каждой узле из соседей v :

$$P(x|(t,v)) = \frac{W(x|(t,v))}{\sum_{y \in N(v)} W(y|(t,v))}$$

6. Случайно выбираем узлу из соседей v , пользуясь вычисленными вероятностями перехода

Таким образом, варьируя параметры p и q , можно генерировать блуждания, которые по-разному "чувствуют" структуру графа: одни лучше выявляют локальные сообщества (большое q), а другие - узлы, выполняющие сходные глобальные роли (малое q).

Алгоритм 10. Генерация контролируемых случайных блужданий

```
def next_node(G, previous, current, p, q):
    neighbors = list(G.neighbors(current))
    alphas = []
    for neighbor in neighbors:
        if neighbor == previous: alpha = 1/p
        elif G.has_edge(neighbor, previous): alpha = 1
        else: alpha = 1/q
    alphas.append(alpha)
    probs = [alpha / sum(alphas) for alpha in alphas]
    next = np.random.choice(neighbors, size=1, p=probs)[0]
    return next

def random_walk(G, start, length, p, q):
    walk = [start]
    for i in range(length):
        current = walk[-1]
        previous = walk[-2] if len(walk) > 1 else None
        next = next_node(G, previous, current, p, q)
        walk.append(next)
    return [int(x) for x in walk]

plt.figure(figsize=(17,17))
for i in range(3):
    plt.subplot(3,3,i+1)
    plt.title('В ширину')
    walks=random_walk(G, 0, 8, p=100, q=100)
    drawGraphWithWalk(G,walks)
for i in range(3):
    plt.subplot(3,3,i+4)
    plt.title('В глубину')
    walks=random_walk(G, 0, 8, p=100, q=0.001)
    drawGraphWithWalk(G,walks)
for i in range(3):
    plt.subplot(3,3,i+7)
    plt.title('С возвратом')
    walks=random_walk(G, 0, 8, p=0.001, q=100)
    drawGraphWithWalk(G,walks)
```

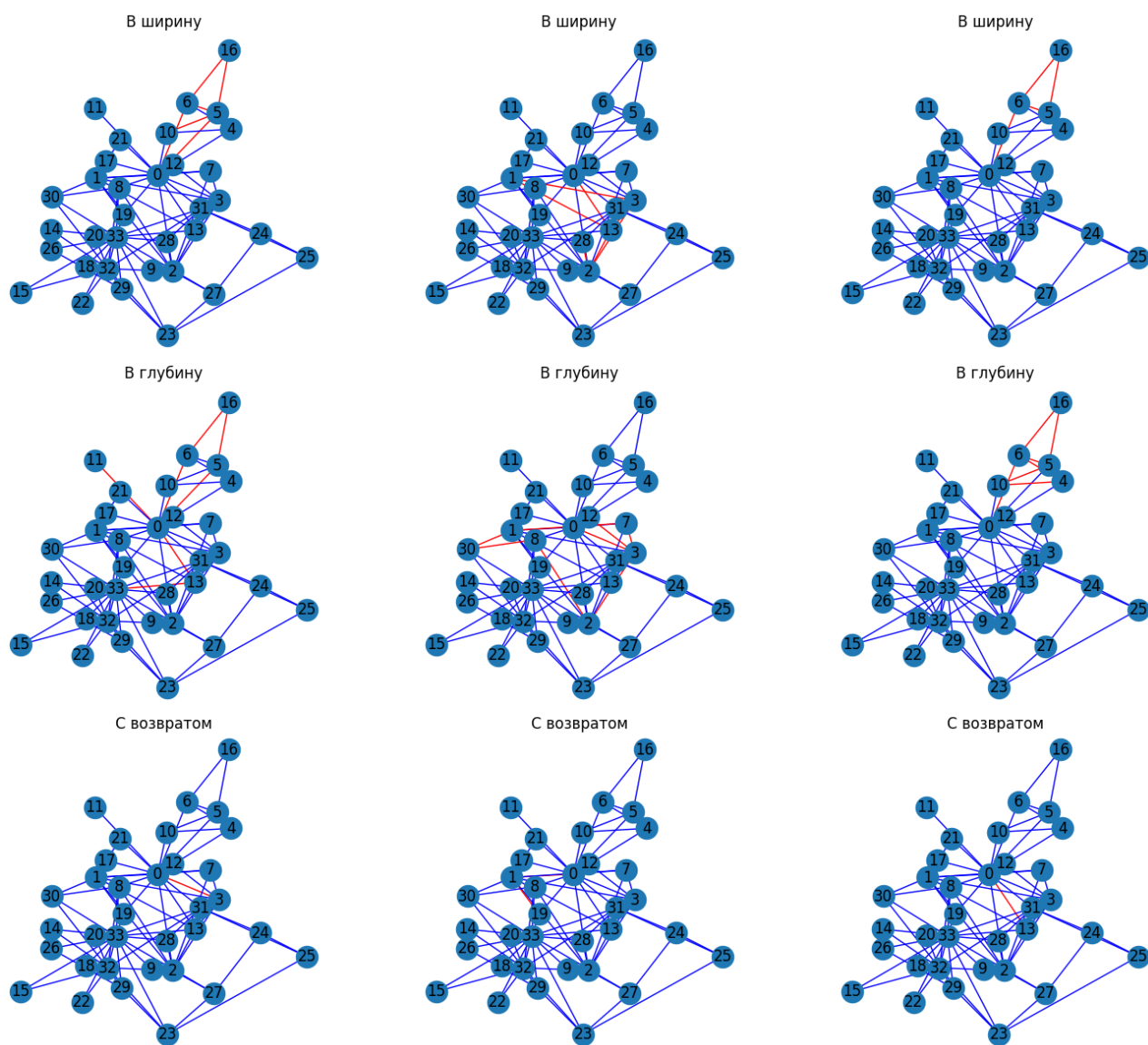



Рис.11.Контролируемые случайные блуждания по графу Karate Club. Красным цветом выделена траектория блуждания.

3.1.2.2.Реализация Node2vec

Архитектура модели - это обучаемая таблица эмбедингов (embedding layer). Для графа с N узлами и желаемой размерностью эмбединга E этот слой является матрицей размерности $(N \times E)$. Извлечение эмбединга принимает на вход индекс ноды u и возвращает соответствующую строку из этой матрицы Z_u - **эмбединг** ноды.

Обычно в решении задач на графах задачу рассматривают как последовательное включение энкодера и декодера, выполняющих различные функции (Рис.10).

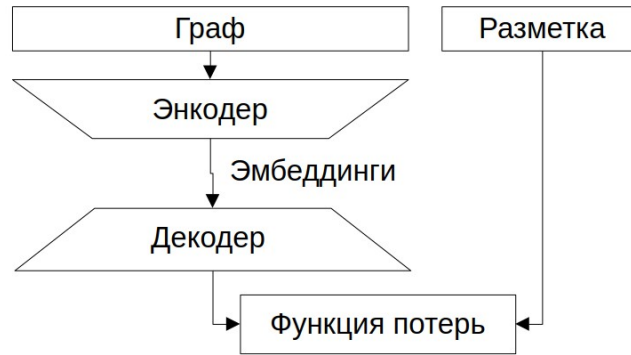


Рис.12. Базовая структура решения задач на графах методами машинного обучения с учителем.

Энкодер - это функция, которая строит векторное представление (эмбединг) вершины на основе её окружения, рассматривая эту вершину как центр. Энкодер отображает каждую вершину в точку (эмбединг) в многомерном пространстве.

Если у двух нод близкие эмбединги (векторы в пространстве близки по расстоянию), значит, они структурно похожи, и, следовательно, близки в графе.

Если эмбединги далеки - ноды не похожи и структурно удалены. Функция, которая оценивает, насколько близки или далеки две ноды по их эмбедингам, называется функцией похожести или **декодером**.

Один из часто используемых декодеров в задачах прогноза ребра (оценки близости нод) - это косинусная близость:

$$COSSIM(Z_u, Z_v) = \frac{Z_u Z_v^T}{\|Z_u\|_2 \cdot \|Z_v\|_2}$$

где Z_u, Z_v - эмбединги двух нод (векторы), $Z_u^T Z_v$ - скалярное произведение векторов, $\|Z_u\|_2, \|Z_v\|_2$ - L2-нормы (евклидовы длины) векторов. Ее лучшее значение достигается при максимальном значении 1.

Почему она косинусная? Потому что геометрически $Z_u Z_v^T = \|Z_u\|_2 \cdot \|Z_v\|_2 \cdot \cos(\widehat{Z_u, Z_v})$, и после нормировки на длины получаем: $COSSIM(Z_u, Z_v) = \cos(\widehat{Z_u, Z_v})$ - косинус угла между эмбедингами Z_u и Z_v .

Тогда:

- если $\cos(\widehat{Z_u, Z_v}) = 1$ - вектора сонаправлены ($\widehat{Z_u, Z_v} = 0^\circ$), эмбединги пропорциональны и ноды максимально близки;
- если $\cos(\widehat{Z_u, Z_v}) = 0$ - вектора перпендикулярны ($\widehat{Z_u, Z_v} = 90^\circ$) и ноды независимы/далеки.

Цель обучения сделать так, чтобы скалярное произведение эмбедингов соседних нод было максимальным. Формально это часто сводится к максимизации логарифма правдоподобия (log-likelihood):

$$L = \sum_{v \in G} \sum_{u \in N(v)} \log P(u|v)$$

где G - все ноды графа, $N(v)$ - окрестность v (например, все ноды в воке из v), $P(u|v)$ - вероятность того, что u близок к v .

Эта сумма - функция правдоподобия: чем выше, тем лучше модель объясняет наблюдаемые связи. Если инвертировать знак, получим **функцию потерь** - кросс-энтропию:

$$Loss = CE = - \sum_{v \in G} \sum_{u \in N(v)} \log P(u|v)$$

и задачей обучения будет минимизировать функцию потерь.

Вероятность перехода от ноды v к ноду u в классическом подходе к классификации вычисляется через эмбединги нод Z_u, Z_v и выглядит так:

$$P(Z_u|Z_v) = \frac{\exp(Z_u Z_v^T)}{\sum_{s \in G} \exp(Z_s Z_v^T)} = \text{Softmax}(Z_u Z_v^T)$$

Прямое вычисление softmax по всем узлам графа (знаменатель) является вычислительно очень дорогим для больших графов. Поэтому на практике используется метод **негативного сэмплирования** (negative sampling), учитывая в знаменателе не все возможные варианты, а только небольшое число (K) элементов суммы, не являющихся соседями заданной ноды:

$$P(Z_u|Z_v) = \frac{\exp(Z_u Z_v^T)}{\exp(Z_u Z_v^T) + \sum_{k=1, s_k \notin N(v)}^K \exp(Z_{s_k} Z_v^T)}$$

При обучении мы проходим по всем сгенерированным блужданиям. Для каждого узла в блуждании (рассматриваемого как центр) мы определяем его соседей внутри заданного окна (положительные примеры) и случайным образом выбираем несколько узлов, не являющихся его соседями в этом контексте (отрицательные примеры). Таким образом, для каждого центрального узла формируется один положительный и несколько (K) отрицательных пар. Все эти пары с метками (1 для положительных, 0 для отрицательных) составляют датасет.

При обучении используется **контрастная функция потерь** (contrastive loss): учитывающая негативное сэмплирование:

$$\text{Loss} \approx -\log(\sigma(Z_u Z_v^T)) + \sum_{k=1}^K \log(\sigma(Z_{s_k} Z_v^T))$$

где σ - сигмоидная функция (логистическая функция):

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$Z_u Z_v^T$ - скалярное произведение эмбедингов пары узлов (u, v); K - количество негативных сэмплов (s_k) на один положительный пример.

Идея контрастной функции потерь - максимизация вероятности для положительных пар и минимизировать вероятность для случайно выбранных отрицательных пар.

Какую задачу обычно решают, если нет разметки? Это - метод самообучения - решать универсальную задачу, не зависящую от конкретной цели - прогноз пропущенной ноды в последовательности случайного блуждания. Алгоритм 9 показывает пример кода, определяющего оптимальные эмбединги в смысле близости ноды.

Алгоритм 11. Обучение оптимальных эмбедингов с использованием контролируемых случайных блужданий по минимуму функции потерь

```
import torch
import torch.nn as nn
import torch.optim as optim

class Encoder(nn.Module):
    def __init__(self, num_nodes, embedding_dim):
        super().__init__()
        self.embeddings = nn.Embedding(num_nodes, embedding_dim)
    def forward(self, node_ids):
        return self.embeddings(node_ids)

def contrastive_loss(pos_scores, neg_scores):
    loss=0
    if len(pos_scores) > 0: loss += -torch.log(torch.sigmoid(pos_scores) + 1e-15).sum()
    if len(neg_scores) > 0: loss += torch.log(torch.sigmoid(neg_scores) + 1e-15).sum()
```

```

    return loss
def generate_training_pairs(nodes, walks, window_size=1, num_neg_samples=5):
    vocab_size, pairs = len(nodes), []
    for walk in walks:
        for i, center in enumerate(walk):
            start = max(0, i - window_size)
            end = min(len(walk), i + window_size + 1)
            for j in range(start, end):
                if i != j:
                    pos_ctx = walk[j]
                    pairs.append((center, pos_ctx, 1)) # Positive pair
            for _ in range(num_neg_samples):
                neg_ctx = random.randint(0, vocab_size - 1)
                while neg_ctx == center or neg_ctx == pos_ctx:
                    neg_ctx = random.randint(0, vocab_size - 1)
                pairs.append((center, neg_ctx, 0)) # Negative pair
    return window_size+num_neg_samples,pairs

device='cpu'
walks = generate_random_walks(G, num_walks=7, walk_length=5)
num_nodes = G.number_of_nodes()
nodes=list(G.nodes())
batch_size,pairs = generate_training_pairs(nodes, walks)
pairs = np.array(pairs) # (N, 3): [center, context, label]
centers = torch.tensor(pairs[:, 0], dtype=torch.long).to(device)
contexts = torch.tensor(pairs[:, 1], dtype=torch.long).to(device)
labels = torch.tensor(pairs[:, 2], dtype=torch.float).to(device)
embedding_dim,lr=2,1e-2
model = Encoder(num_nodes, embedding_dim).to(device)
optimizer = optim.SGD(model.parameters(), lr=lr)
model.train()
num_epochs=100
for epoch in range(num_epochs):
    total_loss = 0.0
    for i in range(0, len(pairs), batch_size):
        batch_centers = centers[i:i+batch_size]
        batch_contexts = contexts[i:i+batch_size]
        batch_labels = labels[i:i+batch_size]
        optimizer.zero_grad()
        center_embeds = model(batch_centers) # (B, d)
        context_embeds = model(batch_contexts) # (B, d)
        decoder = torch.sum(center_embeds * context_embeds, dim=1) # (B,)
        pos_scores = decoder[batch_labels == 1]
        neg_scores = decoder[batch_labels == 0]
        loss = contrastive_loss(pos_scores,neg_scores)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {total_loss:.4f}")

embs=model(torch.tensor(range(num_nodes),dtype=torch.int).view(-1,1)).detach().numpy()
plt.scatter(embs[:,0,0],embs[:,0,1])

```

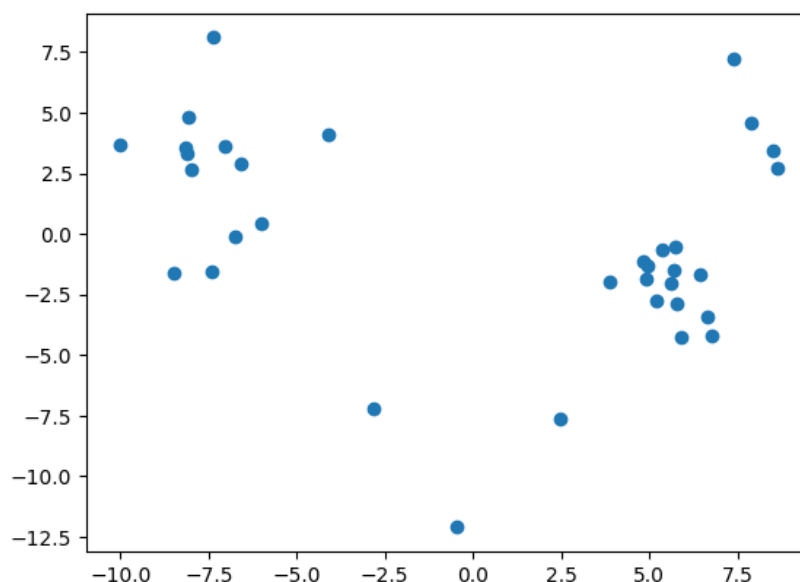


Рис.13. Распределение эмбеддингов нод графа Karate Club.

Из Рис.11 видно, что в графе можно уверенно выделить несколько (как минимум две) групп «близких» нод, похожих друг на друга в смысле векторных представлений.

Реализация Node2vec основана на алгоритме Word2vec [https://arxiv.org/abs/1301.3781], и основана на оптимизации расстояния между центральной нодой и суммой ее соседей в заданном окне анализа (контекстном окне). Этот алгоритм поиска эмбеддинга называется SkipGram.

Алгоритм 12. Обучение оптимальных эмбеддингов с использованием Node2vec

```
from node2vec import Node2Vec
node2vec = Node2Vec(G, dimensions=2, walk_length=10, num_walks=200, p=2, q=1, workers=1)
model = node2vec.fit(window=10, min_count=1, batch_words=4)
embs=model.wv[[str(x) for x in range(num_nodes)]]
plt.scatter(embs[:,0],embs[:,1])
```

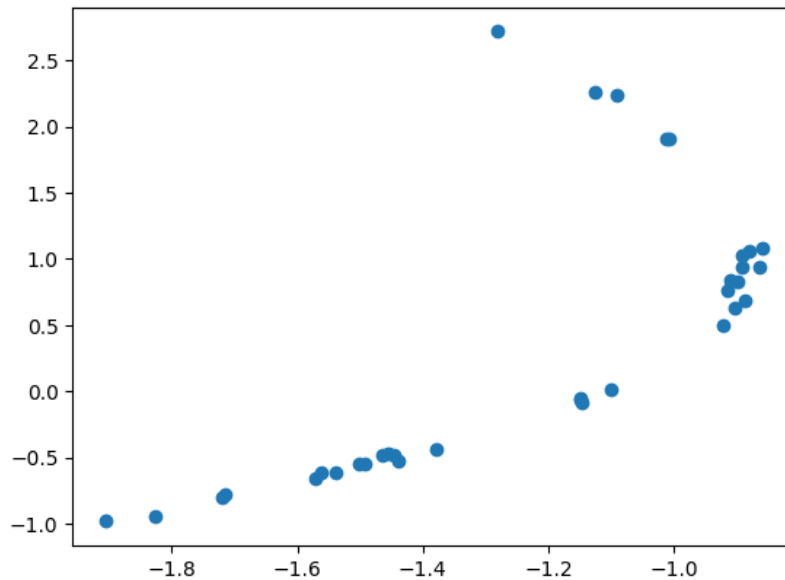


Рис.14. Распределение эмбедингов при их обучении алгоритмом Node2vec

3.1.3.Блуждания с телепортацией и PageRank

Можно рассматривать весь интернет как огромный направленный граф, где ноды - это веб-страницы (или сайты), а рёбра - это гиперссылки, переходы от одной страницы к другой. Такой граф является направленным графом.

Основная идея PageRank [Page, L. and Brin, S. and Motwani, R. and Winograd, T. (1999), The PageRank Citation Ranking: Bringing Order to the Web]:

- Важность (ранг, $PR(v)$) страницы v определяется тем, кто на неё ссылается;
- Чем важнее страницы, ссылающиеся на данную, тем важнее она сама:

$$PR(v) = \sum_{u \in N_{input}(v)} \frac{PR(u)}{deg_{output}(u)}$$

где $input$ соответствует входящим линкам, а $output$ - исходящим. То есть вклад каждой ссылки - это ее ранг исходной страницы, делённый на число её исходящих ссылок.

Это - рекурсивное определение, поскольку ранг каждой страницы определяется рангом всех страниц в интернете.

Это уравнение можно записать в матричной форме:

$$r_v = M_{uv} r_u$$

где r_v - ранг ноды v ; M_{uv} - матрица переходов между нодами:

$$M_{uv} = \begin{cases} 1/deg_{output}(u) & \text{if edge } u \rightarrow v; \\ 0 & \text{otherwise} \end{cases}$$

Как решить это уравнение? Фактически это - задача на собственные векторы и собственные значения: r - собственный вектор матрицы M , соответствующий ее собственному значению $\lambda=1$.

Но не у всех матриц M есть собственное значение 1, а даже если есть - решение не единственно (r определяется с точностью до постоянного множителя - масштаба).

Кроме того есть две практические проблемы:

Проблема 1: Паучьи ловушки» (spider traps). Это циклы (в т.ч. петли: страница ссылается на себя). При итеративном решении системы $r^{(t+1)} = M \cdot r^{(t)}$ ранг таких страниц неограниченно растёт и решение не сходится.

Проблема 2: тупики (dead ends). Это страницы, у которых нет исходящих ссылок. $deg_{output}(v) = 0$ - она не передаёт свой ранг дальше и поэтому ее ранг не участвует в расчетах рангов остальных страниц.

Рассмотрим направленный граф и ноду, из которой не исходит ни одного ребра. Например вы создали сайт, который ссылается на много разных сайтов, но никто не ссылается на ваш сайт. Если начать генерировать случайные блуждания, из этой ноды мы не сможем никуда перейти, и для блуждания это будет тупик (dead end).

Решение этой проблемы - введение параметра телепортации β . При генерации случайного блуждания мы с вероятностью β - идём на следующую ноду (идем по ссылке), а с вероятностью $1 - \beta$ - телепортируемся в случайную ноду графа (равновероятно любую из N его нод). Это моделирует поведение пользователя: пользователь ходит по ссылкам сайтов и вдруг выбирает случайный адрес в интернете.

Такое случайное блуждание называется случайным блужданием с телепортацией.

С введением вероятности телепортации становится возможным обойти тупики и формула PageRank становится:

$$PR(v) = \beta \sum_{u \in N_{input}(v)} \frac{PR(u)}{deg_{output}(u)} + (1 - \beta) \frac{1}{|N|}$$

где N - общее число нод в графе (страниц), β - обычно 0.8-0.9 (т.е. ~10% шагов - случайные телепортации).

В матричном виде уравнение примет вид:

$$r = \beta M \cdot r + (1 - \beta) / |N| \cdot 1$$

Где 1 - вектор из единиц $1 = (1, 1, \dots, 1)^T$.

Его можно переписать как:

$$r = G \cdot r$$

где $G = \beta M + (1 - \beta) / N \cdot E$ а E - матрица из всех единиц ($E_{ij} = 1$). Эта матрица G называется Google Matrix.

Она:

- стохастическая (сумма по столбцам = 1),
- имеет единственное стационарное распределение - искомый r .

Как вычислять решение r ? Можно итеративно по алгоритму:

1. $r^{(0)} = (1/|N|, 1/|N|, \dots, 1/|N|)^T$
2. $r^{(t+1)} = G \cdot r^{(t)}$
3. Повторять, пока не достигнута конверсия: $\|r^{(t+1)} - r^{(t)}\| < \epsilon$.

После многих итераций $r^{(t)}$ сходится к стационарному вектору - PageRank.

Вместо возведения G^t при можно решить линейную систему:

$$(I - \beta \cdot M) \cdot r = (1 - \beta) / |N| \cdot 1$$

По аналогии с вычислением индекса Каца это эквивалентно сумме геометрической прогрессии матриц, сводящейся к обращению матриц. Но на практике чаще используют итеративный метод, т.к. матрица M - разреженная (у страниц мало ссылок), и умножение $M \cdot r$ - быстрое.

Пример определения PageRank приведен в Алгоритме 11, а результат — на Рис.13.

Алгоритм 13. Определение PageRank для нод
<pre>d = nx.pagerank(G) s=[v*3000 for k,v in d.items()]</pre>

```
nx.draw(G,node_size=s, with_labels=True)
```

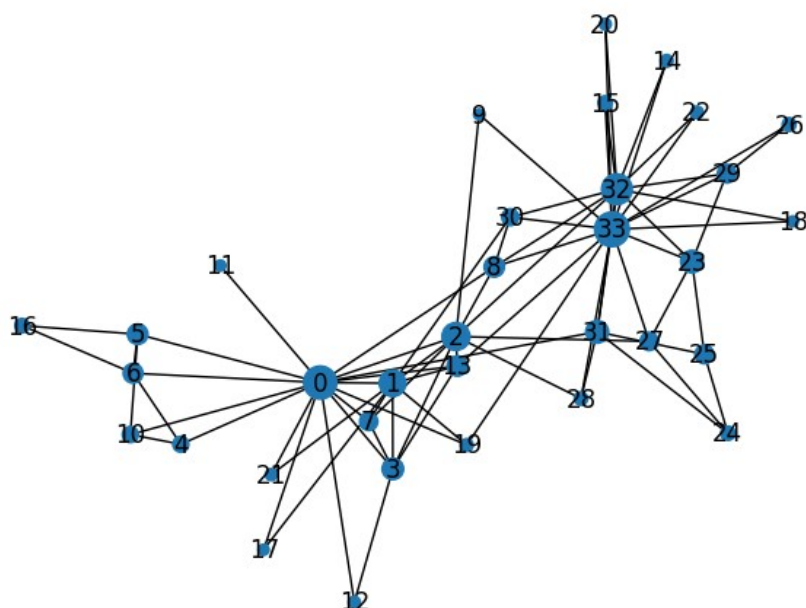


Рис.15. PageRank для нод графа Karate Club. Размер ноды пропорционален PageRank.

3.1.4.Блуждания с перезапуском и рекомендательные системы

Рассмотрим рекомендательную систему, где разные пользователи U покупают разные товары V . Стоит задача: сравнить близость двух товаров. Такая система может описывать двудольным графом: это граф, где вершины делятся на два множества U и V , и все рёбра идут только между U и V . U - пользователи, V - товары, ребро - пользователь купил товар.

При генерации блужданий возникает проблема: в двудольном графе можно застрять на одном из нодов, например на пользователе, купившем только один товар - дальше от такой ноды идти будет некуда.

Решение этой проблемы - случайное блуждание с перезапуском (Random Walk with Restart, RWR) - с вероятностью α (например, $\alpha = 0.1..0.3$) мы возвращаемся в стартовую ноду.

Пример траектории из товара V_0 :

$$V_0 \rightarrow U_5 \rightarrow V_3 \rightarrow U_4 \rightarrow [\text{restart}] \rightarrow V_0 \rightarrow U_7 \rightarrow V_4 \rightarrow \dots$$

Такой обход сочетает и обход в глубину (за счет длинных путей), и обход в ширину (много стартов с V_0). Чем больше α , тем шире обход - чаще возвращаемся к началу и исследуем новых соседей.

Для каждой ноды, посещённой в таких блужданиях, накапливается вес (сколько раз туда попали). Эти веса - оценка вероятности близости к стартовой ноду. Необходимо заметить, что обход возможен лишь в случае, когда граф ненаправленный. Таким образом, для построения рекомендательной системы необходимо перевести граф из направленного в ненаправленный.

Алгоритм 14. Рекомендательная система для фильмов на основе эмбедингов Node2vec, код адаптирован из [M.Labonne, Hands-On Graph Neural Networks Using Python//Packt, 2023]

```
from io import BytesIO
```



```

from urllib.request import urlopen
from zipfile import ZipFile
import pandas as pd
from collections import defaultdict
url = 'https://files.grouplens.org/datasets/movielens/ml-100k.zip'
zurl=urlopen(url)
zfile=ZipFile(BytesIO(zurl.read()))
zfile.extractall('.')
ratings = pd.read_csv('ml-100k/u.data', sep='\t', names=['user_id', 'movie_id', 'rating', 'unix_timestamp'])
movies = pd.read_csv('ml-100k/u.item', sep='|', usecols=range(2), names=['movie_id', 'title'])
ratings = ratings[ratings.rating >= 4]
pairs = defaultdict(int)
for group in ratings.groupby("user_id"):
    user_movies = list(group[1]["movie_id"])
    for i in range(len(user_movies)):
        for j in range(i+1, len(user_movies)):
            pairs[(user_movies[i], user_movies[j])] += 1
G = nx.Graph()
for pair in pairs:
    movie1, movie2 = pair
    score = pairs[pair]
    if score >= 16:
        G.add_edge(movie1, movie2, weight=score)
node2vec = Node2Vec(G, dimensions=64, walk_length=6, num_walks=1000, p=2, q=1, workers=1)
model = node2vec.fit(window=10, min_count=1, batch_words=4)
def recommend(movie):
    movie_id = str(movies[movies.title == movie].movie_id.values[0])
    for id in model.wv.most_similar(movie_id)[:5]:
        title = movies[movies.movie_id == int(id[0])].title.values[0]
        print(f'{title}: {id[1]:.2f}')
    return movie_id
recommend("Toy Story (1995)")

```

```

Return of the Jedi (1983): 0.45
Independence Day (ID4) (1996): 0.45
Rock, The (1996): 0.44
Star Trek: First Contact (1996): 0.42
Raising Arizona (1987): 0.39

```

3.1.5.Обобщение PageRank

PageRank - может рассматриваться не как один алгоритм, а как семейство алгоритмов, различающихся политикой телепортации. Общий вид уравнения:

$$r_v = \beta M_{uv} \cdot r_u + (1 - \beta) T_v$$

где M - матрица переходов, β - вероятность идти по ссылке, T_v - вектор телепортации, определяющий вероятность телепортации в ноду v . Разные T соответствуют разным алгоритмам:

1. Оригинальный PageRank: $T_v = 1/|N|$ для всех v . Телепортация выполняется равновероятно в любую ноду графа.
2. Персонализированный PageRank (PPR): T - распределение, персонализированное под пользователя, телепортация происходит в ноды, с вероятностями, зависящими от предпочтений пользователя. Например, если пользователь любит комедии - выше вес у комедий. T_v = доля рейтингов пользователя, приходящаяся на товар v .
3. Тематически-ориентированный (topic-specific) PageRank: T фиксирован для темы (например, «музыка», «инструменты»), не под конкретного пользователя, а под класс

товаров. Телепортация происходит в ноды той-же тематики, что и заданная исходно тема.

4. Блуждание с рестартом (RWR) - частный случай PPR: телепортация происходит всегда в начальную ноду блуждания, $T_v = \delta_{u_0, v}$ (1 в позиции u_0 , 0 в остальных позициях).

Таким образом, изменяя матрицу телепортаций T можно контролировать вид возврата и, тем самым, какие ноды считаются близкими.

3.2. Эмбединги как факторизация матрицы смежности

Ранее задача нахождения оптимального эмбединга формулировалась, как нахождение таких Z_u, Z_v , что скалярное произведение $Z_u Z_v^T$ максимально, когда u и v находятся в одном случайном блуждании и минимально, когда u и v не находятся в одном случайном блуждании.

Можно сформулировать задачу максимально просто, наиболее короткими блужданиями длины 1:

- $Z_u Z_v^T = 1$ если u и v соединены ребром;
- $Z_u Z_v^T = 0$ если u и v не соединены ребром.

В матричном виде для невзвешанного графа с матрицей смежности A и таблицы эмбедингов Z_{ij} (по вертикали номер ноды, по горизонтали - эмбединги нод) эта формулировка примет вид:

$$A \approx Z \cdot Z^T$$

Это - аппроксимация матрицы смежности скалярным произведением эмбедингов. Математически это разложение называется матричной факторизацией, или представлением матрицы в виде произведения других матриц.

Задачей факторизации будет подобрать таблицу эмбедингов Z_{ij} так, чтобы отличие было минимально. Самый простой критерий - поиск минимума нормы Фробениуса:

$$\|A - Z \cdot Z^T\|_F^2 = \sum_{i,j} (A_{ij} - (Z \cdot Z^T)_{ij})^2 \rightarrow \min$$

Рассмотрим способы такой факторизации, позволяющие получить оптимальные (с точки зрения предсказания ребра между нодами) эмбединги.

3.2.1. Разложение Холецкого

Для ненаправленных графов можно использовать разложение Холецкого (Cholesky). Разложение Холецкого (или метод квадратного корня) - это представление симметричной положительно определённой матрицы A в виде $A = L L^T$, где L - нижняя треугольная матрица со строго положительными элементами на диагонали. Разложение Холецкого всегда существует и единственно для любой симметричной положительно определённой матрицы.

У матриц смежности графов обычно много нулей и матрица не положительно определена, поэтому метод Холецкого не подходит напрямую. Простой способ обхода этого ограничения - добавить $\varepsilon > |\min(\lambda_i)|$ ко диагональным элементам матрицы (где λ_i — отрицательные собственные значения матрицы):

$$A_\varepsilon = A + \varepsilon I$$

где I — единичная матрица.

Либо можно выбрать ε достаточно большим, например $\varepsilon \sim N$, где N число нод в графе (размер матрицы A). Тогда A_ε станет положительно определённой и можно будет применить разложение Холецкого (Алгоритм 13 и Рис.14?).

Алгоритм 15. Представление эмбедингов нод графа Karate Club через разложение Холецкого

```
from scipy.linalg import cholesky
G = nx.karate_club_graph()
A = nx.adjacency_matrix(G).toarray().astype(float)
lsh=A.shape[0]
A_spd = A + lsh * np.eye(A.shape[0])
L = cholesky(A_spd, lower=True)
plt.imshow((L-L.min())/(L.max()-L.min()),cmap='gray_r')
plt.ylabel('u')
plt.xlabel('$Z_u$')
```

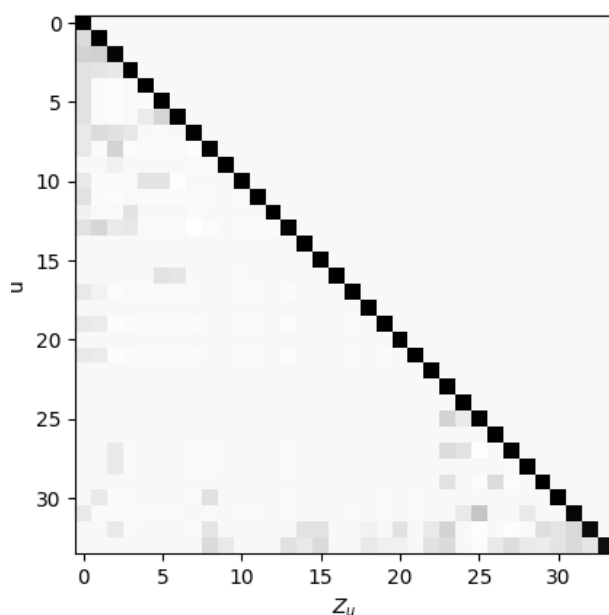


Рис.16. Результат представления эмбедингов нод графа Karate Club через разложение Холецкого

3.2.2. Неотрицательная матричная факторизация

Недостатком разложения Холецкого является высокая размерность получаемых эмбедингов нод, равная количеству нод в графе. На больших графах это приводит к существенной требовательности к ресурсам, и неудобно для применения. Поэтому удобнее применять матричные разложения в произведение матриц существенно меньших размеров, чем оригинальная матрица смежности.

Поскольку обычно матрица смежности в невзвешанном графе неотрицательна ($A_{ij} \geq 0$) можно использовать для решения задачи факторизации неотрицательно матричное разложение (NMF):

$$A \approx W \cdot H^T, \text{ где } W_{ij} \geq 0, H_{ij} \geq 0$$

Обе матрицы W, H - неотрицательные. Разложение известно, существуют алгоритмы для его поиска, и размерность матриц W, H можно контролировать.

Такое представление удобно для рекомендаций: оценки положительные либо нулевые. Кроме того, результат можно интерпретировать для двудольных графов: W - эмбединги пользователей, H - эмбединги товаров.

Однако $W \cdot W^T$ или $H \cdot H^T$ не обязаны быть близки к A . Поэтому если граф не двудольный и нужны взаимные сходства между любыми узлами графа NMF не даёт их напрямую. То есть для произвольного графа такой метод обычно неудобен, хотя и возможен.

Алгоритм 16. Представление эмбедингов нод графа Karate Club через неотрицательную матричную факторизацию с размерностью эмбедингов 10.

```
from sklearn.decomposition import NMF
W=NMF(10).fit_transform(A)
plt.imshow(((W-W.min())/(W.max()-W.min())).T,cmap='gray_r')
plt.xlabel('u')
plt.ylabel('$Z_u$');
```

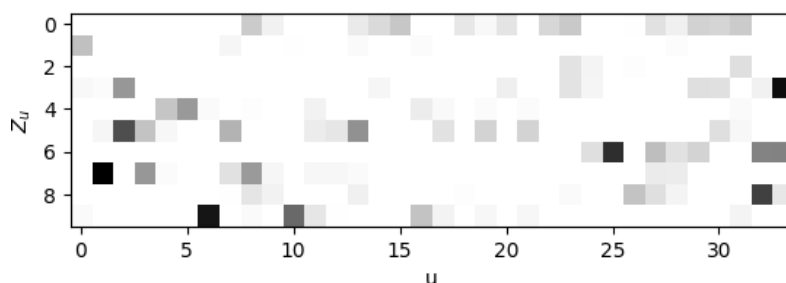


Рис.17. Результат представления эмбедингов нод графа Karate Club через неотрицательную матричную факторизацию с размерностью эмбедингов 10.

3.2.3. Поиск разложением по собственным векторам

Другой способ поиска эмбединга, который в квадрате даёт матрицу смежности, основан на разложении по собственным векторам (Spectral embedding).

Рассмотрим ненаправленный граф с матрицей смежности A . Эта матрица симметрична ($A = A^T$) и можно найти её спектральное разложение:

$$A = Q \cdot \Lambda \cdot Q^T$$

где Q - ортогональная матрица собственных векторов, а $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_N)$ - диагональная матрица собственных значений.

Если все λ_i неотрицательны, то можно найти матрицу Z , такую что $A = Z \cdot Z^T$:

$$Z = Q \cdot \Lambda^{1/2}$$

В общем случае произвольных λ_i можно заменить отрицательные λ_i на 0:

$$\Lambda^{1/2} = \text{diag}(\sqrt{\max(\lambda_1, 0)}, \dots, \sqrt{\max(\lambda_N, 0)})$$

Этот метод устойчивее метода Холецкого для разреженных матриц.

Очевидно что часть информации о нодах однако теряется при занулении отрицательных собственных векторов (сокращая в тоже время размерность эмбединга). Алгоритм 15 представляет вариант такого разложения, а на рис.16 приведен результат разложения.

Алгоритм 17. Представление эмбедингов нод графа Karate Club через разложение по собственным векторам.

```
from scipy.linalg import eigh
eigenvalues, eigenvectors = eigh(A)
eigenvalues = np.maximum(eigenvalues, 0)
sqrt_eigenvalues = np.sqrt(eigenvalues)
L = eigenvectors @ np.diag(sqrt_eigenvalues)
L=L[:,L.sum(axis=0)>0]
plt.imshow((((L-L.min())/(L.max()-L.min()))).T,cmap='gray_r')
plt.xlabel('u')
plt.ylabel('$Z_u$');
```

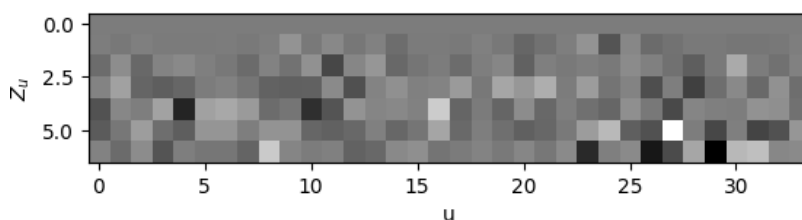


Рис.18. Результат представления эмбедингов нод графа Karate Club через разложение по собственным векторам.

3.2.4. Проблемы факторизации

У факторизации существуют проблемы, которые мешают применять ее в общем случае:

1. Новые ноды (cold start). При появлении нового пользователя/товара матрица A увеличивается на строку/столбец и придётся полностью пересчитывать матрицы эмбединга, а это часто ресурсоемко.
2. Структурное подобие (structural equivalence). Если ноды u и v структурно эквивалентны - их окрестности идентичны (например, за счет симметрии в графе, то их эмбединги должны быть равны. Однако тогда в A можно поменять местами строки u и v и столбцы u и v , и A изменится. Однако разложение должно остаться тем же, поэтому разложение часто неустойчиво.
3. Общая чувствительность разложения к перестановкам строк/столбцов. Перестановка двух любых узлов - это перестановка соответствующих строк и столбцов в матрице смежности. Но большинство разложений не инвариантны к таким перестановкам и эмбединги меняются, хотя граф остаётся тем же.
4. Сложность учёта собственных признаков нод (features). У нод есть дополнительные атрибуты: возраст, пол, жанр фильма и т.д., а факторизация использует только связи между нодами, игнорируя их внутренние признаки. Поэтому в общем случае, когда необходимо будет учитывать и топологические и собственные признаки нод, факторизация усложнит решение задачи.

3.3. Эмбединг для графа и анонимные блуждания

Рассмотрим кодировку всего графа (graph-level embedding). Для создания устойчивого эмбединга желательно, чтобы два графа, которые отличаются лишь перестановкой номеров

нод, имели одинаковый эмбединг. У обычных случайных блужданий это проблема: эмбединги нод зависят от их нумерации и неустойчивы.

Для создания топологического эмбединга для графа необходимо использовать другие подходы.

Решение 1: суммирование эмбедингов.

Эмбединг графа это сумма эмбедингов его нод.

$$Z_G = \sum_{v \in G} Z_v$$

Необходимо заметить, что перестановка номеров нод может изменить эмбединг каждой ноды, поэтому эмбединг графа может быть неустойчив к перестановке номеров нод.

Решение 2: виртуальный узел (virtual node).

Добавляем новую ноду V^{virt} , соединяем её со всеми исходными нодами, обучаем эмбединг на расширенном графе. Найденный эмбединг виртуальной ноды $e_{V^{virt}}$ - это и есть эмбединг всего графа.

Решение 3: анонимные блуждания (anonymous walks).

Идея метода - кодировать структуру обхода, а не номер ноды. Кодировка должна позволить отличать одни ноды от других, возникающие в процессе блуждания.

Алгоритм имеет вид:

1. Создаём пустую хэш-таблицу HASH.
2. Первую ноду u_0 кодируем как 0, добавляем в HASH: $HASH[u_0]=0$.
3. При переходе в новую ноду u :
 - если $u \notin HASH$, присваиваем этой ноде следующий незаполненный номер: $HASH[u] = \text{len}(HASH)$,
 - иначе - берём уже существующий номер.
4. Кодировем случайное блуждание последовательностью номеров из HASH.

В результате алгоритма (Алгоритм 13) все обходы, одинаковые по структуре, получают один и тот же код, и алгоритм устойчив к перестановке номеров нод в графе.

Например:

- Номер нод в блуждании: [30, 8, 7, 7, 2]
- Таблица хэшей: $HASH=\{30:0, 8:1, 7:2, 2:3\}$
- Код анонимного блуждания: [0, 1, 2, 2, 3].

Эмбединг графа в этом случае это гистограмма частот анонимных воков фиксированной длины (Алгоритм 14, Рис.14).

Число уникальных анонимных воков растёт экспоненциально, поэтому для описания больших графов достаточно коротких анонимных воков.

Алгоритм 18. Случайное анонимное блуждание по графу Karate Club и его сравнение с контролируемым случайным блужданием.

```
import numpy as np
def random_anonymous_walk(G, start, length, p, q):
    walk = [0]
    encoder, enc_max = {}, 0
    encoder[start] = 0
    for i in range(length):
        current = walk[-1]
        previous = walk[-2] if len(walk) > 1 else walk[0]
        next = next_node(G, previous, current, p, q)
        if next not in encoder:
            enc_max += 1
            encoder[next] = enc_max
        walk.append(encoder[next])
    return [int(x) for x in walk], encoder
```

```
G=nx.karate_club_graph()
print(random_walk(G,30, 10, 3, 2))
print(random_anonymous_walk(G,30, 10, 3, 2)[0])
```

```
[30, 8, 33, 9, 33, 32, 20, 33, 9, 33, 19]
[0, 1, 2, 1, 2, 3, 4, 3, 5, 6, 7]
```

Алгоритм 19. Построение гистограммы анонимных блужданий длины 3 для графа Karate Club

```
plt.figure(figsize=(4,3))
aw=[]
for i in range(300):
    aw.append(str(random_anonymous_walk(G,np.random.randint(34), 2, 3, 2)[0]))
plt.hist(aw,bins=20)
plt.xlabel('a.walk code')
plt.ylabel('# of cases')
```

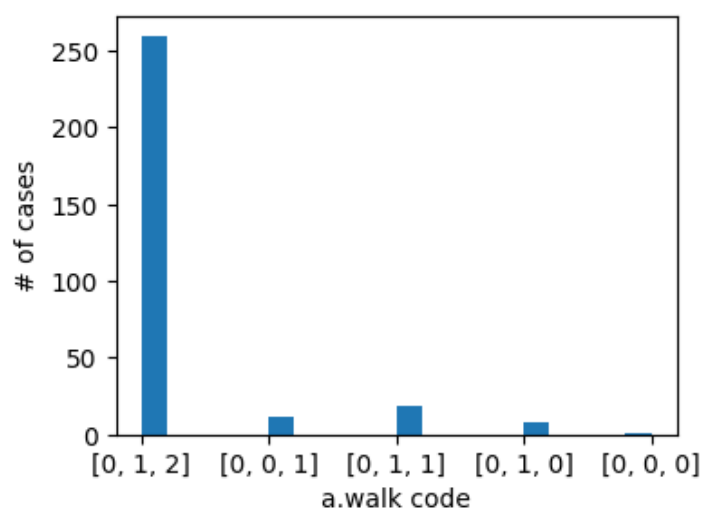


Рис.19. Гистограмма анонимных блужданий длины 3 для графа Karate Club

4. Учет внутренних признаков нод: передача сообщений в графах

4.1. Бинарная классификация нод

Система, построенная только на топологической информации о нодах не учитывает остальные (внутренние) признаки этих нод. Рассмотрим способы учета всех признаков нод, и топологическую информацию (кто с кем связан), и внутреннюю информацию (свойства каждой ноды).

Начинаем с задачи классификации нод. Предположим, мы хотим предсказать бинарное свойство: купит/не купит определённый товар на основе уже известного графа.

Граф, с которым будем работать - это проекция двудольного графа на товары:

- исходный граф состоит из двух долей - пользователи и товары.
- проекция: два товара соединены ребром (похожи), если их купил одновременно хотя бы один пользователь.

Задача: пользователь купил товар u , определить, купит-ли он товар v ? Задача сводится к определению похожести товаров u и v .

Все товары (ноды) можно разделить на три группы: «купит» (про которые известно что их купили), «не купит» (про которые известно что их не купили), и «неизвестно». В задаче необходимо предсказать метку нод с состоянием «неизвестно».

С чем связано сходство нод? Выделяют два ключевых принципа:

1. Гомофилия (homophily): “Рыбак рыбака видит издалека”. Ноды, имеющие похожие признаки, склонны к образованию связей в графе. Например все студенты 2-го курса сидят на одной паре потому что являются студентами одного потока.
2. Влияние (influence): “С волками жить - по-волчьи выть”. Связанные ноды в графе обычно похожи между собой. Например: даже «совы» начинают вставать рано, если учатся в университете с утра.

Оба механизма влияют на то, какую метку получит нода.

4.2. Передача сообщений и агрегация

Передача сообщений и агрегация: две ключевые операции в графовых нейронных сетях.

Передача сообщений (Message passing) - это операция, при которой состояние (признаки) одной ноды передаётся её соседям по рёбрам графа. То есть нода отправляет свою информацию тем, с кем она связана.

Агрегация (Aggregation) - это операция, при которой нода собирает пришедшие от соседей сообщения и формирует на их основе новые признаки. Агрегация нужна, потому что исходные признаки (фичи) редко бывают оптимальными для задачи. Через агрегацию мы трансформируем признаки, чтобы получить более информативные представления — эмбединги. Часто агрегацию можно разделить на две операции: объединение информации о соседях и формирование новых признаков.

Существует две основные архитектурные схемы, различающихся очерёдностью:

1. Сначала message passing, потом агрегация:

- сначала каждая нода получает признаки от соседей (передача);
- затем объединяет их в новое представление (агрегация).

Пример: Графовая свёрточная сеть (GCN).

2. Сначала агрегация, потом message passing.

- сначала для каждой ноды формируется агрегированное представление её окружения;
- затем это представление используется при передаче информации дальше.

Примеры: GAT (Graph Attention Network), GraphSAGE.

Передача сообщений (Message passing) - это методы передачи информации о нодах между нодами. Рассмотрим, как узлы могут понимать состояние друг друга в графе.

Узел 1 имеет собственное представление о себе - это то, как он выглядит с его собственной точки зрения $F_{1,1}$;

Узел 2 тоже формирует своё представление об узле 1 - то, как узел 1 выглядит с точки зрения узла 2 $F_{1,2}$.

Узел 1 комбинирует:

- свою внутреннюю информацию о себе;
- информацию о себе, полученную от других;
- информацию о состоянии других, полученную от других;
- передаёт обновлённое представление о себе дальше;
- передаёт обновлённое представление о других дальше.

Этот процесс продолжается до конверсии, и в результате все узлы формируют представления обо всех узлах и о себе самих.

При передаче сообщений может задействоваться один или несколько механизмов. Этот процесс называется передача сообщений (message passing) - и, в более продвинутой форме, передача доверия (belief propagation).

4.3. Коллективный классификатор

Идея алгоритма: если у некоторых нод метки известны - можно распространить их по графу, усредняя по соседям. Тогда ноды, у которых будет больше соседей с меткой А будут иметь метку А, а у которых больше соседей с меткой В - будут иметь метку В. Таким образом, для нод с неизвестными метками по их соседям определяется вероятность того, что они имеют искомый класс.

Алгоритм состоит из трех этапов:

1. Локальный классификатор - предсказывает метку, используя только внутренние признаки ноды, без учёта топологической информации.
2. Классификатор отношений - предсказывает метку ноды, используя метки (известные или предсказанные) и признаки ее соседей, т.е. учитывает структуру.
3. Коллективные вычисления - итеративное распространение меток по графу до сходимости.

Рассмотрим его составляющие.

Алгоритм 20. Генерация пробного графа
<pre>G=nx.Graph() G.add_nodes_from([(0,{ 'Y':1,'train':1,'F':0}),(1,{ 'Y':1,'train':1,'F':1}),(2,{ 'Y':1,'train':1,'F':1}), (3,{ 'Y':0,'train':1,'F':1}),(4,{ 'Y':0,'train':1,'F':0}),(5,{ 'Y':0,'train':1,'F':0}), (6,{ 'Y':-1,'train':0,'F':0}),(7,{ 'Y':-1,'train':0,'F':1}),(8,{ 'Y':-1,'train':0,'F':1}),]) G.add_edges_from([(0,1),(0,2),(1,2),(3,4),(3,5),(3,6),(1,5),(1,6),(1,7),(4,8)])</pre>

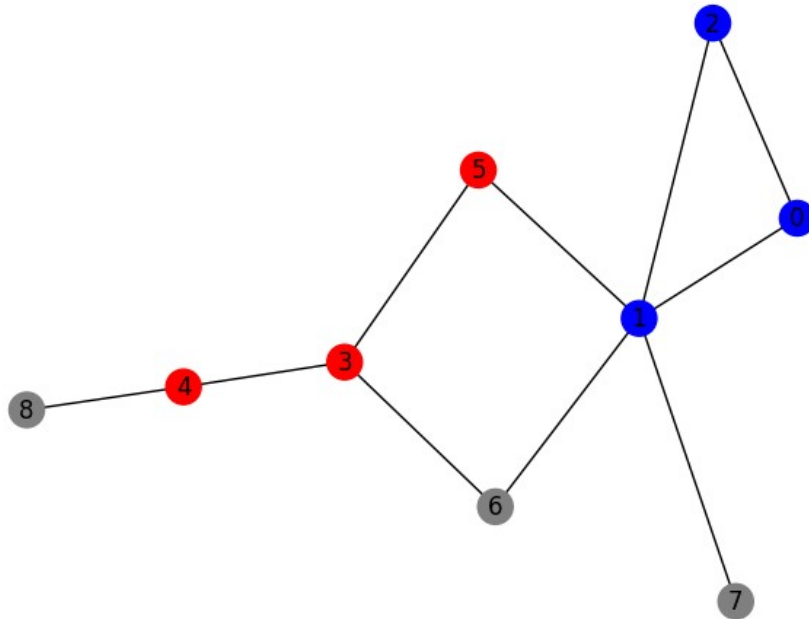


Рис.20. Пробный граф. Красные и синие ноды — ноды с известными метками. Серые ноды - с неизвестными метками.

4.3.1. Локальный классификатор

Он нужен если у ноды нет известных меток среди соседей (или самих соседей), но есть внутренние признаки (например, пол). Годится для прогноза состояний новых нод в графе, для которых еще неизвестны их связи с другими нодами.

В этом случае мы строим обычный классификатор (нейронную сеть, дерево решений и т.п.), где:

- вход - вектор внутренних признаков ноды;
- выход - метка ноды.

Обучается такой классификатор на нодах графа, для которых известны метки. Как обучаем:

1. Проходим по всем нодам с известной меткой;

2. Собираем обучающий датасет:

- вход: вектор фич ноды (возраст, пол) X_i ;
- выход: метка y_i , которую мы предсказываем (1 - купит, 0 - не купит).

Получаем множество пар: $[(X_1, y_1), (X_2, y_2), \dots, (X_k, y_k)]$. Разбиваем это множество на обучающий, валидационный и тестовый датасеты и тренируем классификатор, например решающее дерево.

Алгоритм 21. Обучение локального классификатора и отображение прогнозов

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
x1,y1=[],[]
for n in G.nodes():
    if G.nodes[n]['train']==1:
        x1.append(G.nodes[n]['F'])
        y1.append(G.nodes[n]['Y'])
x1=np.array(x1)[:,:np.newaxis]
y1=np.array(y1)
def f1(x1,y1):
    clf=DecisionTreeClassifier(max_depth=1)
    clf.fit(x1,y1)
    return clf
clf1=f1(x1,y1)
ypred=[]
for node,attr in G.nodes.items():
    if G.nodes[node]['train']==1:
        ypred.append(G.nodes[node]['Y'])
    else:
        ypred.append(int(clf1.predict([[G.nodes[node]['F']]])[0]))
pos=nx.kamada_kawai_layout(G)
cols=np.array(['red','blue'])
ypred=np.array(ypred)
nx.draw(G, pos,node_color=cols[ypred], with_labels=True)
```

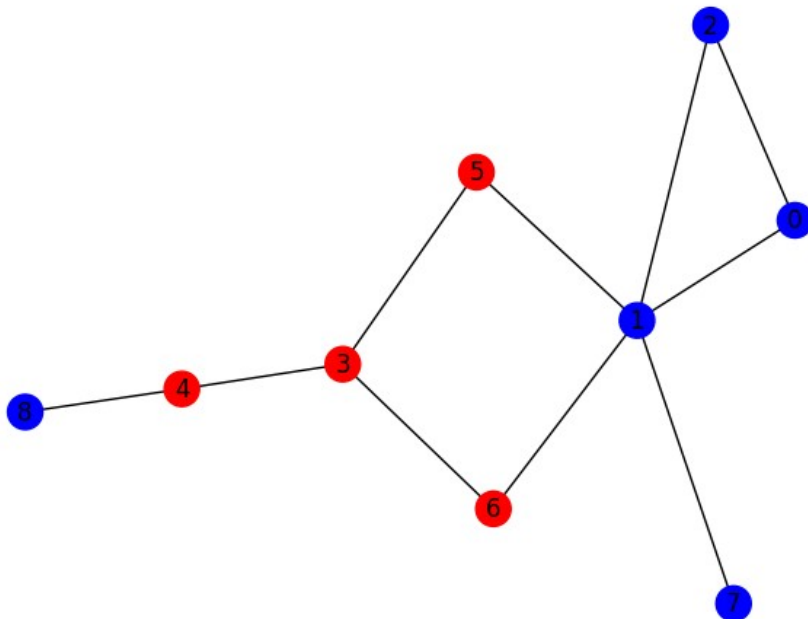


Рис.21. Результаты прогноза неизвестных меток локальным классификатором.

4.3.2.Классификатор отношений (связный классификатор)

Этот классификатор предсказывает метку ноды u , используя:

- метки соседей $v \in N(u)$,
- признаки соседей,
- факт наличия ребра (т.е. сходства по поведению).

Классификатор учитывает структуру графа. Только связанные соседи участвуют в прогнозе.

В локальном классификаторе мы использовали только внутренние признаки. В классификаторе отношений мы учитываем отношения между узлами и оцениваем вероятность того, что узел находится в состоянии 1:

$$P(y_u) = \frac{1}{|N(u)|} \sum_{v \in N(u)} \begin{cases} y_v, & \text{if } v \in \text{Train} \\ F_{\text{local}}(v), & \text{if } v \notin \text{Train} \end{cases}$$

где $F_{\text{local}}(u)$ - выход локального классификатора для узла u , предсказывающий метку по признакам, $\alpha \in [0..1]$. После чего можно выбрать метку для узла:

$$y_u = \begin{cases} 1, & \text{if } P(y_v) > 0.5 \\ 0, & \text{else} \end{cases}$$

Так система учитывает и признаки, и связи. Вместо арифметического усреднения можно использовать геометрическое усреднение. Эти операции устойчивы к перестановкам узлов и изменению их количества.

Проблема: почему нельзя использовать обычную нейросеть для агрегации соседей? Здесь две причины:

1. Переменное число соседей у узла. Нейронная сеть требует фиксированного числа входов.
2. Инвариантность к перестановке. Граф не меняется, если переставить номера узлов местами (изоморфизм). А результат агрегации должен быть одинаковым при любой перестановке соседей.

Какие операции удовлетворяют обоим условиям и могут использоваться для агрегации (объединения) информации о соседях? Только те, что инвариантны к перестановкам аргументов и работают с переменным количеством аргументов.

Самые простые операции это - сумма и произведение фич (а также их нормированные версии - арифметическое и геометрическое среднее). Именно среднее по соседям и используется при агрегации соседей.

Алгоритм 22. Обучение классификатора отношений

```
x2,y2=[],[]
for n in G.nodes():
    zacc=[]
    for n_n in G.neighbors(n):
        if G.nodes[n_n]['train']==1: zacc.append(G.nodes[n_n]['Y'])
        else: zacc.append(int(clf1.predict([[G.nodes[n_n]['F']]])[0]))
    zm=np.array(zacc)
    G.nodes[n]['F2']=float(zm.sum()/(zm.shape[0]+1e-2))
for n in G.nodes():
    if(G.nodes[n]['train']==1):
        x2.append([G.nodes[n]['F2']])
        y2.append(G.nodes[n]['Y'])

x2,y2=np.array(x2),np.array(y2)
def f2(x2,y2):
    clf=DecisionTreeClassifier(max_depth=1)
    clf.fit(x2,y2)
    return clf
clf2=f2(x2,y2)
ypred=[]
for node,attr in G.nodes.items():
```

```

if G.nodes[node]['train']==1: ypred.append(G.nodes[node]['Y'])
else: ypred.append(int(clf2.predict([[G.nodes[node]['F2']]])(0)))
pos=nx.kamada_kawai_layout(G)
cols=np.array(['red','blue'])
ypred=np.array(ypred)
nx.draw(G, pos,node_color=cols[ypred], with_labels=True)

```

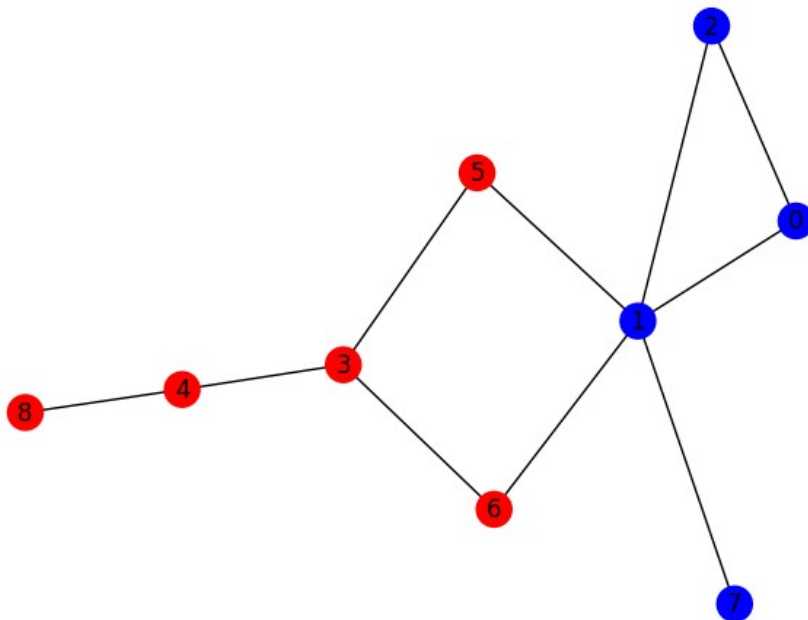


Рис.22. Результат работы классификатора отношений

4.3.3. Коллективные вычисления

Используются для распространения меток по графу. Это итеративный процесс:

1. Для бинарной классификации инициализируем все ноды с неизвестным состоянием значением 0.5 (50% вероятность состояния 1) или прогнозным значением локального классификатора $F_{local}(u)$.
2. В случайном порядке выбираем ноду u ,
3. Если u - не тренировочная (метка ее неизвестна), то обновляем её метку как среднее по соседям:

$$y_u = \frac{1}{|N(u)|} \sum_{v \in N(u)} y_v$$

4. Повторяем шаги 2-3 до тех пор, пока не достигнута конверсия - все метки не стабилизируются.

По результату первой итерации можно составить новый обучающий набор (предсказанные метки для ранее неразмеченных) и дообучить локальный классификатор. Часто достигается точность ~ 1.0 даже на малых наборах данных.

Алгоритм 23. Проведение коллективных вычислений

```

G1=G.copy()
for itt in range(1000):
    for n in G1.nodes():
        if G1.nodes[n]['train']!=1:
            G1.nodes[n]['Y']=int(clf2.predict([[G1.nodes[n]['F2']]])[0])
    for n in G1.nodes():
        zacc=[]
        for n_n in G1.neighbors(n):
            zacc.append(G1.nodes[n_n]['Y'])
        zm=np.array(zacc)
        G1.nodes[n]['F2']=np.round(float(zm.sum()/(zm.shape[0]+1e-2)),4)
c = []
for node,attr in G1.nodes.items():
    c.append(cols[attr['Y']])
labeldict = {}
for node,attr in G1.nodes.items():
    labeldict[node] = attr['F2']
pos=nx.kamada_kawai_layout(G1)
nx.draw(G1, pos,node_color="gray", labels=labeldict, with_labels=True)

```

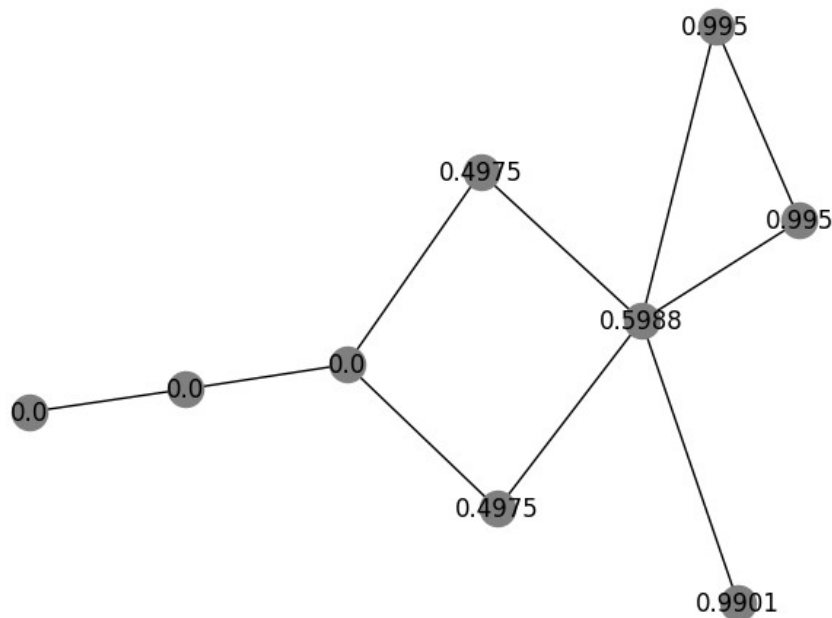


Рис.23. Результат работы коллективных вычислений. Число в ноде — вероятность синего класса

4.3.4.Обобщение на взвешенные графы

Взвешенный граф - это граф, где у рёбер есть веса и матрица A составлена из неотрицательных чисел: $A_{uv} \geq 0$. При этом $A_{uv} = 0$ если связи между нодами u, v нет.

Тогда обновление метки - взвешенное среднее:

$$y_u \leftarrow \frac{\sum_{v \in N(u)} A_{uv} \cdot y_v}{\sum_{v \in N(u)} A_{uv}}$$

Недостаток метода: сходимость не гарантирована - особенно при наличии циклов в графе

4.4. Распространение доверия

Алгоритмы распространения доверия [https://arxiv.org/abs/2007.00295] позволяют обеспечить сходимость и конверсию на вероятностном языке. В алгоритме используются:

- Сообщения $\mu_{v \rightarrow u}(x_u)$ - вероятности состояний ноды u в состояниях x с точки зрения ноды v ;
- Внутренняя информация ноды u о своем состоянии $b_u(x_u)$ - вероятности состояний ноды u в состояниях x ;
- Условные вероятности для корректировки чужого мнения: $\psi(x_u|x_v)$.

Алгоритм состоит из следующих шагов:

1. В начале все сообщения инициализируются из равномерного распределения
2. Вычисление доверия о себе $b_u(x_u)$ по данным соседей:

$$b_u(x_u) = \prod_{v \in N(u)} \mu_{v \rightarrow u}(x_u)$$

3. Нормализовать результат так, чтобы $b_u(x_u)$ имела смысл вероятности нахождения ноды в одном из состояний x_u : $\sum_{x_u} b_u(x_u) = 1$.
4. Передача неизвестных вероятностей своих состояний $\mu_{v \rightarrow u}(x_u)$ по данным соседей остальным соседям:

$$\mu_{u \rightarrow v}(x_u) = \prod_{s \in N(u) \setminus v} \mu_{s \rightarrow u}(x_u)$$

5. Вычисление условных вероятностей своих состояний с учетом данных соседей

$$\psi(x_u|x_v) = \mu_{v \rightarrow u}(x_u) \otimes \mu_{u \rightarrow v}(x_v)$$

6. Корректировка чужого мнения о себе с учетом условных вероятностей и априорной информации $\phi_u(x_u)$:

$$\mu_{v \rightarrow u}(x_u) = \sum_{x_v} \psi_v(x_u|x_v) \phi_u(x_u) \prod_{s \in N(u) \setminus v} \mu_{s \rightarrow v}(x_v)$$

7. Этапы 2-6 выполняются до достижения конверсии:

$$b_u(x_u) = \phi_u(x_u) \prod_{v \in N(u)} \mu_{v \rightarrow u}(x_v)$$

Основная сложность распространения доверия - наличие циклов. В этом случае сообщение может обойти цикл и вернуться к отправителю в изменённом виде, и получатель не знает, что это - его же собственное сообщение. Возникает положительная обратная связь, приводящая к раскачке (нестабильности) или медленной/неполной сходимости.

Одно из решений проблемы - демпфирование (damping) - при обновлении сообщения берётся взвешенное среднее старого и нового значения:

$$m_{new} = \alpha \cdot m_{old} + (1 - \alpha) \cdot m_{computed}$$

где $0 < \alpha < 1$ - гиперпараметр (обычно 0.1-0.5). Как следствие сообщения постепенно затухают, и система становится устойчивее.

Другим методом повышения устойчивости прогноза является использование алгоритма "Корректировка и сглаживание" (Correct & Smooth).

4.5. Корректировка и сглаживание

Этот метод не передаёт сообщения и не является классификатором, но корректирует вероятностные предсказания выдаваемые другими моделями (базовым классификатором) глобально по графу.

Алгоритм состоит из следующих шагов:

1. Обучаем базовый классификатор, предсказывающий вероятности состояния каждой ноды $P(y_u=1|f_u)$, а не жёсткие метки, и делаем прогнозы вероятностей каждого состояния p_u каждой ноды:

$$p_u = P(y_u=1|f_u)$$

2. Корректируем ошибки: для известных нод (обучающий датасет) вычисляем ошибку прогноза с учительским значением y_u :

$$e_u = y_u - p_u$$

3. Распространяем (усредняем) ошибку по графу через умножение на скорректированную матрицу смежности $\tilde{A}$:

$$e' = \tilde{A}^T \cdot e$$

здесь e'_u - суммарная ошибка, пришедшая к ноде u от соседей.

4. Корректируем вероятности:

$$p_{u,new} \leftarrow p_u + \alpha \cdot e'_u$$

где α - гиперпараметр корректировки (обычно $0 < \alpha < 1$).

5. Нормализуем полученные скорректированные вероятности, например через softmax:
 $p_{u,new} \leftarrow \exp(p_u) / \sum_v \exp(p_v)$

6. Сглаживаем вероятности: берём скорректированные p_{new} , снова суммируем по нодам, делаем взвешенное среднее с исходными:

$$p_{u,smooth} \leftarrow (1 - \beta) \cdot p_{u,new} + \beta \cdot (\tilde{A} \cdot p_{new})_u$$

7. Повторяем шаги 2-6 до конверсии.

В качестве скорректированной матрицы смежности используется

$$\tilde{A} = D^{-1/2} A D^{-1/2},$$

являющийся симметричным матричным представлением усреднения. Здесь $D = \text{diag}([deg(1), deg(2), \dots, deg(N)])$ - диагональная матрица степеней нод.

Такой алгоритм более стабилен, даже на циклических графах.

5. Графовые нейронные сети

5.1. Введение

Зачем нужны графовые нейронные сети?

- В изображениях данные организованы в регулярную сетку, для решения задач на изображениях применяются свёртки.
- В текстах данные организованы в последовательность токенов - применяются RNN/LSTM, трансформеры и другие сложные архитектуры.
- В графах структура произвольна: одни связи образуют матрицу, другие - цепочку, третьи - дерево.

Поэтому стандартные архитектуры (CNN, RNN) не универсальны и не годятся для обработки графовых данных. Графовые нейронные сети [<https://doi.org/10.1109/TNN.2008.2005605>] созданы для обработки произвольных структур в данных.

Ранее нами задачи решались гибридным способом - сначала мы искали эмбединг для элементов графа - нод, ребер, и графов, а после это пытались решать различные задачи с учителем и без учителя.

Рассмотрим другой вариант: end-to-end - оптимизационным задачам, где решение получается автоматически, без исходного создания эмбединга. Решение основывается на графовых нейронных сетях.

Графовая нейронная сеть состоит из двух частей:

1. Энкодер - получает на вход граф (вместе с внутренними признаками узлов) и выдаёт эмбединги - векторные представления узлов.
2. Декодер - принимает эмбединги и выдаёт конечный результат (например, вероятность связи между двумя узлами).

В отличие от гибридного подхода, где обучение энкодера и декодера происходит по отдельности, при end-to-end решении обе части нейронной сети тренируются совместно.

Раньше, при решении задач прогноза ребра, в качестве декодера мы использовали скалярное произведение эмбедингов:

$$\text{score}(u, v) = Z_u Z_v^T$$

или вероятность связи:

$$\text{score}(u, v) = \frac{\exp(Z_u Z_v^T)}{\sum_{s \in G} \exp(Z_u Z_s^T)}$$

Чем ближе векторы, тем выше схожесть и тем вероятнее связь.

При решении более сложных задач декодер может быть более сложным, и будет зависеть от решаемой задачи.

Таким образом, наша основная задача - научиться строить хорошие энкодеры и совместно с декодером оптимизировать их под целевую функцию.

5.2. Простейшее решение задачи прогноза на графе

5.2.1. Простейший подход и его недостатки

Самый простой способ построить нейронную сеть способную обрабатывать данные в графовом представлении - это объединить топологию и признаки на уровне матриц: взять матрицу смежности A и дописать к каждой строке A_u , описывающей связи ноды u с другими нодами графа, соответствующий вектор внутренних признаков этой ноды X_u . Получится прямоугольная матрица (а не квадратная, как A), которую можно подать на вход свёрточной сети, трансформера и т.п., сведя задачу анализа графа к известной задаче анализа изображений.

У этого подхода есть серьёзные проблемы:

1. Отсутствие масштабируемости.

При добавлении одного нового узла

- в матрице смежности добавляется одна строка и один столбец;
- в объединённой матрице размерность резко меняется.

Одновременно, это существенно усложняет разбиение данных на обучающую, валидационную, и тестовые выборки, и, соответственно, проверку решения.

Традиционно свёрточные сети, используемые при анализе изображений, требуют фиксированного размера входа (или достаточно сложной адаптации), поэтому такой подход не масштабируется.

2. Отсутствие инвариантности к перестановкам узлов.

В изображении пиксели упорядочены по двум (или трём, с каналами) координатам:

- ширина (x),
- высота (y),
- каналы/цвета (c) и т.д.

И любая перестановка пикселей может изменить смысл изображения. В графе же узлы - категориальные объекты без естественного порядка. Перестановка двух узлов (соответственно, двух строк и столбцов в A) не должна менять результат. Свёрточные сети не инвариантны к таким перестановкам и нужно проводить дополнительные действия для обеспечения такой инвариантности (например аугментацию), что замедляет и усложняет обучение.

3. Множественность и размерность признаков.

Признаков может быть очень много: разные типы связей (соседство, родство, обучение в одной группе), что особенно характерно для графов знаний. Разные рёбра при этом описываются разными матрицами смежности. Как итог, один граф может быть представлен как набор матриц смежности -трёхмерным тензором $A \in R^{N \times N \times r}$, где r - число типов рёбер, а N - число нод. В итоге простое дописывание столбцов к матрице быстро приведёт к взрыву размерности и классические методы не справятся.

5.5.2.Граф вычислений графовой нейронной сети

Графовая нейронная сеть - это архитектура, которая одновременно учитывает:

- внутренние признаки узлов,
- топологические признаки узлов (связи между узлами),
- выходные метки.

Такая архитектура называется графовой нейронной сетью. Простейшим элементом графовых нейронных сетей является слой графовой свертки, являющегося неким вариантом распространения доверия, где в качестве сообщений выступают признаки нод, модифицируемые от слоя к слою сети.

В рамках этого подхода, для обновления признаков узла A необходимо:

1. Взять признаки его соседей - например, узлов B, C, D;
2. Применить к ним коллективную операцию - агрегацию,
3. Получить новое значение признаков для A.

Одновременно все узлы должны обновлять свои признаки. Вся эта схема образует вычислительный граф - направленный граф зависимостей, где каждая операция над признаками является узлом, а используемые данные(признаки) формируют рёбра. Пример исходного графа данных и вычислительных графов для двух нод приведен на Рис.18. Из рисунка видно, что архитектура вычислительного графа для каждой ноды может быть различна, что и является одной из основных сложностей и особенностей графовых нейронных сетей: у каждой ноды может быть различное число соседей и на разных уровнях вычислений одна и та же нода может иметь на разных уровнях вычислений различные признаки и их значения.

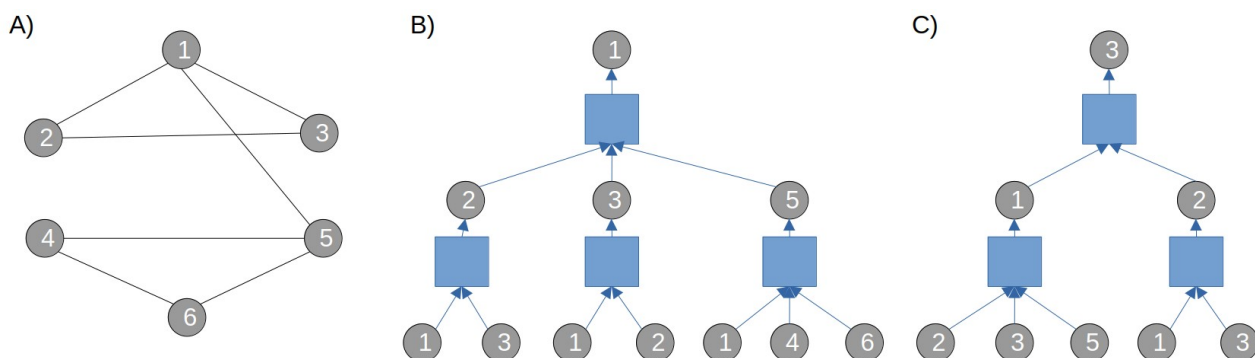


Рис.24. Исходный граф (A) и граф вычислений для ноды 1 (B) и ноды 3 (C). Круги — ноды, квадраты — операции над признаками.

Все современные фреймворки (TensorFlow, PyTorch) работают именно с такими вычислительными графами:

- граф строится при прямом проходе,
- затем вычисляются градиенты (обратное распространение ошибки),
- и обновляются параметры.

Таким образом, при построении графовой нейронной сети для каждого графа данных необходимо строить граф вычислений, представляющий архитектуру этой графовой нейронной сети.

Каждый графовый слой состоит из нескольких этапов:

1. Передача сообщений (message passing): вычисление сообщения $m_{u \rightarrow v} = h_u^{(l)}$ от соседа u к ноде v .
2. Агрегация (aggregation): объединение всех сообщений в новое представление ноды:

$$a_v^{(l+1)} = AGG(h_v^{(l)} \cup m_{u \rightarrow v} | u \in N(v)),$$
3. Трансформация одних типов признаков нод в другие типы признаков:

$$h_v^{(l+1)} = \phi(W^{(l)} a_v^{(l+1)}),$$

где ϕ - поэлементная нелинейная функция активации (например, ReLU), W - обучаемая матрица весов, h_v^k - представление ноды ($h_v^0 = x_v$) после слоя k . Обычно агрегацию и трансформацию объединяют в агрегацию.

Проблемы непосредственного применения нейронных сетей к графам связаны со сложностью агрегации информации от соседей, обычно выполняемой в традиционных нейронных сетях нейроном или сверткой.

Существенные проблемы применения этого подхода на графах связаны с:

1. Разным числом входов: каждый узел имеет своё число соседей (равным степени вершины). Поэтому простая полносвязная сеть не применима: у неё фиксированное число входов.
2. Изменением числа признаков на каждом слое вычислительного графа: например в трёхслойной сети $F3(F2(F1(X)))$ вход H_u (признаки узла u) на каждом слое меняется:

- после F1 - $H_u^{(1)}$,
- после F2 - $H_u^{(2)}$,
- после F3 - $H_u^{(3)}$.

Однако, все эти векторы описывают один и тот же узел, просто на разных участках графа вычислений. Поэтому легко потерять признаки исходного объекта и его связь с другими объектами.

Отсюда возникают требования к любой функции агрегации, применяемой в графовых нейронных сетях. Любая операция $AGG(H_u | u \in N(v))$, используемая для объединения соседей, должна удовлетворять двум условиям:

1. Инвариантность к числу аргументов: результат должен быть корректен при любом количестве соседей.
2. Инвариантность к перестановкам: если поменять порядок соседей, результат не должен меняться.

Этими свойствами обладают несколько операций:

- Среднее арифметическое: $AGG = (1/|N(v)|) \sum_{u \in N(v)} H_u$
- Среднее геометрическое (для положительных значений): $AGG = \left(\prod_{u \in N(v)} H_u \right)^{\frac{1}{|N(v)|}}$

Эти функции перестановочно-инвариантны и корректны при любом $|N(v)| > 0$.

Таким образом, графовая нейронная сеть реализует итеративное обновление признаков узлов через:

1. агрегацию соседей (инвариантную к порядку и числу),
2. линейные преобразования с общими весами и нелинейные активации (проекции и передачи сообщений).

И всё это встраивается в стандартный вычислительный граф для обучения с помощью градиентного спуска.

5.5.3. Графовая свертка

Слой графовой свертки имеет вид [<https://doi.org/10.1109/TNN.2008.2010350>]:

$$H_v^{new} = \varphi^o \left(W \frac{1}{deg(v)} \sum_{u \in N(v)} H_u + W_{self} H_v \right)$$

где W, W_{self} - матрицы весов $N \times N$, одинаковые для всех узлов; φ^o - поэлементная функция активации (ReLU, tanh и т.д.) над матрицей признаков. Это эквивалентно передаче доверия: узел корректирует своё состояние, основываясь на мнении соседей.

На каждом слое признаки узла H_v меняются: $H_v^0 \rightarrow H_v^1 \rightarrow H_v^2$, а структура графа сохраняется.

Вид графовой свертки связан рассматриваемым уже методом распространения сообщений и с требованиями к операции объединения соседей:

1. Инвариантность к перестановкам;
2. Инвариантность к изменению числа аргументов.

Простейшие подходящие операции:

- Сумма: $AGG(X) = \sum_{u \in N(v)} H_u$

- Среднее: $AGG(X) = \frac{1}{|N(v)|} \sum_{u \in N(v)} H_u$, где $|N(v)| = deg(v)$ - степень узла.

Слой графовой свертки может быть записан в матричном виде:

$$H^{new} = \varphi^o(W \tilde{A} H + W_{self} H)$$

где $\tilde{A}$ - скорректированная матрица смежности, W, W_{self} - матрицы коэффициентов.

Скорректированная матрица смежности имеет вид:

$$\tilde{A} = D^{-1/2} (A + I) D^{-1/2}$$

где D - диагональная матрица степеней вершин, I - единичная матрица, добавляющая самоциклы к каждому узлу, что важно для учета изолированных узлов.

Матрица W может быть прямоугольной, тогда размерность вектора признаков на входе и на выходе различаются. Таким образом, в рамках графовой свертки возможно сжимать и расширять признаковое пространство, получая на выходе столько признаков для каждого узла, сколько потребуется решаемой задаче.

Пошагово, графовую свертку можно представить в виде суперпозиции трех шагов:

1. Передача сообщений (Message Passing) - сбор информации от соседей: каждая соседняя узел отправляет своё состояние (признаки) текущей;
2. Агрегация (Aggregation): умножаем вектор признаков H на $\tilde{A}$, в этом случае каждый узел получает усреднённые фичи соседей;
3. Проецирование (Transformation): умножаем полученные значения признаков на матрицу W (проецируем признаки на другое пространство), добавляем проекцию собственных признаков и поэлементно применяем функцию активации φ^o к каждому признаку, получая новые признаки в пространстве новой размерности.

Повторяя операцию несколько раз с разными коэффициентами, получаем графовую сверточную нейронную сеть. На вход такой сети подаём:

- H (признаки),
- edge_index (список рёбер: $2 \times E$), или A (матрицу смежности),
- (опционально) веса рёбер.

На выходе получаем N узлов с d' признаками каждый, размер которых выбирается исходя из задачи: для классификации, регрессии, прогноз ребра и т.д. и может быть в дальнейшем дополнительно преобразован в зависимости от задачи.

Последовательность преобразований в графовом сверточном слое приведена на Рис.25.

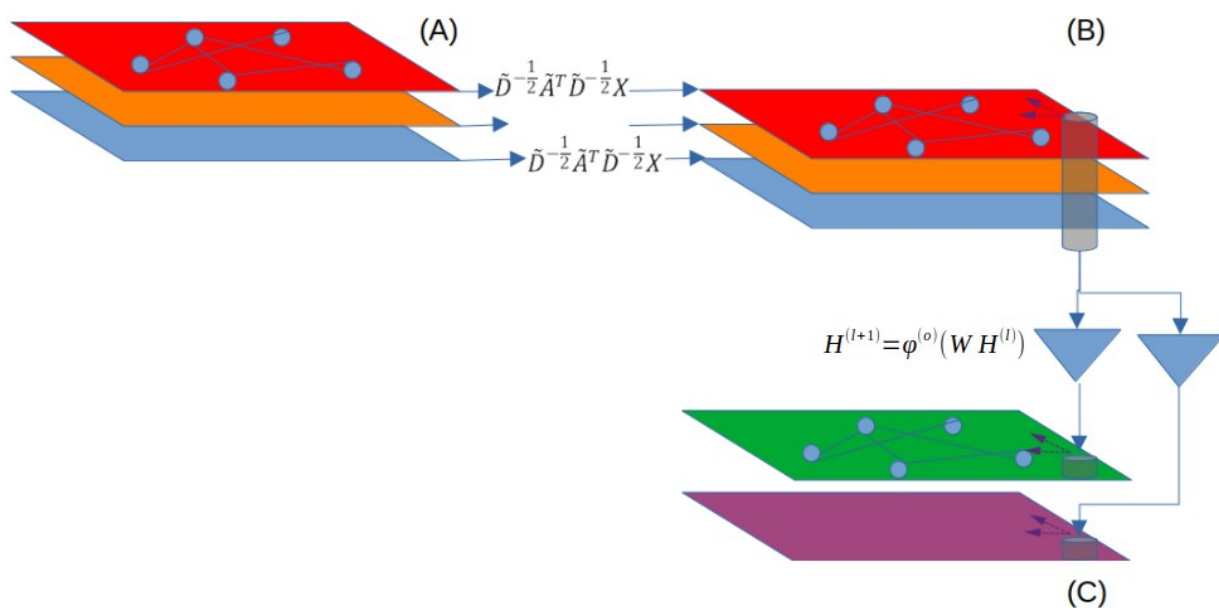


Рис.25. Схема графовой свертки. (A) → (B) Агрегация признаков; (B) → (C) Трансформация признаков.

5.3.Обучение графовых нейронных сетей

5.3.1.Типы задач и обучение графовых нейронных сетей

Структура типичной прогнозной графовой нейронной сети приведена на Рис.20.

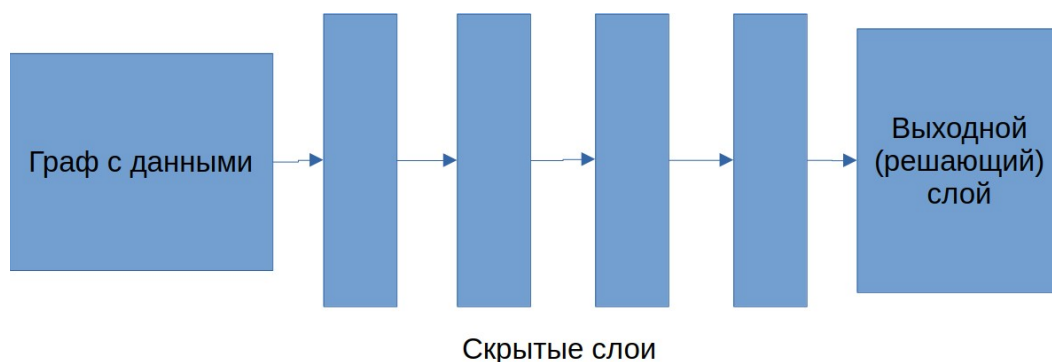


Рис.26. Структура графовой нейронной сети

При работе с графами обычно решают одну из четырёх базовых задач:

1. Задачи уровня узла (Node-Level Prediction) - предсказание свойств (признаков или меток) отдельных узлов;
2. Задачи уровня ребра (Edge-Level Prediction) - предсказание свойств рёбер (например, существует ли ребро, или каков его вес);
3. Задачи уровня графа (Graph-Level Prediction) - предсказание свойств всего графа целиком (например, является-ли молекулярный граф лекарством).

4. Задачи уровня подграфа (Subgraph-Level Prediction) - предсказание свойств подграфа или оценка вероятности его вхождения в другой граф.

В зависимости от типа задачи меняется структура выходного (решающего) слоя нейронной сети, при этом структура скрытых слоев может оставаться неизменной.

5.3.1.1. Задача уровня узла

Структура сети:

Вход - граф - x (матрица признаков узлов) и $edge_index$ (список рёбер);

Скрытые слои - графовые операции - Message Passing и Aggregation (например, GCN, GraphSAGE, GAT);

Выходной слой зависит от задачи:

- Регрессия (предсказываем несколько чисел на узел Rd , чаще всего 1):
выходной слой - линейный, размер Rd ;
функция потерь - MAE или MSE.
- Классификация (предсказываем принадлежность к одному из C классов):
выходной слой - линейный, размер C ;
с функцией активации $softmax$;
функция потерь - кросс-энтропия.

5.3.1.2. Задача уровня ребра

Для предсказания свойств ребра (u, v) нужно объединить информацию от двух узлов.

Обычно действуют так:

1. Пропускают граф через GNN и получают эмбединги узлов h_u, h_v (векторы длины d);
2. Строят комбинацию из них - варианты:
 - Конкатенация: $z = (h_u || h_v) \rightarrow$ вектор длины $2d$, затем линейный слой;
 - Скалярное произведение: $z = h_u^T h_v$ (одно число);
 - Поэлементное произведение: $z = h_u \odot h_v$;
 - Разность по модулю: $z = |h_u - h_v|$;
 - или любая другая симметричная функция, если граф ненаправленный.

Затем z подаётся в выходной слой (линейный + сигмоид для бинарной классификации, или линейный для регрессии).

Функции потерь - те же: MSE/MAE для регрессии, кросс-энтропия для классификации.

5.3.1.3. Задача уровня графа

Требуется получить один вектор-эмбединг на весь граф. Делается это так:

1. Пропускаем граф через нейронную сеть и получаем эмбединги всех узлов: $Z = [z_1, z_2, \dots, z_N]$;
2. Применяем глобальную агрегацию (инвариантную к перестановке узлов):
 - Усреднение: $Z_G = (1/N) \sum_{v=1}^N z_v$;
 - Сумму: $Z_G = \sum_{v=1}^N z_v$;
 - Максимум по признакам: $Z_G(j) = \max_{v=1..N} h_v(j) \forall j \in [1..d]$.

3. Полученный эмбединг Z_G подаётся в классический MLP (полносвязную сеть) для финального предсказания (регрессия или классификация).

5.3.2. Методы обучения на графах

При анализе графов обычно используются три основных метода обучения:

1. Обучение без учителя (unsupervised learning).

Например кластеризация узлов / рёбер / графов; методы: K-Means, DBSCAN, GM и др.

2. Обучение с учителем (supervised learning).

- Есть размеченные узлы (например, классы нод);
- Узлы разбиваются на обучающие, валидационные, и тестовые;
- На обучающих ведётся обучение, прогнозируется метка.

3. Самообучение (self-supervised learning).

Метки генерируются из самой структуры графа самой моделью. При самообучении создаётся большой датасет из одного графа без внешней разметки.

Например:

- Предсказание удалённого ребра: удаляем случайное ребро и предсказываем, существует ли оно. Рёбра для обучения называются supervision edges, остальные - message edges (используются только для передачи сообщений).
- Предсказание любой характеристики ноды по остальным нодам в графе и их характеристикам.

5.3.3. Типы задач на графах и выходные головы

После последнего графового слоя ставится выходной слой (решающий слой, “голова”), который зависит от задачи. Как обычно в нейронных сетях, характеристики головы (архитектура и функции активации выходного слоя) полностью определяются решаемой задачей. Примеры приведены в Таблице 1.

Таблица 1.

Задача	Активации
Регрессия	Неограниченные значения - линейная активация; Ограниченные значения - ReLU, σ , tanh
Классификация	Два класса с предсказанием вероятности одного - σ ; Несколько классов с предсказанием вероятностей каждого — Softmax или его вариации.

Если задача - предсказание рёбер, то объем вариантов ребра растёт как квадрат числа нод, учитывая что нужно прогнозировать не только существующие ноды, но и анализировать все возможные пары нод в графе, число которых может достигать $|N|(|N|-1)$ или $|N|(|N|-1)/2$ в зависимости от вида графа (направленный или ненаправленный).

Поэтому в этой задаче вместо классической активации softmax, чаще используется ее укороченные варианты, например версии негативного саплинга:

$$\phi(v|u) = \frac{\exp(Z_u Z_v^T)}{\exp(Z_u Z_v^T) + \sum_{s \in S, S \in G \setminus (v \cup N(v)), |S| \in [5..20]} \exp(Z_u Z_s^T)}$$

или

$$\varphi(v|u) = \frac{\exp(Z_u Z_v^T)}{\sum_{s \in S, S \in G \setminus (v \cup N(v)), |S| \in [5..20]} \exp(Z_u Z_s^T)}$$

Переход от одной задачи к другой в часто сводится к замене головы и функции потерь, остальная архитектура нейронной сети может остаться неизменной.

1. Задачи уровня ноды (Node-Level task)

Прогноз неких значений на каждом узле графа.

Голова: $Z_u \rightarrow \varphi$, где φ - активация, соответствующая задаче.

2. Задачи уровня ребра (Edge-Level task)

Прогноз неких значений для каждой пары узлов (u, v), например близости или существования ребра между ними, прогнозирование веса ребра (в взвешенном графе).

Голова:

- конкатенация: $\varphi(MLP([Z_u || Z_v]))$;
- скалярное произведение: $\varphi(Z_u Z_v^T)$;
- внешнее произведение: $AGG(Z_u \otimes Z_v^T)$;
- внимание: $Softmax(a^T [W Z_u || W Z_v])$.

3. Задачи уровня графа (Graph-Level task):

Прогноз неких значений для всего графа: классификация целых графов, предсказание какого-либо свойства графа.

Голова - нужен глобальный пулинг, агрегация по всем узлам:

- среднее: $Z_G = (1/N) \sum_{v \in G} Z_v$
- максимум (max-pooling): $Z_G[i] = \max_v Z_v[i]$
- сумма: $Z_G = \sum_{v \in G} Z_v$

Затем $\varphi(MLP(Z_G))$.

Для сложных графов часто делают иерархический пулинг - сначала делят на подграфы, агрегируют внутри, потом - между подграфами. Затем объединяют результаты конкатенацией, суммой, или средним.

4. Задачи уровня подграфа (Subgraph level task)

Прогноз неких значений для части графа.

5.3.4. Деление данных на обучающую, валидационную, и тестовую выборки

Для каждого типа задачи деление на обучающую, валидационную, и тестовую выборки делается по-разному.

5.3.4.1. Деление при задачах уровня ноды

Задача уровня ноды - это задача, в которой требуется определить некоторый параметр отдельной ноды графа.

Например:

- класс ноды (классификация);
- численная характеристика - фича ноды (регрессия);
- любая другая величина, связанная с конкретной вершиной графа.

Чтобы обучить нейронную сеть на такой задаче, граф необходимо разделить на части:

- обучающую выборку (training set);

- валидационную (validation set);
- тестовую (test set).

Способ 1: разбиение по нодам

Разделение по нодам (Transductive Learning). Разбиваем датасет на подмножества узлов: обучающую, валидационную, и тестовую. Выбираются ноды для каждой из частей (обучающей, валидационной, тестовой), и из исходного графа выделяются подграфы, содержащие только эти ноды и все рёбра, в которых соответствующие ноды участвуют.

При обучении нейронная сеть видит весь граф целиком, но при подсчёте признаков использует только информацию из обучающей и валидационной частей. Этот подход называется Transductive Learning (трансдуктивное обучение).

Проблема: подходит только для задач классификации/регрессии на узлах.

При кластеризации структура графа нарушается, и информация может искажаться.

Способ 2: разбиение по рёбрам через минимальное остовное дерево (MST)

Полная изоляция подграфов (Inductive Learning) - граф разбивается на непересекающиеся подграфы - то есть удаляются не только ноды, но и все рёбра, соединяющие их с остальной частью графа. Таким образом, обучающий, валидационный и тестовый подграфы становятся полностью независимыми.

При обучении сеть работает только с обучающим подграфом и «не знает» о существовании других частей. Этот подход называется Inductive Learning (индуктивное обучение).

Простой способ - строим минимальное остовное дерево - подграф без циклов, соединяющий все узлы, с минимальной суммой весов рёбер:

$$MST = \operatorname{argmin}_{T \subseteq G, T: \text{tree}} \left(\sum_{e \in T} w_e \right)$$

Затем последовательно удаляем самые длинные (дорогие) рёбра в MST:

- 1 удаление - 2 подмножества нод,
- 2 удаления - 3 подмножества нод.

Назначаем подмножества: обучение, валидация, тест.

Такой подход даёт слабосвязные подграфы и минимизирует утечку информации между выборками.

5.3.4.2. Деление при задачах уровня ребра

Задача уровня ребра - это задача, в которой требуется определить:

- характеристики самого ребра (например, вес, тип);
- существует ли ребро между двумя заданными нодами (предсказание связей / link prediction).

В последнем случае - предсказания существования связи - вводится понятие supervision edge (контролируемое / целевое ребро): это ребро, для которого известно, существует оно или нет, и которое используется как целевая переменная в обучении.

Все рёбра графа разделяются на обучающую, валидационную и тестовую выборки. В каждой из этих выборок случайным образом выбирается подмножество рёбер - supervision edges, которые и используются для формирования целевой переменной.

Это разделение можно проводить в двух режимах:

- Индуктивный режим: как и ранее, подграфы полностью изолируются, и предсказание делается только на основе внутренней структуры подграфа.

- Трансдуктивный режим: используется весь граф целиком, но supervision edges выбираются разными в разных фазах (обучение/валидация/тест), при этом структура графа не меняется.

Задачи прогноза ребра часто сводятся к самообучению - когда нужно предсказать существование ребра. В этой задаче граф нужен целиком для передачи сообщений, поэтому используется трансдуктивный режим.

При решении задачи link prediction граф необходимо разделить на четыре типа наборов рёбер:

1. Обучающие рёбра (train edges) - по ним происходит распространение сообщений (message passing) и обновление весов.
2. Supervision рёбра (supervision edges) - рёбра, скрытые от модели при обучении, но используемые для оценки функции потерь. Это механизм самообучения (self-supervised learning): часть реальных рёбер удаляется из графа, модель пытается их восстановить, и loss считается по этим «выдернутым» рёбрам.
3. Валидационные рёбра (validation edges) - используются для подбора гиперпараметров и ранней остановки (early stopping).
4. Тестовые рёбра (test edges) - используются для финальной, независимой оценки качества.

Важно, что обучающие и supervision рёбра - непересекающиеся подмножества ребер одного и того же исходного графа.

5.3.4.3. Деление при задачах уровня графа

Задача уровня графа - это задача, где целевая величина характеризует весь граф в целом, например:

- классификация графов (какой тип имеет граф);
- регрессия по глобальному параметру (например, диаметр, энергия);
- сравнение графов между собой.

Обычно в таких задачах датасет это множество графов (например, молекулы, документы как графы). Поэтому просто разбиваем набор графов на три части: обучение, валидация, тест. Это не приносит никаких особых сложностей - графы независимы.

5.3.5. Функции потерь

Часто функции потерь в графовых нейронных сетях те же, что и в обычных нейронных сетях, и зависят от задачи. Однако суммирование идёт по узлам или рёбрам графа, а не по сэмплам датасета, как в классических нейронных сетях. Выбор функции потерь часто зависит не только от задачи, но и от архитектуры сети и метода решения.

Если стоит задача **регрессии** (на нотах, ребрах, графах) то чаще всего используются:

- Среднеквадратичная ошибка (MSE, Mean Squared Error):

$$L = \frac{1}{|N|} \sum_{u \in N} (y_u - y_{pred,u})^2$$

- Средняя абсолютная ошибка (MAE, Mean Absolute Error):

$$L = \frac{1}{|N|} \sum_{u \in N} |y_u - y_{pred,u}|$$

Суммирование ведётся по нодам u .

В задачах **классификации** часто используются:

Косинусная близость (в задачах прогноза ребра или поиска эмбединга для ноды):

$$L = 1 - \frac{Z_u Z_v^T}{|Z_u| |Z_v|}$$

или

$$L = 1 - \sigma(Z_u Z_v^T)$$

Кросс-энтропия (cross-entropy):

$$CE = -\frac{1}{|N|} \sum_{n \in N} \sum_{c=1}^C y_{u,c} \log(p_{u,c})$$

где $y_{u,c}$ - истинная метка ноды u для класса c (0/1), $p_{u,c}$ - предсказанная вероятность для этой ноды и этого класса. Суммирование ведется по нодам N и классам C .

Бинарная кросс-энтропия (binary cross-entropy):

$$BCE = -\frac{1}{|N|} \sum_{i=1}^N (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

где y_i - метка бинарного класса (0/1) для ноды i , p_i - предсказанная вероятность класса 1. Суммирование ведется по нодам N .

Дивергенция Кульбака-Лейблера (Kullback-Leibler Divergence):

$$KL(P||Q) = \sum_x P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

где P и Q - два распределения вероятностей. Чаще всего используется в вероятностных или генеративных моделях, например в вариационном автоэнкодере (VAE). Она более устойчива к выбросам, чем кросс-энтропия.

5.3.6. Метрики качества

В качестве метрик качества используются стандартные метрики качества - MSE, MAE для задач регрессии, Матрица ошибок, F1, AUC-RP, AUC-ROC для классификации.

5.3.7. Пример прогноза класса ноды сверточной сетью

Датасет Cora — один из стандартных датасетов для проверки графовых нейронных сетей. Cora состоит из 2708 научных публикаций и цитирований между ними, классифицированных по одной из семи категорий. Каждая публикация в наборе данных описывается векторным представлением слова со значениями 0/1, указывающим на отсутствие/наличие соответствующего слова из словаря в этой публикации. Словарь содержит 1433 уникальных слова. В графовом представлении это направленный граф, состоящий из 2708 нод, 10556 ребер, каждый нод имеет 1433 бинарных признака. Ноды разделены на 7 классов.

Алгоритм 19 представляет собой простейшее обучение двухслойной графовой сверточной сети для классификации нод на датасете Cora.

Нейронная сеть для классификации нод состоит из двух сверточных слоев, на первом слое активация ReLU, на втором слое активация Softmax. Количество признаков в первом (скрытом) слое 16, количество признаков во втором (решающем) слое — определяется датасетом и равно 7 (классам). Входное количество признаков нод определяется датасетом и равно порядка 1.5 тыс.

В качестве функции потерь используется кросс-энтропия (в реализации torch — NLLLoss, требующий присутствия Softmax на выходе нейронной сети). В качестве оптимизатора ADAM с скоростью обучения 0.01 и L2 регуляризацией коэффициентов сети с

уровнем $5e-4$. Обучение классификатора идет по 100 эпохам на обучающей части датасета, качество классификатора ($F_1^{(macro\ avg)} = 0.8$) определено по тестовой части датасета.

Алгоритм 24. Простейшее обучение двухслойной графовой сверточной сети для классификации нод на датасете Cora

```
import torch
from torch_geometric.datasets import Planetoid
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
import numpy as np
from sklearn.metrics import classification_report

dataset = Planetoid(root=".", name="Cora")
data = dataset[0]
class GCN(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out):
        super().__init__()
        self.gcn1 = GCNConv(dim_in, dim_h)
        self.gcn2 = GCNConv(dim_h, dim_out)
    def forward(self, x, edge_index):
        h = self.gcn1(x, edge_index)
        h = torch.relu(h)
        h = self.gcn2(h, edge_index)
        return F.log_softmax(h, dim=1)
    def fit(self, data, epochs):
        criterion = torch.nn.NLLLoss()
        optimizer = torch.optim.Adam(self.parameters(), lr=0.01, weight_decay=5e-4)
        self.train()
        for epoch in range(epochs+1):
            optimizer.zero_grad()
            out = self(data.x, data.edge_index)
            loss = criterion(out[data.train_mask], data.y[data.train_mask])
            loss.backward()
            optimizer.step()

gcn = GCN(dataset.num_features, 16, dataset.num_classes)
gcn.fit(data, epochs=100)
yp=gcn(data.x,data.edge_index)
yp=np.argmax(yp.detach().numpy(),axis=-1)
print(classification_report(data.y[data.test_mask],yp[data.test_mask]))
```

	precision	recall	f1-score	support
0	0.64	0.76	0.70	130
1	0.79	0.89	0.84	91
2	0.86	0.91	0.89	144
3	0.91	0.75	0.82	319
4	0.78	0.83	0.81	149
5	0.81	0.75	0.78	103
6	0.70	0.84	0.77	64
accuracy			0.80	1000
macro avg	0.79	0.82	0.80	1000
weighted avg	0.82	0.80	0.81	1000

6. Повышение качества графовых нейронных сетей

Зона восприятия (Receptive field) узла - это множество узлов, которые влияют на его финальное состояние после прохождения через графовую нейронную сеть. Один сверточный слой соответствует учету соседей 1-го порядка (расстояние до которых не превышает 1), два сверточных слоя - учету соседей, расстояние до которых не превышает 2, k сверточных слоёв - учету соседей на расстояниях до k . На Рис.21 приведены зоны восприятия двух нод после прохождения через разное число сверточных слоев.

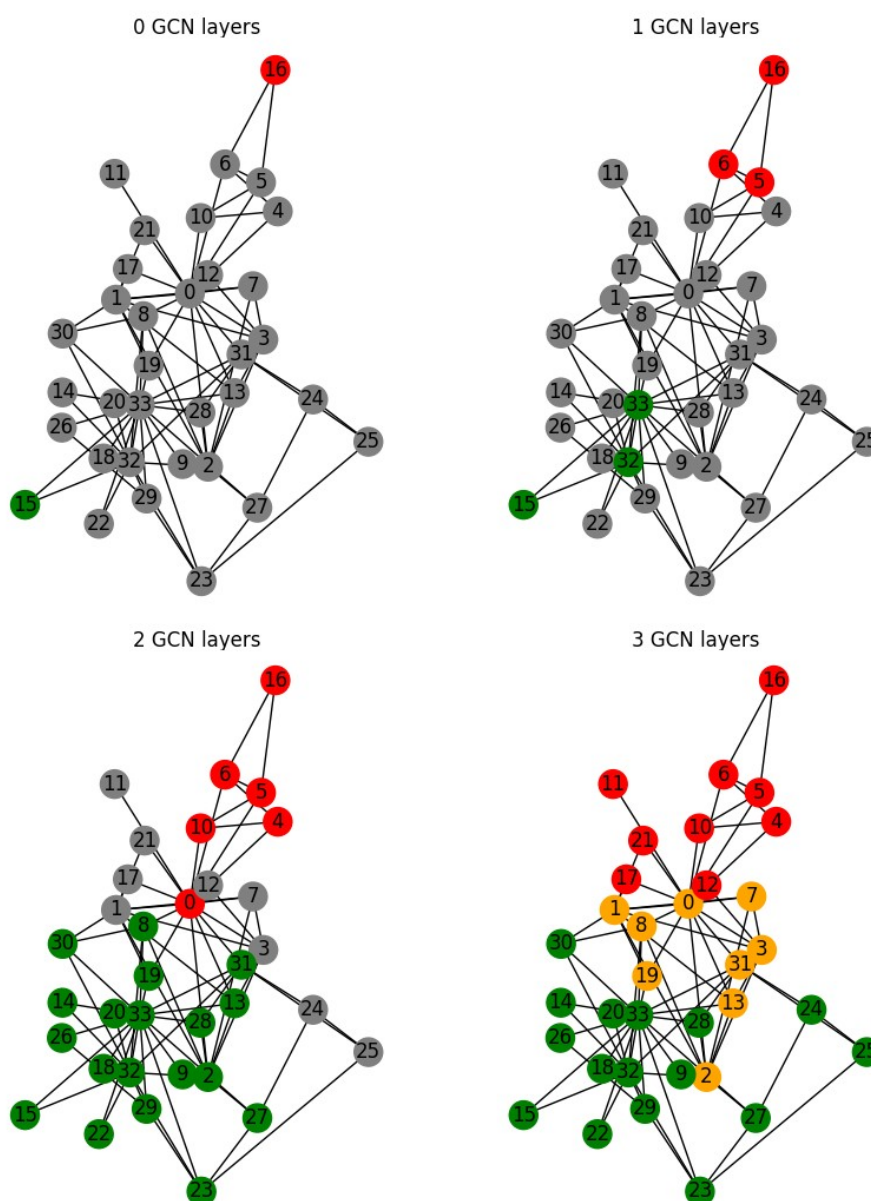


Рис.27. Зона восприятия двух нод (красный и зеленый) для датасета Karate Club при прохождении разного числа сверточных слоев. Оранжевые ноды — область пересечения зон восприятия. Серые ноды - не участвующие в зонах восприятия.

Таким образом, при рассмотрении близости двух нод (задача прогноза ребра) в большинстве графов зоны восприятия для любой пары нод перекрываются, если взять достаточно большое число сверточных слоев в нейронной сети. Очевидно, таким образом, что число сверточных слоев не стоит делать больше, чем половина диаметра графа. Для обычных социальных сетей (с диаметром 6-7) это соответствует максимальному количеству сверточных слоев 3. Если в нейронной сети больше графовых сверточных слоёв, это приводит к переусреднению (over-smoothing), и все эмбединги становятся близкими (одинаковыми).

Поэтому на практике часто приходится ограничиваться 2-3 сверточными слоями, и это ограничивает обобщающие свойства сети. Чтобы с этим бороться, существует несколько базовых способов:

- Добавить аугментацию в данные или сеть;
- Усложнить агрегацию каждого слоя;
- Изменить структуру связей между слоями или усложнить архитектуру сети.

6.1. Аугментация графа

Аугментация применяется с одной стороны, чтобы бороться с дисбалансом между структурой (числом свободных параметров) нейронной сети и объемом обучающих данных, с другой стороны - чтобы адаптировать решение к возможным искажениям входных данных после того, как сеть обучена.

В основном методы аугментации сводятся к следующим.

6.1.1. Ноды с малым числом признаков

Часто возникает ситуация, когда у нод недостаточно признаков, или вообще нет, поэтому передача сообщений неэффективна. В таких случаях стоит добавить к нодам какие-нибудь дополнительные признаки:

- Можно добавить локальные топологические признаки узлам. Это могут быть например классические топологические признаки.
- Иногда можно добавить фиктивный константный признак (например, 1 у всех) - это иногда улучшает сходимость. Это позволяет косвенно учитывать степень ноды при агрегации.
- Можно добавить уникальный ID узла через one-hot-кодирование: для N узлов - вектор $c \in R^N [0,0,...,1,...,0]$. Этот метод работает для небольших графов ($N < 1000$); для больших графов можно использовать хеширование или positional encoding. Плюсы: позволяет модели различать ноды глобально, частично учитывать структуру циклов. Минусы: размерность растёт линейно с числом нод, поэтому метод не подходит для больших графов.
- Можно добавить глобальные признаки, например PageRank. Графовые свёрточные сети плохо чувствуют циклы, так как работают локально. Добавление глобальных признаков - один из способов компенсировать это ограничение.

Пример добавления степени ноды, как нового признака к датасету Cora, приведен в Алгоритме 25.

Алгоритм 25. Добавление признака к датасету Cora

```
import torch
from torch_geometric.datasets import Planetoid
from torch_geometric.utils import degree
dataset = Planetoid(root='.', name='Cora')
data = dataset[0]
print("Original feature shape:", data.x.shape)
deg = degree(data.edge_index[0], num_nodes=data.num_nodes, dtype=torch.float)
data.x = torch.cat([data.x, deg.unsqueeze(-1)], dim=-1)
print("New feature shape:", data.x.shape)
```

```
Original feature shape: torch.Size([2708, 1433])
New feature shape: torch.Size([2708, 1434])
```

6.1.2. Разреженные графы

Можно добавить виртуальные рёбра:

- Через общего соседа (2-hop связи): если есть путь $u \rightarrow w \rightarrow v$, но нет прямого $u \rightarrow v$, добавляем виртуальное ребро;
- Через виртуальную (глобальную) ноду: создаём одну дополнительную ноду g , соединяем ее со всеми нодами графа. Тогда расстояние между любыми двумя узлами ≤ 2 , и граф становится эффективно плотным, улучшается Message Passing.

Алгоритм 26. Добавление виртуальной ноды к датасету Karate Club

```
import networkx as nx
g0=nx.karate_club_graph()
g1=nx.Graph()
v_new=-1
for u,v in g0.edges():
    g1.add_edge(u,v)
    if v>v_new: vnew=v
    if u>v_new: vnew=u
for u in g0.nodes():
    g1.add_edge(v_new,u)
pos=nx.kamada_kawai_layout(g1)
nx.draw(g1,pos)
```

6.1.3. Плотные графы

Проблема плотных графов - один слой свертки охватывает соседями каждого нода весь граф, в итоге сложно строить глубокие модели с большим количеством сверточных слоев. Можно использовать следующие решения:

- Использовать прореживание (subsampling) ребер, оставив только сильные ребра (по весу или минимальному остовному дереву).
- Применить сэмплирование соседей (neighbor sampling): при агрегации для ноды v выбирается подмножество соседей $S \subset N(v)$, $|S|=k \ll \deg(v)$, для их выбора можно использовать методы случайного сэмплирования (random sampling) или выбирать наилучшие ноды по весу/важности (top-k).
- Применять дропаут рёбер (edge dropout): случайно удалять часть рёбер при каждой итерации обучения, аналогично дропауту в обычных сетях это эквивалентно некой регуляризации.

- Использовать внимание - и выбирать важные рёбра динамически, при обучении сети, на основе признаков нод, которые это ребро соединяет.

6.2. Усложнение агрегации слоя

6.2.1. Модернизация признаков

Самое простое решение усложнения сети - улучшить агрегацию в графовой нейронной сети, не трогая Message Passing.

Как усложнить агрегацию:

- после усреднения по соседям использовать вместо одного нейрона многослойную сеть (MLP), механизм внимания, LSTM и т.д. (усложнение трансформации):

$$H_v^{agg} = MLP(\tilde{A} H)$$

или

$$H_v^{agg} = MLP([H_v \parallel AGG(H_u : u \in N(v))])$$

так добавляется больше нелинейностей, хотя сверточный слой остаётся один.

- Предобработка признаков (усложнение передачи сообщений):
 $Y = NETWORK(MLP(X))$
- Постобработка признаков (усложнение трансформации): $Y = MLP(NETWORK(X))$.

Алгоритм 27. Добавление предобработки признаков в сеть

```
class GCN(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out, layers=3):
        super().__init__()
        self.layers_num=layers
        self.lin1 = Linear(dim_in,dim_h)
        self.gcn0 = GCNConv(dim_h, dim_h)
        self.gcn1 = GCNConv(dim_h, dim_h)
        self.gcn_out = GCNConv(dim_h, dim_out)

    def forward(self, x, edge_index):
        h = self.lin1(x)
        h = torch.relu(h)
        h = self.gcn0(h, edge_index)
        h = torch.relu(h)
        h = self.gcn1(h, edge_index)
        h = torch.relu(h)
        h = self.gcn_out(h, edge_index)
        return h
```

6.2.2. Проблема циклов в графах

Алгоритмы передачи сообщений (Message Passing) плохо различают циклы в графе. Например, алгоритм не отличит цикл длины 3 от цикла длины 4, если не использовать глобальные признаки.

В качестве решения можно добавить структурные признаки, кодирующие наличие циклов или повторные посещения узлов. При обходе графа (например, случайным блужданием) можно завести вектор-индикатор, который сигнализирует, если при обходе мы вернулись в исходный узел v (встретили его повторно), по аналогии с анонимными

блужданиями. Этот признак помогает детектировать цикличность и повышает устойчивость модели.

6.3. Улучшенные архитектуры сетей

6.3.1. Skip-connections и Residual blocks

Когда мы вставляем много промежуточных слоёв между графовыми свёртками, или много графовых свёрток, качество ухудшается.

Это связано с тем, что:

- Message Passing - нелинейная операция (нормализация, суммирование);
- Градиенты затухают при прохождении через нелинейные блоки, и сеть плохо обучается.

Одним из способов повысить чувствительность сети является добавление skip-связей и формирование residual-блоков. Добавление skip-связей фактически позволяет напрямую учесть влияние соседей второго и третьего порядка на нод, минуя и не изменяя его непосредственных соседей. Это позволяет учитывать более сложные зависимости.

Добавление skip-связей имеет вид:

$$H^{(l+1)} = \phi^o(W^{(l)} AGG(H^{(l)})) + H^{(l)}$$

Таким образом мы сохраняем локальную информацию из предыдущего слоя, улучшаем поток градиента (что позволяет строить и обучать глубокие сети). Кроме того узел учитывает и ближайших, и дальних соседей одновременно.

Примеры skip-связей:

- от входа напрямую к выходу;
- после каждого слоя - сумма или конкатенация с выходом всей сети;
- суммирование через несколько слоёв (residual-блоки).

Skip-связи со сложением признаков можно применять, когда количество признаков каждой ноды не меняется при прохождении сети.

Skip-связи со объединением признаков (конкатенация) можно применять практически всегда, потому что графовая свёртка не меняет число нод - вход и выход практически всегда совместимы по размерности.

Алгоритм 28. Добавление скип-связей в сеть

```
class GCN(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out, layers=3):
        super().__init__()
        self.layers_num=layers
        self.gcn0 = GCNConv(dim_in, dim_h)
        self.gcn1 = GCNConv(dim_h, dim_h)
        self.gcn2 = GCNConv(dim_h, dim_h)
        self.gcn_out = GCNConv(dim_h, dim_out)
    def forward(self, x, edge_index):
        h = self.gcn0(x, edge_index)
        h = torch.selu(h)
        h_old =h
        h = self.gcn1(h, edge_index)
        h = torch.selu(h)
        h +=h_old
        h_old =h
        h = self.gcn2(h, edge_index)
        h = torch.selu(h)
```

```

h += h_old
h = self.gcn_out(h, edge_index)
return h

```

6.3.2. Сеть графового внимания (GAT, Graph Attention Network)

В графовой сверточной сети (GCN) признаки соседей усредняются с одинаковыми весами. Идея сетей графового внимания (GAT) - усреднять признаки соседей с различными весами, причем вес определяется из комбинации признаков ноды и ее соседа.

Алгоритм работы сети графового внимания имеет вид [https://arxiv.org/pdf/1710.10903]:

1. Для пары узлов u и v вычисляем вес:

$$e_{uv} = \text{leakyReLU}(\vec{w}_{att}^T [W_{tr} H_u || W_{tr} H_v])$$

где $W_{tr}, \vec{w}_{att}$ - обучаемая матрица преобразования признаков (проекция) и коэффициенты нейрона для формирования нелинейных признаков внимания.

2. Нормализуем вес через softmax по соседям u , получая матрицу внимания:

$$a_{uv} = \frac{\exp(e_{uv})}{\sum_{s \in N(u)} \exp(e_{us})} = \text{softmax}_v(e_{uv})$$

3. Новые признаки получаются усреднением проекций признаков с весом, определяемым матрицей внимания:

$$H_u^{new} = \phi^o \left(\frac{1}{|N(u)|} \sum_{v \in N(u)} a_{uv} W H_v \right)$$

или в матричной записи:

$$H^{new} = \phi^o(\tilde{A} a W H)$$

Как и в обычных нейронных сетях, можно сформировать многоголовое внимание: повторяем K раз с разными $W_{tr}^{(k)}, W_{att}^{(k)}, W^{(k)}$, результаты конкатенировать:

$$H^{new} = \phi^o \left(\left\| \sum_{k=1}^K \tilde{A} a^{(k)} W^{(k)} H \right\| \right)$$

или усреднить:

$$H^{new} = \phi^o \left(\frac{1}{K} \sum_{k=1}^K \tilde{A} a^{(k)} W^{(k)} H \right)$$

Многоголовое внимание - это механизм, позволяющий модели одновременно учитывать разные аспекты окружения ноды.

Недавние исследования показали, что более стабильный результат даёт изменённый порядок операций - так называемая Graph Attention Network версии 2 (GATv2):

1. Применение линейного преобразования к конкатенированному вектору признаков, без их линейного преобразования:

$$e_{uv} = \text{LeakyReLU}(\vec{w}_{att}^T [H_u || H_v])$$

2. Затем softmax и агрегация, как раньше.

Ключевое отличие: вектор внимания $\vec{w}_{att}^T$ применяется уже к объединённому вектору, а не к отдельно преобразованным признакам.

Графовое внимание оптимизирует усреднение по соседям, подбирая оптимальные веса, поэтому часто работает лучше, чем сеть с обычным усреднением.

Алгоритм 29. Использование сети с графовым вниманием GATv.2

```

import numpy as np
from torch_geometric.datasets import Planetoid
import torch
import torch.nn.functional as F
from torch_geometric.nn import GATv2Conv
dataset = Planetoid(root=".", name="Cora")
data = dataset[0]

def accuracy(y_pred, y_true):
    return torch.sum(y_pred == y_true) / len(y_true)

class GAT(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out, heads=8):
        super().__init__()
        self.gat1 = GATv2Conv(dim_in, dim_h, heads=heads)
        self.gat2 = GATv2Conv(dim_h*heads, dim_out, heads=1)
    def forward(self, x, edge_index):
        h = F.dropout(x, p=0.6, training=self.training)
        h = self.gat1(h, edge_index)
        h = F.elu(h)
        h = F.dropout(h, p=0.6, training=self.training)
        h = self.gat2(h, edge_index)
        return F.log_softmax(h, dim=1)
    def fit(self, data, epochs):
        criterion = torch.nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(self.parameters(), lr=0.01, weight_decay=0.01)
        self.train()
        for epoch in range(epochs+1):
            optimizer.zero_grad()
            out = self(data.x, data.edge_index)
            loss = criterion(out[data.train_mask], data.y[data.train_mask])
            acc = accuracy(out[data.train_mask].argmax(dim=1), data.y[data.train_mask])
            loss.backward()
            optimizer.step()
            if(epoch % 20 == 0):
                val_loss = criterion(out[data.val_mask], data.y[data.val_mask])
                val_acc = accuracy(out[data.val_mask].argmax(dim=1), data.y[data.val_mask])
                print(f'Epoch {epoch} | Train Loss:{loss:.3f} | Val Loss:{val_loss:.2f} | Val Acc: {val_acc*100:.2f}%')
        @torch.no_grad()
    def test(self, data):
        self.eval()
        out = self(data.x, data.edge_index)
        acc = accuracy(out.argmax(dim=1)[data.test_mask], data.y[data.test_mask])
        return acc

gat = GAT(dataset.num_features, 32, dataset.num_classes)
gat.fit(data, epochs=100)

```

6.3.3.Сэмплирование и агрегирование (GraphSAGE)

Идея GraphSAGE(Graph Sample and AGgregatE): устранить переполнение при глубоких сетях в плотных графах [<https://arxiv.org/abs/1706.02216>].

Основная идея модели - вместо усреднения по всем соседям сэмплируем их фиксированное число (например, 5). Вторым ее отличием является последовательность операций - сначала применяется агрегация (aggregation) - собираются признаки соседей, а

затем - message passing (передача сообщений): полученное агрегированное представление комбинируется с собственным признаком узла.

Алгоритм имеет вид:

```
for  $k \in [1..K]$  do
  for  $v \in G$  do
     $h_v^k = \phi^o(W^k \text{CONCAT}(h_v^{k-1}, \text{AGGREGATE}_k(\{h_u^{k-1}, \forall u \in N(v)\})))$ 
  end
   $h_v^k = h_v^k / \|h_v^k\|^2, v \in G$ 
end
output =  $H^K$ 
```

здесь AGGREGATE_k - K обучаемых агрегирующих функций.

В GraphSAGE главный параметр - метод агрегации (aggregation method). В качестве агрегации может выступать:

среднее:

$$h_v^{agg} = \text{mean}(\{MLP(h_u) | u \in \text{sample}(N(v))\})$$

max-pooling:

$$h_v^{agg} = \max(\{MLP(h_u) | u \in \text{sample}(N(v))\})$$

LSTM:

$$h_v^{agg} = \text{LSTM}(\text{PERTURB}(\{h_u | u \in \text{sample}(N(v))\}))$$

В режиме LSTM признаки соседей подаются в случайном порядке (PERTURB, чтобы обеспечить инвариантность к перестановкам), затем берём скрытое состояние LSTM как результат агрегации.

Кроме того GraphSAGE выполняет сэмплирование соседей (sample): на каждом шаге из всех соседей узла v случайно выбирается фиксированное подмножество соседних узлов (например, 10 штук), независимо от $\deg(v)$. Это делает модель масштабируемой и особенно эффективной на плотных графах (где у многих узлов сотни/тысячи соседей).

Это похоже на GAT, но без механизма внимания - веса соседей вычисляются не динамически, а задаются агрегирующей функцией. GAT и GraphSAGE - два мощных подхода для задач уровня узла. GAT использует внимание, чтобы взвешивать вклад соседей; GraphSAGE - комбинацию сэмплирования + агрегации.

Алгоритм 30. Использование сети с GraphSAGE

```
from torch_geometric.datasets import Planetoid
import torch
import torch.nn.functional as F
from torch_geometric.nn import SAGEConv

dataset = Planetoid(root='.', name="Pubmed")
data = dataset[0]

def accuracy(pred_y, y):
    return ((pred_y == y).sum() / len(y)).item()

class GraphSAGE(torch.nn.Module):
    def __init__(self, dim_in, dim_h, dim_out):
        super().__init__()
        self.sage1 = SAGEConv(dim_in, dim_h, aggr='max')
        self.sage2 = SAGEConv(dim_h, dim_out, aggr='max')
    def forward(self, x, edge_index):
        h = self.sage1(x, edge_index)
        h = torch.relu(h)
```

```

    h = self.sage2(h, edge_index)
    return h
def fit(self, data, epochs):
    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(self.parameters(),lr=0.01)
    self.train()
    for epoch in range(epochs+1):
        total_loss, val_loss, acc, val_acc = 0, 0, 0, 0
        optimizer.zero_grad()
        out = self(data.x, data.edge_index)
        loss = criterion(out[data.train_mask],data.y[data.train_mask])
        total_loss += loss
        acc += accuracy(out[data.train_mask].argmax(dim=1), data.y[data.train_mask])
        loss.backward()
        optimizer.step()
    @torch.no_grad()
    def test(self, data):
        self.eval()
        out = self(data.x, data.edge_index)
        acc = accuracy(out.argmax(dim=1)[data.test_mask],data.y[data.test_mask])
        return acc

graphsage = GraphSAGE(dataset.num_features, 64, dataset.num_classes)
graphsage.fit(data, 200)
acc = graphsage.test(data)

```

6.3.4. Вариационный графовый автоэнкодер (VGAE)

Вариационный графовый автоэнкодер [<https://arxiv.org/pdf/1611.07308>] часто используется при решении задачи прогноза существования ребра (u, v). Чем он отличается от обычного GAE?

Обычный графовый автоэнкодер (GAE) состоит из:

- encoder: $Z = f_{\theta}(G)$ - детерминированное отображение графа в эмбединги;
- decoder: $p(u, v) = \sigma(Z_u^T Z_v)$ - вероятность связи;
- loss: бинарная кросс-энтропия между p и истинными метками (1 - ребро есть, 0 - нет).

VGAE делает эмбединги стохастическими.

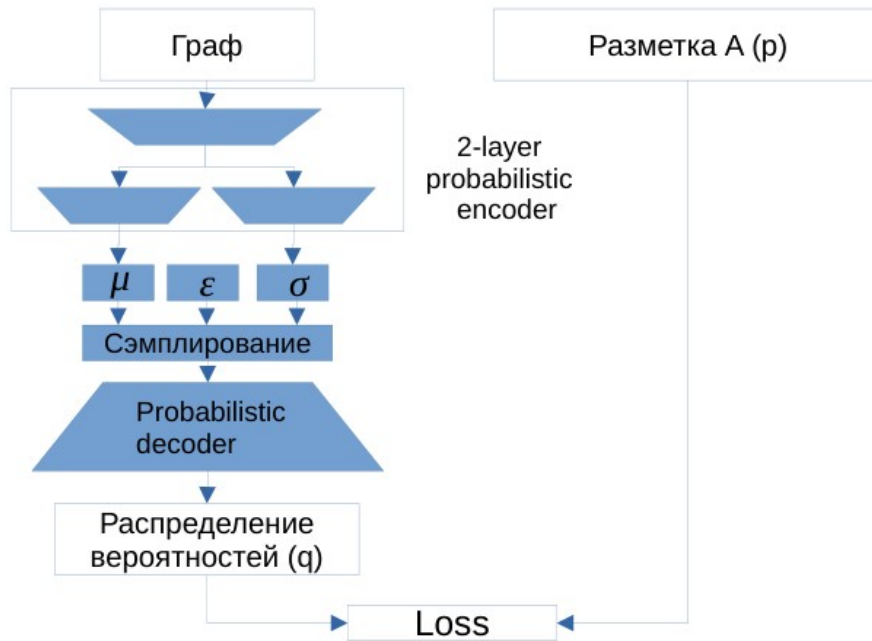


Рис.28 Архитектура графового вариационного автоэнкодера и его обучения

Архитектура encoder'a:

1. Общий слой (например, GCN): $h = \text{GCN}(X, G)$;
2. Два независимых линейных слоя:
 - $\mu = \text{Linear}_{\mu}(h)$ - предсказывает среднее нормального распределения;
 - $\log \sigma^2 = \text{Linear}_{\sigma}(h)$ - предсказывает логарифм дисперсии.

Никаких активаций после Linear_{μ} и Linear_{σ}

3. Сэмплирование:

Генерация эмбединга для ноды v из нормального распределения с заданным средним и среднеквадратичным отклонением:

$$Z_v = \mu_v + \varepsilon \sigma_v, \text{ где } \varepsilon \sim N(0, I)$$

На каждой эпохе - разные Z_v , даже при одинаковом входе $(\mu, \log \sigma^2)$ за счет случайности ε . Это способствует регуляризации и борьбе с переобучением.

Loss в VGAE:

$$L = (1 - \beta) L_{recon} + \beta \cdot KL$$

где β - некий вес (0.5).

здесь L_{recon} - потери реконструкции, чаще всего бинарная кросс-энтропия:

$$L_{recon} = - \sum_{(u,v) \in E^{ps}} \log \sigma(Z_u^T Z_v) - \sum_{(u,v) \in E^{ng}} \log (1 - \sigma(Z_u^T Z_v))$$

а суммирование идет по позитивным (E^{ps}) и негативным (E^{ng}) примерам ребер.

KL - дивергенция Кульбака-Лейблера между аппроксимирующим распределением $q(Z|X, G)$ и априорным $p(Z) = N(0, I)$:

$$KL = (1/2) \cdot \sum_{i=1}^d (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2)$$

Эта регуляризация штрафует отклонение эмбедингов от стандартного нормального распределения, заставляя схожие ноды иметь близкие μ, σ , т.е. перекрывающиеся распределения.

Алгоритм 31. Использование сети с вариационным графовым автоэнкодером

```

import numpy as np
import torch
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.utils import negative_sampling, train_test_split_edges
from torch_geometric.nn import GCNConv, VGAE

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
dataset = Planetoid(root='/tmp/Cora', name='Cora')
data = dataset[0]
data = train_test_split_edges(data, val_ratio=0.05, test_ratio=0.1)
train_edge_index = data.train_pos_edge_index
x = data.x.to(device)
train_pos_edge_index = data.train_pos_edge_index.to(device)
train_edge_type = torch.zeros(train_pos_edge_index.size(1), dtype=torch.float, device=device)

class GEncoder(torch.nn.Module):
    def __init__(self, num_features, hidden_channels):
        super().__init__()
        self.conv1_shared = GCNConv(num_features, hidden_channels)
        self.conv2_mean = GCNConv(hidden_channels, hidden_channels)
        self.conv2_logstd = GCNConv(hidden_channels, hidden_channels)
    def forward(self, x, edge_index, edge_type):
        mean1 = F.relu(self.conv1_shared(x, edge_index, edge_type))
        mean = self.conv2_mean(mean1, edge_index, edge_type)
        logstd = self.conv2_logstd(mean1, edge_index, edge_type)
        return mean, logstd
model = VGAE(GEncoder(dataset.num_features, 64)).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
def train():
    model.train()
    optimizer.zero_grad()
    pos_edge_index = data.train_pos_edge_index.to(device)
    z = model.encode(x, pos_edge_index, train_edge_type)
    loss = model.recon_loss(z, pos_edge_index)
    loss = loss + (1 / data.num_nodes) * model.kl_loss()
    loss.backward()
    optimizer.step()
    return loss.item()
for epoch in range(1, 201):
    loss = train()
    print(loss)

```


7.Решение задач уровня графа нейронными сетями

Задачи уровня графа обычно сводятся к прогнозу по графу каких-то величин: категориальных (классификация или кластеризация) или числовых (регрессия). Основная идея при решении таких задач - построить такое отображение графа в его векторное представление (эмбеddинг), при котором разные графы вычислений (computational graphs) получают разные эмбеddинги, а одинаковые графы вычислений — одинаковые эмбеddинги.

7.1.Простые способы кодирования графа

Самый простой способ построения эмбеddинга для графа вычислений - **конкатенация состояний соседей**:

$$H_v = [||_{u \in N(v)} H_u]$$

Его минус в том, что оно не инвариантно к перестановке соседей, и при перестановке номеров нод даст другое представление графа: $[1, 0] \neq [0, 1]$, хотя соседи те же - просто порядок другой.

Его плюс - оно зависит от числа соседей и позволяет отличать графы разных размеров: например для 2 узлов с эмбеddингами 1 и 0 итоговый эмбеddинг будет $[1, 0]$, а для 4х узлов эмбеddинг будет иметь вид $[1, 0, 1, 0]$. Это разные вектора (у них разная размерность), даже если пропорции эмбеddингов нод в графах одинаковы.

Неинвариантность к перестановке номеров нод приводит к тому, что конкатенация не подходит построения эмбеddинга для графов.

Часто используемый метод построения эмбеddинга для графа - это **усреднение соседей**:

$$H_v = \frac{1}{|N(v)|} \sum_{u \in N(v)} H_u$$

Его плюс в том, что он инвариантен к перестановке аргументов. Минус в том что он инвариантен к изменению числа аргументов (при сохранении пропорций признаков узлов):

Ситуация А: 1 узел с эмбеddингом «0», 1 узел с эмбеddингом «1» дают эмбеddинг для графа (среднее) 0.5;

Ситуация Б: 2 узла с эмбеddингом «0», 2 узла с эмбеddингом «1» дают эмбеddинг для графа тоже 0.5.

Таким образом, разные графы в результате имеют одинаковый эмбеddинг. Несмотря на удобство, оно не сохраняет кратность - теряет информацию о числе соседей и может плохо сравнивать графы разных размеров.

Другой часто используемый метод - это **максимальный пуллинг**:

$$H_v = \max_{u \in N(v)} H_u$$

Его плюс в том, что он инвариантен инвариантно к перестановке аргументов.

Его минус в инвариантности к изменению числа аргументов (при сохранении максимального значения признака узлов):

Ситуация А: $[0, 1] : \max = 1$,

Ситуация Б: $[0, 0, 1, 1] : \max = 1$.

Разные по размеру графы в результате могут иметь одинаковый эмбеddинг и метод может не различать графы различных размеров.

Алгоритм 32. Определение эмбеddинга графа методом усреднения нод

```

import torch
import numpy as np
from torch_geometric.datasets import TUDataset
from torch_geometric.utils import to_networkx
from torch.nn import Linear, Sequential, BatchNorm1d, ReLU, Dropout
import torch.nn.functional as F
from torch_geometric.nn import GCNConv, global_mean_pool
from torch_geometric.loader import DataLoader
dataset = TUDataset(root='.', name='PROTEINS').shuffle()
G = to_networkx(dataset[2], to_undirected=True)
train_dataset = dataset[:int(len(dataset)*0.8)]
val_dataset = dataset[int(len(dataset)*0.8):int(len(dataset)*0.9)]
test_dataset = dataset[int(len(dataset)*0.9):]
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
class GCN(torch.nn.Module):
    def __init__(self, dim_h):
        super(GCN, self).__init__()
        self.conv1 = GCNConv(dataset.num_node_features, dim_h)
        self.conv2 = GCNConv(dim_h, dim_h)
        self.conv3 = GCNConv(dim_h, dim_h)
        self.lin = Linear(dim_h, dataset.num_classes)
    def forward(self, x, edge_index, batch):
        h = self.conv1(x, edge_index).relu()
        h = self.conv2(h, edge_index).relu()
        h = self.conv3(h, edge_index)
        self.hG = global_mean_pool(h, batch)
        h = F.dropout(self.hG, p=0.5, training=self.training)
        h = self.lin(h)
        return self.hG, F.log_softmax(h, dim=1)
def accuracy(pred_y, y):
    return ((pred_y == y).sum() / len(y)).item()
def train(model, loader):
    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=0.01)
    epochs = 10
    model.train()
    for epoch in range(epochs):
        total_loss, acc = 0, 0
        for data in loader:
            optimizer.zero_grad()
            Gemb, out = model(data.x, data.edge_index, data.batch)
            loss = criterion(out, data.y)
            total_loss += loss / len(loader)
            acc += accuracy(out.argmax(dim=1), data.y) / len(loader)
            loss.backward()
            optimizer.step()
        return model
gcn = GCN(dim_h=32)
gcn = train(gcn, train_loader)
for data in test_loader:
    Gemb, out = gcn(data.x, data.edge_index, data.batch)
    print(Gemb[0])

```

7.2. Сеть графового изоморфизма (GIN)

Наилучшее с точки зрения различения графов отображение графа в пространство эмбедингов - это инъекция (взаимно-однозначное отображение). Это такое отображение, при котором:

- если два графа различны, то их эмбединги различны,
- если графы имеют одинаковые эмбединги, то графы одинаковы.

Инъекция сохраняет различимость структур. Доказано [https://arxiv.org/pdf/1810.00826] что отображение инъективно тогда и только тогда, когда отображение имеет вид:

$$Z = [H^{(0)}, H^{(1)}, \dots, H^{(K)}],$$

где

$$H_v^k = \varphi(H_v^{k-1}, \sum_{u \in N(v)} \psi(H_u^{k-1}))$$

а φ и ψ - две нелинейные функции (внешняя и внутренняя соответственно). Этот алгоритм реализуется через сеть графового изоморфизма (GIN, Graph Isomorphism Network) - одну из самых выразительных графовых архитектур для различения изоморфных графов. Она описана в [https://arxiv.org/pdf/1810.00826].

Базовый k-й слой GIN (k-я итерация алгоритма) имеет вид:

$$H_v^{(k)} = MLP^{(k)}((1 + \epsilon^{(k)}) H_v^{(k-1)} + \sum_{u \in N(v)} H_u^{(k-1)})$$

где:

- MLP - многослойный перцептрон с нелинейной активацией (например, ReLU),
- ϵ - скаляр (обучаемый или фиксированный).

Для кодирования графа используется результат конкатенации выходов нескольких слоев. Количество слоев в GIN не превышает диаметр графа.

Это непрерывный аналог преобразования Вейсфейлера-Лемана (Weisfeiler-Lehman, WL), где хэш заменён на MLP и при подходящей мощности MLP он аппроксимирует инъективное отображение.

GIN - одна из самых выразительных графовых архитектур для различения изоморфных графов. Важные требования для инъективности отображения:

1. Нелинейность (MLP) обязательна: линейные функции не различают симметрий, например, [1,0] и [0,1] при усреднении, а нелинейность (ReLU, sigmoid и др.) “ломает” эти симметрии.
2. Самосвязь (self-loop) нужна чтобы узел учитывал своё собственное состояние, а не только состояния соседей. Формально для этого мы добавляем единичную матрицу E к матрице смежности:

$$\tilde{A} = A + E$$

или умножаем H_v на $(1 + \epsilon)$, как в GIN.

3. Многомерные признаки H_v предпочтительны, с одной бинарной фичей различимость ограничена. Поэтому к признакам ноды часто добавляют:

- фейковую константную фичу (всем узлам = 1);
- топологические признаки (степень, PageRank, кластерность);
- one-hot ID (для малых графов).

Чем больше измерений у признаков - тем проще инъективное отображение.

Алгоритм 33. Определение эмбединга графа сетью графового изоморфизма

```
from torch_geometric.nn import GINConv
class GIN(torch.nn.Module):
    def __init__(self, dim_h):
        super(GIN, self).__init__()
        self.conv1 = GINConv(
            Sequential(Linear(dataset.num_node_features, dim_h), BatchNorm1d(dim_h), ReLU(),
                Linear(dim_h, dim_h), ReLU()))
```

```

self.conv2 = GINConv(
    Sequential(Linear(dim_h, dim_h), BatchNorm1d(dim_h), ReLU(), Linear(dim_h, dim_h), ReLU()))
self.conv3 = GINConv(
    Sequential(Linear(dim_h, dim_h), BatchNorm1d(dim_h), ReLU(), Linear(dim_h, dim_h), ReLU()))
self.lin1 = Linear(dim_h*3, dim_h*3)
self.lin2 = Linear(dim_h*3, dataset.num_classes)
def forward(self, x, edge_index, batch):
    h1 = self.conv1(x, edge_index)
    h2 = self.conv2(h1, edge_index)
    h3 = self.conv3(h2, edge_index)
    h1 = global_add_pool(h1, batch)
    h2 = global_add_pool(h2, batch)
    h3 = global_add_pool(h3, batch)
    h = torch.cat((h1, h2, h3), dim=1)
    self.Gemb=h
    h = self.lin1(h).relu()
    h = F.dropout(h, p=0.5, training=self.training)
    h = self.lin2(h)
    return self.Gemb, F.log_softmax(h, dim=1)

gin = GIN(dim_h=32)
gin = train(gin, train_loader)
for data in test_loader:
    _,out=gin(data.x,data.edge_index, data.batch)
    print(gin.Gemb[0])

```

8. Неоднородные графы

Изучавшиеся нами ранее однородные (homogeneous) графы обладают следующими основными свойствами:

- Все рёбра одного типа (например, «ссылка» в интернете),
- Описывается одной матрицей смежности A .

Для решения многих задач кроме однородных могут использоваться неоднородные графы. Неоднородный граф это граф, в котором могут быть ребра и ноды разных типов. Таким образом, свойства неоднородного графа (гетерогенный, heterogeneous):

- Несколько типов рёбер, например «просмотр», «покупка», «оценка» - в рекомендациях; «лечит», «состоит из» - в графах знаний; «переход», «регистрация», «оплата» - в веб-аналитике.
- Описывается множеством матриц смежности:

$$\{A^{(1)}, A^{(2)}, \dots, A^{(R)}\}$$

где R - число типов рёбер.

Примеры однородного и неоднородного графов приведены на Рис.29.

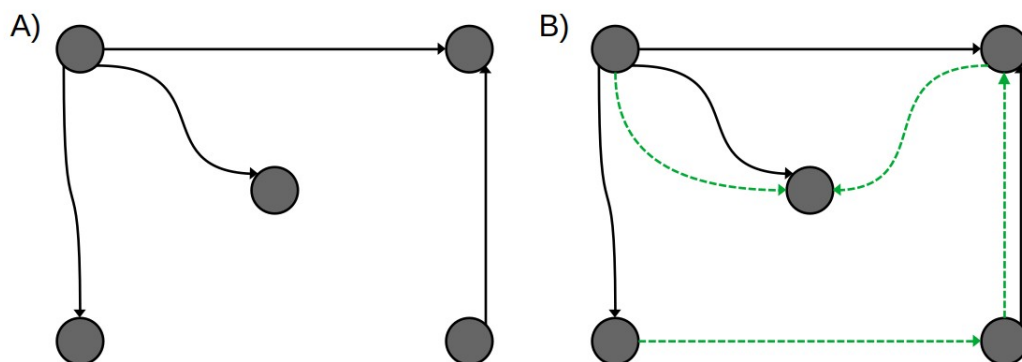


Рис.29 Пример однородного (A) и неоднородного (B) графов

Узлы в неоднородном графе также могут быть разного типа («пользователь», «товар», «болезнь», «препарат»).

Основная проблема работы с неоднородными графами: стандартные GNN архитектуры (GCN, GAT) работают только с однородными графами - один тип нод, один тип рёбер, одна матрица смежности при агрегации нод.

8.1. Графовая сверточная нейронная сеть отношений (R-GCN)

Первый простой способ учесть множественность типов рёбер - это графовая сверточная нейронная сеть отношений (Relation Graph Convolutional Network, R-GCN) [https://arxiv.org/abs/1703.06103]. Основная идея метода - при агрегации признаков соседей сделать веса зависимыми от типа ребра. Обработать каждый тип рёбер отдельно, затем комбинировать.

Это работает следующим образом. Пусть у ноды v есть соседи по разным типам связей: например, r_1 : «снялся в», r_2 : «режиссёр», r_3 : «жанр» и т.д.

В стандартной GCN агрегация выглядит так:

$$h_v^{(l+1)} = \varphi(W_0 h_v + W^{(l)} \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(l)})$$

В R-GCN она переписывается как:

$$h_v^{(l+1)} = \varphi(W_0 h_v + \sum_{r \in R} W_r^{(l)} \frac{1}{|N_r(v)|} \sum_{u \in N_r(v)} h_u^{(l)})$$

где: R - множество всех типов рёбер; $N_r(v)$ - соседи v по связи типа r ; $W_r^{(l)}$ - своя обучаемая матрица весов для каждого типа r . W_0 - обучаемая матрица весов обратной связи.

То есть каждый тип связи обрабатывается независимой нейронной подсетью, а результаты суммируются в единое новое представление ноды. Фактически, каждый тип связи представляется как отдельный графовый слой (подграф), а выходы всех слоёв складываются. То есть для каждого типа связи выполняется отдельный message passing (передача через W_r), а затем - единая агрегация (суммирование по всем типам связей).

8.2.Обучение R-GCN

При обучении используется модифицированная бинарная кросс-энтропия с негативным сэмплингом в качестве функции потерь.

Пусть:

- E^{ps} - множество позитивных (скрытых, но реальных) рёбер;
- E^{ng} - множество негативных (искусственных) рёбер.

Для пары нод (u, v) модель выдаёт логит связи s_{uv} (например, скалярное произведение эмбеддингов: $s_{uv} = Z_u Z_v^T$).

Тогда loss:

$$L = - \sum_{(u,v) \in E^{ps}} \log(\sigma(s_{uv})) - \sum_{(u,v) \in E^{ng}} \log(1 - \sigma(s_{uv}))$$

Негативное сэмплирование - это генерация E^{ng} . Часто берут: для каждого позитивного ребра (u, v) - k негативных (u, v') , где v' - случайная нода, не сосед u .

Аналогично softmax - при большом N полную сумму считать трудоемко и сэмплируем негативные ребра:

- вместо

$$p(u \rightarrow v) = \frac{\exp(s(u, v))}{\sum_{k=1}^N \exp(s(u, k))}$$

- используем приближение:

$$\tilde{p}(u \rightarrow v) = \frac{\exp(s(u, v))}{\exp(s(u, v)) + \sum_{k=1}^K \exp(s(u, v'_k))}$$

где K - малое число (5-10), v'_k - негативные сэмплы.

Это эквивалентно усечённому Softmax и делает обучение вычислительно реальным.

Алгоритм 34. Пример обучения RGCN

```
from torch_geometric.nn import RGCNConv
from torch_geometric.utils import negative_sampling, train_test_split_edges

data = dataset[0]
```

```

data = train_test_split_edges(data, val_ratio=0.05, test_ratio=0.1)
train_edge_index = data.train_pos_edge_index
num_relations = 1
def get_link_labels(pos_edge_index, neg_edge_index):
    E = pos_edge_index.size(1) + neg_edge_index.size(1)
    link_labels = torch.zeros(E, dtype=torch.float, device=device)
    link_labels[:pos_edge_index.size(1)] = 1.
    return link_labels
class RGCNLinkPredictor(torch.nn.Module):
    def __init__(self, num_features, hidden_channels, num_relations):
        super().__init__()
        self.convs = torch.nn.ModuleList()
        self.convs.append(RGCNConv(num_features, hidden_channels, num_relations))
        self.convs.append(RGCNConv(hidden_channels, hidden_channels, num_relations))
    def encode(self, x, edge_index, edge_type):
        x = self.convs[0](x, edge_index, edge_type)
        x = F.relu(x)
        x = F.dropout(x, p=0.5, training=self.training)
        x = self.convs[1](x, edge_index, edge_type)
        return x
    def decode(self, z, pos_edge_index, neg_edge_index):
        edge_index = torch.cat([pos_edge_index, neg_edge_index], dim=-1)
        logits = (z[edge_index[0]] * z[edge_index[1]]).sum(dim=-1)
        return logits

model = RGCNLinkPredictor(num_features=dataset.num_features,
    hidden_channels=64, num_relations=num_relations).to(device)
x = data.x.to(device)
train_pos_edge_index = data.train_pos_edge_index.to(device)
train_edge_type = torch.zeros(train_pos_edge_index.size(1), dtype=torch.long, device=device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

def train(model):
    model.train()
    neg_edge_index = negative_sampling(edge_index=train_pos_edge_index,
        num_nodes=data.num_nodes, num_neg_samples=train_pos_edge_index.size(1)).to(device)
    z = model.encode(x, train_pos_edge_index, train_edge_type)
    logits = model.decode(z, train_pos_edge_index, neg_edge_index)
    link_labels = get_link_labels(train_pos_edge_index, neg_edge_index)
    loss = F.binary_cross_entropy_with_logits(logits, link_labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()
for epoch in range(1, 201):
    loss = train(model)

```

8.3. Особенности разделения датасетов

При работе с неоднородными графами (содержащими много типов рёбер) разделение на обучающую/валидационную/тестовую выборки требует выполнения двух условий:

1. Независимое разбиение по каждому типу связи. Нельзя просто разбить по нодам: при фиксированных нодах некоторые типы связей могут исчезнуть из обучающей части, поэтому разбивать надо отдельно для каждого типа ребра r .
2. Стратификация по типам связей. Пропорции появления каждого типа ребра (r) в обучающей / валидационной / тестовой выборках должны быть примерно

одинаковыми. Иначе может получиться ситуация, когда модель обучится на частых ребрах, а тестировать будут на редких ребрах.

8.4. Проблема масштабируемости R-GCN

Реальные неоднородные графы, например графы знаний, характеризуются следующими свойствами:

- число узлов N - огромно (миллионы-миллиарды);
- число фич на узел d - велико (часто тысячи);
- число типов рёбер R - велико (сотни-тысячи).

В R-GCN-слое участвует матрица весов размером $d \times d \times R$, поэтому объём вычислений растёт кубически с ростом этих параметров. Такая архитектура практически неприменима к большим графам знаний. Поэтому R-GCN работает только на малых графах или с небольшим числом типов связей ($|R| \leq 10 - 20$).

Для более крупных неоднородных графов нужно использовать эффективные способы уменьшения размерности этих матриц.

Способ 1. Блочно-диагональная матрица

Разбиваем признаки на B блоков (например, по смыслу). Тогда матрица W_r - блочно-диагональная:

$$W_r = \text{block}_{diag}(W_r^{(1)}, W_r^{(2)}, \dots, W_r^{(B)})$$

и мы уменьшаем межблоковые взаимодействия в матрице и, тем самым, количество вычислений.

Способ 2: Понижение размерности

Идея состоит в том, чтобы весовые матрицы W_r для различных типов ребер можно представить как линейную комбинацию нескольких базисных матриц:

$$W_r \approx \alpha_{r,0} B_0 + \alpha_{r,1} B_1 + \dots + \alpha_{r,K-1} B_{K-1}$$

где $B_0, \dots, B_{K-1}$ - общие для всех типов «базисные» обучаемые матрицы; $\alpha_{r,k}$ - скалярные коэффициенты для типа r , тоже обучаемые.

Тогда вместо $|R|$ матриц весов W_r остаётся K базисных матриц B_k ($K \ll |R|$, например, 4-5); и $|R| \cdot K$ коэффициентов $\alpha_{r,k}$. Это приводит к гораздо меньшему количеству параметров.

9. Графы знаний

9.1. Особенности задач на графах знаний

Одним из примеров неоднородных графов является граф знаний. Граф знаний можно рассматривать как несколько графов, построенных по одним и тем же нодам.

Графы знаний по большей части ориентированные: утверждение «актер X снялся в фильме Y» не равно утверждению «фильм Y снялся в актере X».

Главная проблема всех графов знаний - их существенная неполнота, они содержат лишь часть истинных фактов. Задача модели, работающей на графе знаний - восстановить отсутствующие связи, используя структуру и известные факты, то есть дополнять граф знаний новыми ребрами (необходимые ноды в графе знаний обычно присутствуют) или дополнять отсутствующие признаки нод.

Другая проблема графов знаний заключается в их большом объеме:

- Очень большое число нод (объектов - людей, фильмов, организаций и т.д.);
- Очень большое число рёбер (связей между нодами);
- Очень большое число типов рёбер.

Традиционные задачи на графах знаний часто отличаются от задач на обычных графах: во многих задачах на графах знаний необходимо определить не то, есть ли ребро между нодами u и v , а то, какие узлы v наиболее вероятно связаны с нодой u через тип ребра r . Для решения этой задачи методом полного перебора нужно проверить все узлы, а это неэффективно. Поэтому требуются эффективные методы поиска ответа по графу.

В качестве примера можно рассматривать небольшой фрагмент графа знаний - датасет Freebase 15k (FB15k). Характеристики датасета:

- примерно 15 000 нод (объектов);
- 237 типов связей (отношений);
- ~300 000 рёбер (фактов).

Алгоритм 35. Загрузка датасета FB15k
<pre>import torch from pykeen.datasets import FB15k237 dataset = FB15k237() print(f"Entities: {dataset.num_entities}") print(f"Relations: {dataset.num_relations}") print(f"Training triples: {dataset.training.num_triples}") print(f"Validation triples: {dataset.validation.num_triples}") print(f"Test triples: {dataset.testing.num_triples}")</pre>
Entities: 14505 Relations: 237 Training triples: 272115 Validation triples: 17526 Test triples: 20438

9.2. Типы отношений в графах знаний

Для выбора адекватной модели графовой сети важно понимать, какими свойствами могут обладать отношения, которые мы пытаемся анализировать. Часто их делят на следующие базовые типы:

1. Симметричные отношения

$$(h, r, t) \Rightarrow (t, r, h)$$

здесь h - головной нод (head), t - хвостовой нод (tail); r - тип отношения (ребра, relation).

Для симметричных отношений характерно то, что две ноды связаны этим отношением и в одну, и в другую сторону. Симметричное отношение эквивалентно ненаправленному ребру. Примером может являться соотношение “является соседом”: если A - сосед B , то и B - сосед A .

Если за $score((h, r), t)$ принимать меру расстояния от вопроса (h, r) до ответа t , то на языке расстояний для симметричного отношения выполняется условие:

$$score((h, r), t) = score((t, r), h).$$

2. Антисимметричные отношения

$$(h, r, t) \Rightarrow \neg(t, r, h)$$

Смысл его в том, что если (h, r, t) истинно, то (t, r, h) — ложно. Примером может являться отношение “больше”: если A - больше B , то B - не больше A . На языке расстояний это может быть сформулировано, как выполнение условия

$$score((h, r), t) \ll score((t, r), h).$$

3. Обратные отношения

$$(h, r, t) \Rightarrow (t, r^{-1}, h)$$

Обратные отношения подразумевают, что если заданное отношение r связывает голову h и хвост t , то существует обратное ему отношение r^{-1} , в этом случае всегда связывающее хвост t и голову h . Примером обратных отношений может быть отношения “родитель”(тип ребра r) и “ребёнок”(тип ребра r^{-1}): если A - родитель B , то B - ребенок A . На языке расстояний это формулируется, как

$$score((h, r), t) = score((t, r^{-1}), h)$$

4. Композиционные (транзитивные) отношения

$$(h, r_1, t) \wedge (t, r_2, u) \Rightarrow (h, r_3, u)$$

Композиционные соотношения подразумевают, что если объект A связан с объектом B соотношением r_1 , а объект B связан с объектом C соотношением r_2 , то объект A обязательно связан с объектом C отношением r_3 , которое называется композицией отношений r_1 и r_2 . Например, если “Коля - отец(r_1) Оли”, “Оля - мать(r_2) Пети” то “Коля - дед(r_3) Пети”.

На языке расстояний требуется, чтобы

$$score((h, r_1), t) + score((t, r_2), u) = score((h, r_3), u)$$

5. Отношения “один-ко-многим” (1-to-N)

$$\exists t_1, t_2: (h, r, t_1), (h, r, t_2)$$

Эти отношения подразумевают, что для головы h может существовать несколько хвостов t_1, t_2 . Например: A имеет детей B, C, D или A - автор статей B, C, D . На языке расстояний это можно сформулировать, как

$$score((h, r), t_1) = score((h, r), t_2)$$

Разные модели по-разному справляются с прогнозом различных типов отношений, и это определяет области их применимости.

9.3. Задача прогноза в графе знаний

Простейшая задача прогноза на графе знаний имеет вид прогноза ближайших к заданной ноде нод по ребру заданного типа (еще не соединенных ребрами этого типа) или комбинации таких прогнозов. Она может например формулироваться так: “С кем ещё, кроме А, Б и В, снимался этот актёр?”. Она может например формулироваться так: “Найти человека, который снимался в фильмах с актерами А, Б и В”. Это задачи из области self-supervised learning (самообучения) — обучение на примерах, которые извлекаются из самих данных.

Как организуется самообучение? Вначале из графа удаляются (временно скрываются) некоторые рёбра - это supervision edges (контрольные рёбра). После этого на примере контрольных ребер модель обучается предсказывать неизвестные (скрытые) рёбра на позитивных примерах ребер, которые существуют, и на негативных примерах ребер, которые не существуют (негативные примеры обычно выбираются, как случайно сгенерированные пары нод, не соединенные ребром).

Обученная модель должна предсказывать высокую вероятность ребра (высокую степень близости нод) для первых примеров и низкую - для вторых.

9.3. Translating Embeddings (TransE)

9.3.1. Идея метода

Рассмотрим метод TransE (от англ. Translating Embeddings), применяемый для работы с графами знаний [Bordes et al, 2013. Translating embeddings for modeling multi-relational data// In Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2 (NIPS'13), Vol. 2. Curran Associates Inc., Red Hook, NY, USA, pp.2787-2795]. Цель алгоритма - быстро находить ответы на вопросы, связанные с нодами в графе знаний.

Прямой перебор всех узлов с помощью нейронной сети — вычислительно крайне затратная операция. Поэтому TransE предлагает более эффективный способ: использовать векторные представления (эмбеддинги) для узлов и типов рёбер, а затем применять простую арифметику над этими векторами для поиска ответов.

Идея алгоритма - представить каждую ноду и каждый тип связи как вектор (эмбеддинг) в одном и том же векторном пространстве R^d размерности d :

$$H_h, H_t, H_r \in R^d$$

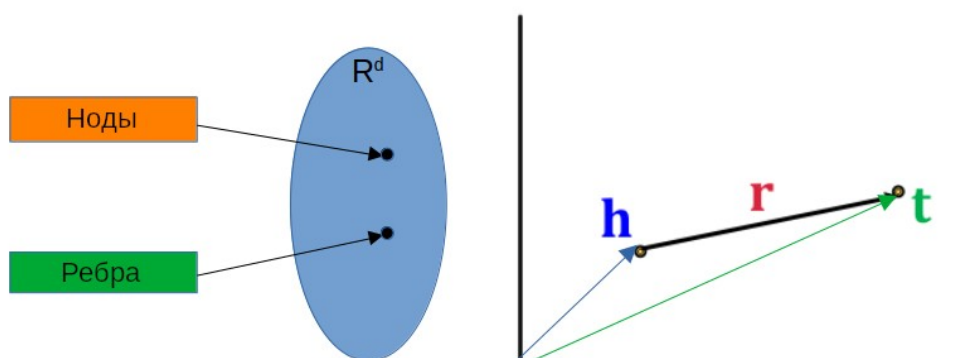


Рис.30. Идея алгоритма TransE

Базовое предположение метода - для истинного факта (h, r, t) должно выполняться примерное равенство:

$$Z_h + Z_r \approx Z_t$$

То есть, если к эмбедингу ноды h прибавить эмбединг типа отношения r, мы должны прийти близко к эмбедингу ноды t. Похожая аналогия существует в алгоритме Word2Vec: $Z_{king} - Z_{man} + Z_{woman} \approx Z_{queen}$.

Для ответа на вопрос: “Какой узел связан с нодой h через отношение r?” алгоритм:

- вычисляет вектор $Z_h + Z_r$;
- сравнивает его с эмбедингами всех узлов в графе Z_v ;
- возвращает тот t, чей эмбединг Z_t ближе всего (например, по евклидову расстоянию).

Можно также ранжировать все узлы и выдавать топ-K кандидатов.

Функция оценки истинности (scoring function) будет стандартной нормой:

$$score((h, r), t) = -\|Z_h + Z_r - Z_t\|.$$

Чем ближе к нулю норма разности - тем выше скор, тем истиннее факт.

Для обучения модели используются известные тройки (h, r, t) из обучающего графа и оптимизируется функция потерь, штрафующая за: большое расстояние для истинных троек $\|Z_h + Z_r - Z_t\|$; маленькое расстояние для ложных (сгенерированных) троек $\|Z_{h'} + Z_r - Z_t\|, \|Z_h + Z_r - Z_{t'}\|$.

Фактически это сводится к решению задачи одновременного поиска эмбединга для всех нод h, t и типов ребер r.

9.3.2. Обучение модели TransE

Для обучения используется размеченный граф знаний, содержащий тройки вида (h, r, t) - то есть “из узла h по отношению r ведёт ребро к узлу t”.

Процесс обучения:

- Инициализируются эмбединги для всех узлов и всех типов отношений.
- Для каждой известной тройки (h, r, t) модель стремится минимизировать расстояние между $Z_h + Z_r$ и H_t .
- Для негативных примеров (неверных троек) - максимизировать это расстояние.

Поскольку полный перебор всех возможных t неэффективен, используется негативная выборка (negative sampling): для каждой позитивной тройки генерируются 5-10 негативных, где t заменяется на случайный узел, не связанный с h через отношение r.

Функция потерь часто основана на Softmax с негативной выборкой. Например, в упрощённой форме:

$$L(t|(h, r)) = \frac{\exp(score((h, r), t))}{\exp(score((h, r), t)) + \exp(score((h, r), t'))}$$

где t' - негативный кандидат.

9.3.3. Ограничения TransE

TransE может моделировать:

- антисимметричные отношения (благодаря $\vec{r} \neq -\vec{r} | \vec{r} \neq 0$);
- обратные отношения (через $Z_{r^{-1}} = -Z_r$);
- транзитивность (благодаря линейности сложения векторов $\vec{r}_1 + \vec{r}_2 = \vec{r}_3$).

Что TransE не может моделировать:

- симметричные отношения (благодаря $\vec{r} \neq -\vec{r} | \vec{r} \neq 0$);
- 1-to-N отношения (благодаря линейности сложения векторов $\vec{r}_1 + \vec{r}_2 = \vec{r}_3$ и как следствие - однозначности результата).

9.4. Transition in relations space(TransR)

9.4.1.Идея алгоритма

Алгоритм Transition in relations space (TransR)[Lin, Y., Liu, Z., Sun, M., Liu, Y., & Zhu, X. (2015). Learning Entity and Relation Embeddings for Knowledge Graph Completion. *Proceedings of the AAAI Conference on Artificial Intelligence*, 29(1). <https://doi.org/10.1609/aaai.v29i1.9491>] является модификацией TransE, расширяющей область его применимости.

Главная идея метода: пусть ноды h, t и связи r кодируются эмбедингами в разных векторных пространствах.

$$Z_h, Z_t \in R^{D_n}; Z_r \in R^{D_r}$$

Чтобы сложить их и сделать прогноз - сначала спроецируем ноды в пространство связи r . Для проецирования между пространствами, кроме обучаемых эмбедингов Z_h, Z_t, Z_r используются обучаемые матрицы проекций M_r . $M_r \in R^{D_r \times D_n}$ - матрицы проекций из пространства нод в пространство связей, зависящие от типа связи r .

При этом проекция ноды в пространство связи:

$$Z_{h \rightarrow r} = M_r Z_h, Z_{t \rightarrow r} = M_r Z_t, Z_{h \rightarrow r}, Z_{t \rightarrow r} \in R^{D_r}.$$

В этом случае расстояние от истинного ответа до вычисленного (scoring function) вычисляется в пространстве r :

$$\text{score}((h, r), t) = -\|h \rightarrow r + r - t \rightarrow r\| = -\|M_r Z_h + Z_r - M_r Z_t\|$$

Почему это лучше, чем TransE? В добавок к плюсам TransE могут реализовываться отношения 1-to-N (разные Z_t могут проецироваться в одну точку $Z_{t \rightarrow r}$, даже при фиксированных Z_h, Z_r) и симметричные отношения (аналогичные причины).

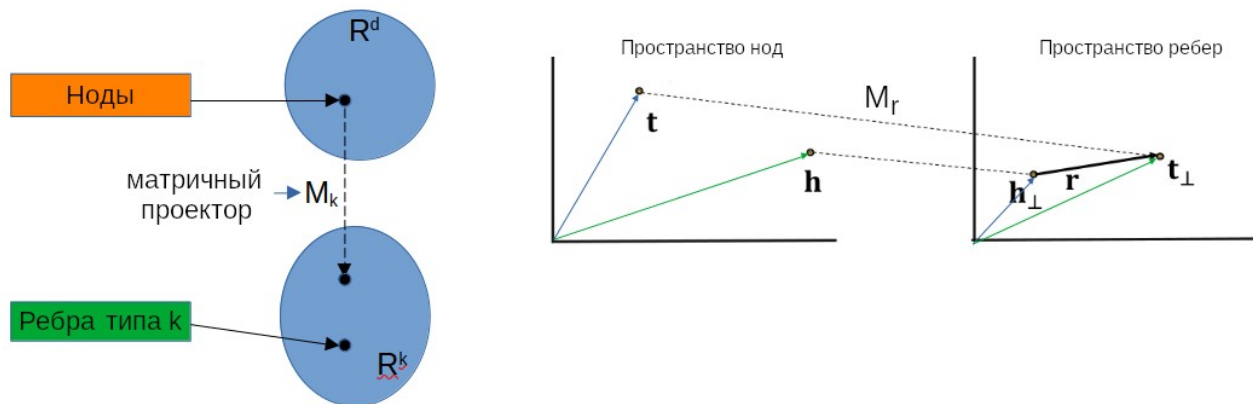


Рис.31. Идея алгоритма TransR

9.4.2.Обучение

1. При обучении используется негативное сэмплирование: позитивные примеры извлекаются из графа: это тройки (h, r, t) , существующие в обучающем графе ($t \in N_r(h)$).
2. Негативные примеры генерируются заменой t на случайную несоседнюю к h по ребру r ноду: (h, r, t') ($t' \notin N_r(h)$) или заменой h на h' : $t \notin N_r(h')$.
3. В качестве функции потерь используется margin-based ranking loss:

$$L = \sum_{(h,r) \in G, t \in N_r(h), t' \notin N_r(h)} \max(0, \gamma + \text{score}(h, r, t') - \text{score}(h, r, t)) \rightarrow \min$$

или

$$L = \sum_{(h,r) \in G, t \in N_r(h), t' \notin N_r(h)} \text{ReLU}(\gamma + \text{score}(h, r, t') - \text{score}(h, r, t)) \rightarrow \min$$

где $\gamma > 0$ - некое смещение (margin), обычно принимаемое за 1, $\text{score}(h, r, t) = -\|Z_r H_h + Z_r - M_r Z_t\|_2^2$ - среднеквадратичное отклонение, $N_r(h)$ - соседи ноды h по отношению r .

9.4.3.Оценка итогового качества

Для каждой позитивной тестовой тройки $(h, r, t) \in Q$:

- фиксируем h, r , перебираем все возможные t' как кандидаты;
- вычисляем $\text{score}(h, r, t')$ для каждого;
- сортируем по убыванию скоров;
- посмотрим, на каком месте оказался истинный ответ t - $\text{rank}(t)$.

Для оценки качества можно использовать метрики:

- Средний обратный ранг (Mean Reciprocal Rank, MRR):

$$\text{MRR} = \frac{1}{|Q|} \sum_{h,r,t \in Q} \frac{1}{\text{rank}(t)}$$

- Доля запросов, где истинный ответ попадает в топ-k ответов сети (Hits@k):

$$\text{Hits@k} = \frac{1}{|Q|} \sum_{h,r,t \in Q} [\text{rank}(t) \leq k]$$

- При бинарной классификационной формулировке задачи (на основе оценки вероятности верных оценок позитивных и негативных ребер) можно использовать AUC-ROC, AUC-PR или F1, но эта метрика намного чувствительнее, чем предыдущие две.

Алгоритм 36. Пример реализации и обучения TransR

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.utils import negative_sampling, train_test_split_edges
from torch_geometric.nn import MessagePassing
from sklearn.metrics import roc_auc_score, average_precision_score
import numpy as np
from torch_geometric.datasets import Planetoid

class TransR(nn.Module):
    def __init__(self, num_entities, num_relations, ent_dim, rel_dim):
```

```

    super().__init__()
    self.num_entities = num_entities
    self.num_relations = num_relations
    self.ent_dim = ent_dim
    self.rel_dim = rel_dim
    self.ent_emb = nn.Embedding(num_entities, ent_dim)
    self.rel_emb = nn.Embedding(num_relations, rel_dim)
    self.rel_proj = nn.Embedding(num_relations, ent_dim * rel_dim)
def _project(self, ent_emb, rel_id):
    proj_mat = self.rel_proj(rel_id).view(-1, self.ent_dim, self.rel_dim)
    projected = torch.bmm(ent_emb.unsqueeze(1), proj_mat).squeeze(1)
    return projected
def forward(self, head_idx, rel_idx, tail_idx):
    h_emb = self.ent_emb(head_idx)
    r_emb = self.rel_emb(rel_idx)
    t_emb = self.ent_emb(tail_idx)
    h_proj = self._project(h_emb, rel_idx)
    t_proj = self._project(t_emb, rel_idx)
    score = torch.norm(h_proj + r_emb - t_proj, p=2, dim=1)
    return -score # Negative distance (higher = better)

def train(model, optimizer, data, device):
    model.train()
    optimizer.zero_grad()
    pos_edge_index = data.train_pos_edge_index
    neg_edge_index = negative_sampling(edge_index=data.train_pos_edge_index,
        num_nodes=data.num_nodes,num_neg_samples=pos_edge_index.size(1),
        method='sparse')
    rel_idx = torch.zeros(pos_edge_index.size(1), dtype=torch.long, device=device)
    pos_scores = model(pos_edge_index[0], rel_idx, pos_edge_index[1])
    neg_scores = model(neg_edge_index[0], rel_idx, neg_edge_index[1])
    margin = 1.0
    loss = torch.mean(F.relu(margin - pos_scores + neg_scores))
    loss.backward()
    optimizer.step()
    return loss.item()

@torch.no_grad()
def evaluate(model, data, device, pos_edge_index, neg_edge_index):
    model.eval()
    rel_idx = torch.zeros(pos_edge_index.size(1), dtype=torch.long, device=device)
    pos_scores = model(pos_edge_index[0], rel_idx, pos_edge_index[1])
    neg_scores = model(neg_edge_index[0], rel_idx, neg_edge_index[1])
    scores = torch.cat([pos_scores, neg_scores]).cpu().numpy()
    labels = torch.cat([torch.ones(pos_scores.size(0)), torch.zeros(neg_scores.size(0))]).cpu().numpy()
    auc = roc_auc_score(labels, scores)
    ap = average_precision_score(labels, scores)
    return auc, ap

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
data = Planetoid(root='./', name='Cora')[0]
data = train_test_split_edges(data, val_ratio=0.05, test_ratio=0.1)
num_entities = data.num_nodes
num_relations = 1 # Cora has single relation type (citation)
ent_dim,rel_dim = 100, 100
model = TransR(num_entities, num_relations, ent_dim, rel_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=1e-5)
for epoch in range(1, 501):
    loss = train(model, optimizer, data.to(device), device)

```

10. Рассуждения в графах знаний

Часто возникает проблемы выяснить по графу знаний не ответ на простой вопрос, который представляет из себя пару (исходный нод, тип ребра), а ответ на более сложный вопрос, который требует последовательного перемещения по нескольким нодам и/или нескольким типам ребер. Процесс поиска такого ответа (выявление оптимальной последовательности переходов по графу знаний) называется рассуждением или логической цепочкой рассуждений.

Основной проблемой решения этой задачи является неполнота графа знаний: в реальных графах знаний может отсутствовать >90% связей, поэтому прямой перебор возможных путей не является решением. Даже если логическая цепочка верна, одно пропущенное ребро разрывает весь путь рассуждения.

Поэтому нужны модели, способные предсказывать отсутствующие связи на лету, в процессе рассуждения. Такое моделирование называется рассуждениями в графах знаний (Knowledge Graph Reasoning).

10.1. Типы сложных запросов

Запросы для рассуждений делятся на три основные категории:

1. Одношаговые запросы (One-hop queries).

Например: “Кто папа Коли?”. Такие запросы решаются напрямую через TransR, как прогноз одного перехода по ребру.

2. Многошаговые запросы (Path queries).

Например: “Кто дедушка Коли?”. Решение таких запросов требует двух шагов:

- Сначала найти родителей Коли (папа и мама),
- Затем найти их отцов (два деда).

Следует заметить, что промежуточные ноды спрашивающему неинтересны, ему требуется итоговый ответ. Следует заметить, что на каждом шаге рассуждения число кандидатов в ответы растёт. При этом в графе много рёбер отсутствует, поэтому нельзя просто пройти по нодам графа используя известные ребра, а нужно предсказывать промежуточные узлы, а по ним - следующие.

3. Conjunctive queries (конъюнктивные, несколько запросов соединенных логическим “И”).

Например: “Кто из студентов ходил на занятия и получил хорошую оценку?”. Решение такого запроса требует:

- Найти множество студентов,
- Найти тех, кто ходил на занятия,
- Найти тех, кто получил хорошую оценку,
- Пересечь все три множества.

Ключевые слова “И” или “Из” указывают на необходимость пересечения результатов.

10.2. Коробочные эмбединги

Ограничение точечных эмбедингов (TransE, TransR) для решения задач рассуждения на графах знаний в том, что они представляют ответ как точку в пространстве эмбедингов. Это приемливо при одношаговых запросах, когда можно сразу ранжировать результаты и выбрать лучшие. При многошаговых или конъюнктивных запросах ранжировать ответы нужно на каждом шаге рассуждений, если этого не делать - ошибка накапливается и предсказание ошибается. Проблема выбора лучшего ответа в результате цепочки рассуждений реализуется в алгоритме коробочного эмбединга (Box Embeddings).

В этом алгоритме каждый ответ (нод) представляется не точкой, а гиперпараллелепипедом (“ящиком”) в пространстве эмбедингов R^d . Его параметры (Box = (center, offset)) определяются двумя векторами: $center \in R^d$ - центр ящика; $offset \in R^d$ - вектор из центра на угол ящика. Таким образом, для каждого запроса мы решаем не только задачу суммирования эмбедингов в пространстве отношений (предсказание center как в TransR), но и задачу прогноза размеров ящика (предсказание offset). Преимущества такого подхода в том, что он естественно моделирует неопределённость и вариативность ответа, а некоторые операции над множествами (например, “является подкатегорией” или “И”) реализуются как вложение одного ящика в другой или пересечение ящиков соответственно.

При выполнении запроса каждый шаг рассуждений смещает центр и расширяет его размеры. Пересечение двух боксов формируют новый бокс (или пустое множество если ответа нет).

BoxE и Query2Box - современные модели, использующие этот подход для сложных рассуждений.

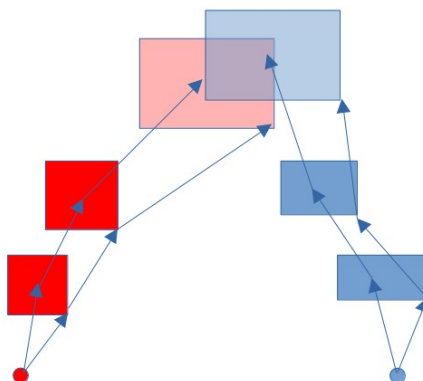


Рис.32 Иллюстрация рассуждений в пространстве ребер с помощью коробочного эмбединга

10.3.Ограничения коробочного эмбединга

При использовании бокс-эмбединга мы предполагаем, что для представления сущностей мы должны проецировать не только центр коробки (box), но и её границы (края). Основная проблема метода возникает при попытке выполнить логические операции над такими представлениями.

Точное вычисление объединения, например, оказывается невозможным: такая область не может быть описана параллелепипедом. Точное представление отрицания практически невозможно в терминах коробок, поскольку сводится к вычитанию коробки из всего множества ответов (максимально возможная коробка в пространстве эмбедингов ответов).

Таким образом, объединение описывается только охватывающим боксом (менее точно), а отрицание НЕ - нельзя выразить через бокс вообще.

Точное вычисление пересечения также может вызывать проблемы. Когда мы строим коробочные эмбединги для ответов на запросы, у нас могут получаться разные коробки для разных условий. Если таких условий (коробок) мало, их пересечение можно вычислить - и мы получим валидный ответ (например, коробку в пересечении). Но если коробок слишком много, коробки перестают пересекаться, и мы не можем однозначно определить ответ - модель теряет способность точно представлять сложные логические комбинации.

Поэтому точнее всего работает операция “И” (пересечение).

Таким образом, не все логические запросы могут быть обработаны напрямую с помощью коробочных эмбедингов. Однако существует способ обойти ограничение - использовать только запросы, в которых присутствует только одна логическая операция на выходе запроса, а все остальные логические операции - И были выполнены до финальной операции.

Применяя такие преобразования, можно все сложные конструкции внутрь, оставляя на выходе только операции объединения, которую можно эффективно обработать с помощью коробочных эмбедингов.

11.Задачи уровня подграфа

К задачам уровня подграфа относятся, например, задачи поиска подграфов в графе по заданным условиям и проверки вхождения одного графа в другой.

Эта задача актуальна, например, при разработке лекарств. Предположим, у нас есть несколько действующих препаратов, и мы хотим найти новые препараты с общими структурными фрагментами (мотивами), присутствующими в известных действующих препаратах. Мотив - это повторяющийся подграф, характерный данного графа или семейства графов. Такая задача будет задачей уровня подграфа.

11.1.Выделение подграфов

Существует два основных способа выделения подграфов из графа:

1. Индуцированный узлами (Node-induced subgraph):

- Выбираем подмножество узлов;
- Включаем все рёбра, соединяющие эти узлы в исходном графе.

Метод чаще применяется в химии и фармакологии, когда решается задача прогноза ребра, и важны именно группы атомов, а связи между ними скорее вторичны.

2. Индуцированный рёбрами (Edge-induced subgraph):

- Выбираем подмножество рёбер;
- Включаем все узлы, на которые опираются эти рёбра.

Метод чаще используется в графах знаний, когда решается задача поиска нод, и важны отношения (типы ребер), а сами ноды (сущности) скорее вторичны.

11.2.Проверка изоморфизма графов - классические подходы

Задача проверки, содержится ли заданный подграф в большом графе, может рассматриваться как общий случай проверки изоморфности двух графов, то есть совпадают ли эти два графа при соответствующей перестановке номеров нод.

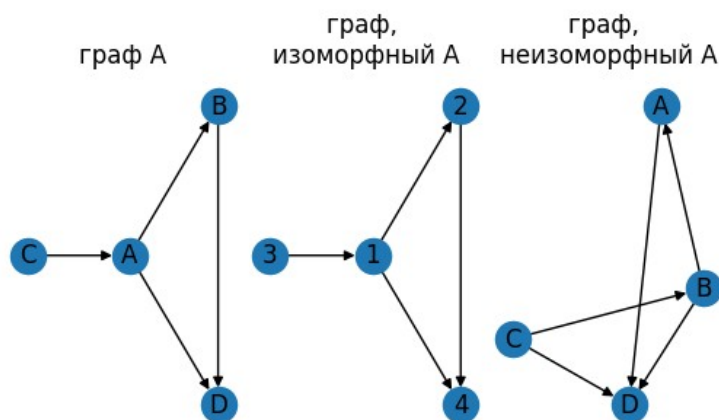


Рис.33 Примеры изоморфных и неизоморфных графов.

Сложность задачи проверки изоморфизма двух графов состоит в том, что все узлы графа могут быть однотипными (например, молекулы состоящие только из углерода), и их нельзя различить по меткам. Тогда приходится перебирать все возможные соответствия узлов, а это NP-трудная задача и для больших графов неразрешима за разумное время.

Чтобы уменьшить число переборов при перестановках, можно провести дополнительные проверки для отсева явно неизоморфных графов. Например:

- Сравнить общее число нод в графе - если число нод графах отличаются - они неизоморфны;
- Добавить к каждой ноде структурные признаки и использовать представление нод в пространстве классических признаков, для поиска наиболее вероятных кандидатов в соответствующие ноды в двух графах;
- В качестве признаков можно использовать классические детерминированные признаки, например степень ноды или гистограмму графлетов, в которых она участвует. Гистограмма классических признаков, построенных по всему графу, также является достаточно уникальным портретом графа и позволяет быстро отсеять явно неизоморфные графы;
- Создать графовый эмбединг на основе топологических признаков, например ядра Вейсфелера-Лемана, если эмбединги не совпадают - графы неизоморфны.

11.3.Проверка вхождения графов методами машинного обучения

Задача определения, содержится ли подграф G_1 в графе G_0 является не менее сложной, чем проверка изоморфности двух графов. По аналогии с проверкой на изоморфизм, в классическом варианте можно использовать полный перебор, однако это будет NP-сложной задачей, вычислительно невозможной (или крайне ресурсоемкой) при больших размерах графов. В этом подходе можно лишь сократить варианты перебора методами, упомянутыми выше.

11.3.1.Неотрицательные упорядоченные эмбединги

Один из способов решения задачи - использовать графовые нейронные сети (GNN) для сравнения графов через эмбединги.

Одним из простых нейросетевых способов решения задачи вхождения графов является сравнение через локальные окрестности. В этом случае для каждого графа выделяются «локальные подграфы» вокруг якорных узлов (anchor nodes). При этом окрестность уровня K - это множество узлов, достижимых из якорной ноды за $\leq K$ шагов. Эти окрестности можно генерировать через поиск в ширину (BFS), или через случайные блуждания (Random Walk) с контролем политики обхода (вперёд, назад, по кругу). После этого каждая окрестность кодируется в эмбединг с помощью графовой нейронной сети (GNN), и эмбединги сравниваются.

Другой способ — это метод неотрицательных матричных эмбедингов. Ключевая идея метода состоит в том, что должно выполняться транзитивное свойство вложения над эмбедингами графов также, как и над самими графами. Это означает, что если граф G_1 является подграфом G_0 , а граф G_2 — подграф G_1 , то G_2 — подграф G_0 . Аналогичное отношение должно выполняться над их эмбедингами:

$$Z_{G_0} \geq Z_{G_1} \geq Z_{G_2}$$

Чтобы это обеспечить выполнение этого свойства, вводится упорядоченное пространство эмбедингов. Это неотрицательные упорядоченные эмбединги, обладающие следующими свойствами:

1. Каждому графу сопоставляется эмбединг - вектор $Z_G \in R^d$.
2. Эмбединг строится неотрицательным: $Z_{G,i} \geq 0, \forall i$ (все компоненты неотрицательны)
3. Выполняется условие вложения: если граф А - подграф графа В ($A \subseteq B$), то $Z_{A,i} \leq Z_{B,i}, \forall i$ (все компоненты А не превышают соответствующих компонент В).

Геометрически эмбединг графа В определяет гиперпараллелепипед от нуля до Z_B ; все подграфы В лежат внутри этого гиперпараллелепипеда. То есть эмбединг подграфа лежит ближе к нулю, чем эмбединг графа.

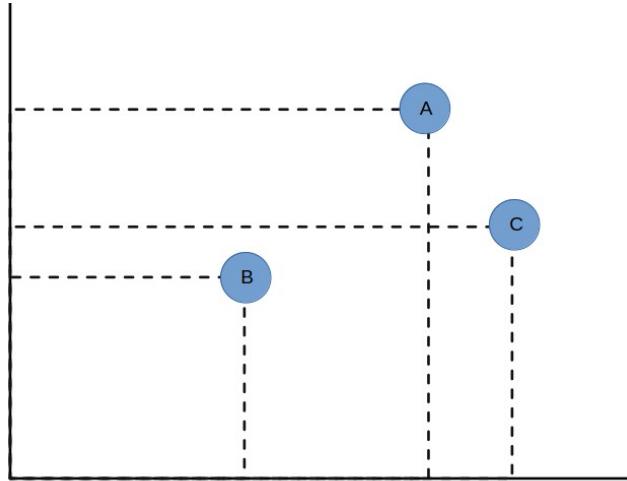


Рис.34 Пример неотрицательных эмбедингов в двумерном пространстве. Граф В вложен в графы А и С, граф С не является вложенным в граф А.

11.3.2. Функция потерь для поиска неотрицательных эмбедингов

Для обучения используется контрастная функция потерь, состоящая из двух частей. Для позитивных пар ($A \subseteq B$) минимизируем разницу между эмбедингами (нарушение порядка):

$$L_{pos} = \|ReLU(Z_A - Z_B)\|^2$$

Для негативных пар ($A \not\subseteq B$) максимизируем разницу между эмбедингами, вводя отступ (margin) $\gamma > 0$:

$$L_{ng} = ReLU(\gamma - \|Z_A - Z_B\|)$$

Полная функция потерь будет иметь вид:

$$L = L_{pos} + L_{ng}$$

Эта функция заставляет нейросеть раздвигать эмбединги невложенных графов друг от друга, сохраняя порядок для вложенных графов. Построенные таким образом неотрицательные эмбединги обладают следующими свойствами:

- $L_{pos} = 0$ тогда и только тогда, когда $Z_{A,i} \leq Z_{B,i}, \forall i$;
- L_{ng} штрафует, если невложенные графы слишком близки по эмбедингам (ближе чем отступ γ).

Генерация обучающих данных обычно идет следующим образом:

1. Берем или генерируем набор графов
2. Позитивные примеры выбираем таким образом:
 - Берём некий граф G ;
 - Случайно выбираем подмножество узлов;
 - Строим node-induced subgraph (включаем все рёбра между выбранными узлами);
 - Эта пара (подграф, G) будет положительным примером
3. Негативные примеры выбираем следующим образом:
 - Берём два случайных графа G_1, G_2 из набора;
 - Эта пара (G_1, G_2) будет отрицательным примером.

Это грубое приближение: иногда случайные графы могут быть изоморфны или быть вложенными друг в друга, но для больших размеров графов в среднем применимо.

Алгоритм 37. Пример обучения модели для вычисления неотрицательных эмбедингов по графу

```
import torch
import torch.nn as nn
from torch_geometric.nn import GCNConv
from torch_geometric.utils import subgraph
from torch_geometric.data import Data, Batch
import torch.nn.functional as F

def get_subgraph(data, node_idx):
    node_idx = torch.tensor(node_idx) if not isinstance(node_idx, torch.Tensor) else node_idx
    edge_index, _ = subgraph(node_idx, data.edge_index, relabel_nodes=True)
    x = data.x[node_idx] if data.x is not None else None
    return Data(x=x, edge_index=edge_index)

class OrderEmbedding(nn.Module):
    def __init__(self, node_dim, hidden_dim, embed_dim):
        super().__init__()
        self.gnn = GCNConv(node_dim, hidden_dim)
        self.projection = nn.Sequential(nn.Linear(hidden_dim, hidden_dim),
                                         nn.ReLU(), nn.Linear(hidden_dim, embed_dim))
        self.embed_dim = embed_dim

    def forward(self, x, edge_index, batch):
        h = F.relu(self.gnn(x, edge_index))
        graph_embed = torch.zeros(batch.max()+1, h.size(1), device=h.device)
        graph_embed = graph_embed.index_add_(0, batch, h)
        counts = torch.bincount(batch)
        graph_embed = graph_embed / counts.unsqueeze(1).float()
        return self.projection(graph_embed)

def generate_training_pairs(graphs, num_pairs=1000):
    pairs = []
    labels = []
    for _ in range(num_pairs):
        try:
            g1 = random.sample(graphs, 1)[0]
            nodes = random.sample(range(g1.x.shape[0]), 4)
            g2 = get_subgraph(g1, nodes)
            pairs.append((g1, g2))
            labels.append(1)
            g3, g4 = random.sample(graphs, 2)
            if True:
                pairs.append((g3, g4))
                labels.append(0)
        except:
            pass
```

```

    if len(labels)>num_pairs:
        break
    return pairs, labels
def order_loss(sub_embed, super_embed, margin=1.0):
    diff = F.relu(sub_embed - super_embed)
    return torch.sum(diff ** 2, dim=1)
def contrastive_loss(embeddings, labels, margin=1.1):
    sub_embed, super_embed = embeddings
    pos_loss = order_loss(sub_embed[labels==1], super_embed[labels==1])
    neg_loss = F.relu(margin -
        torch.norm(sub_embed[labels==0] - super_embed[labels==0], dim=1)) ** 2
    return pos_loss.mean() + neg_loss.mean()
pairs, labels = generate_training_pairs(graphs, num_pairs=500)
labels = torch.tensor(labels)
model = OrderEmbedding(node_dim=4, hidden_dim=16, embed_dim=8)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
model.train()
for epoch in range(100):
    total_loss = 0
    for i in range(0, len(pairs), 32): # Batch processing
        batch_pairs = pairs[i:i+32]
        batch_labels = labels[i:i+32]
        sub_batch = Batch.from_data_list([p[0] for p in batch_pairs])
        sub_embed = model(sub_batch.x, sub_batch.edge_index, sub_batch.batch)
        super_batch = Batch.from_data_list([p[1] for p in batch_pairs])
        super_embed = model(super_batch.x, super_batch.edge_index, super_batch.batch)
        loss = contrastive_loss((sub_embed, super_embed), batch_labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

11.4. Поиск значимых подграфов

11.4.1. Оценка частоты подграфов

Чтобы определить, является ли подграф значимым (а не случайным), нужно оценить, насколько часто он встречается в графе. Это сводится к решению двух задач - определение частоты появления подграфа в графе и оценка ее значимости.

Существуют два подхода к определению частоты появления подграфа в графе:

1. Частота на уровне графа (graph-level frequency).

Напрямую считается общее число вхождений подграфа во всём графе. Проблема метода в том, что число может быть большим и резко растёт с ростом числа нод. Например, если искать звезду из 6 лучей в звезде из 25 лучей, число вхождений $C_{25}^6 = \frac{25!}{19!6!} \approx 1.8 \cdot 10^4$.

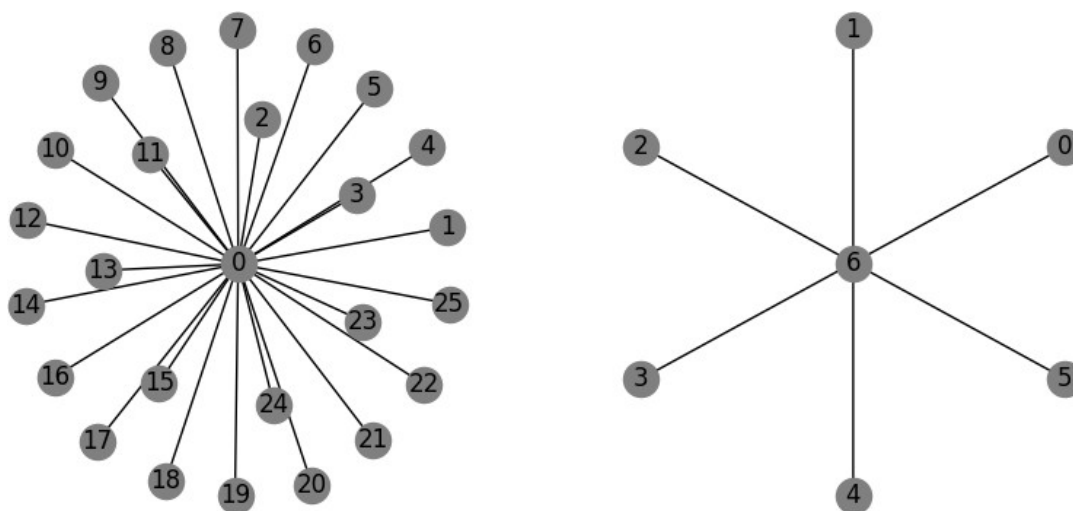


Рис.35 К расчету частоты на уровне графа

2. Частота на уровне узла (node-level frequency).

Выбираем якорный узел (anchor node) в подграфе и считаем, сколькими способами можно разместить этот якорный узел в исходном графе так, чтобы подграф разместился в графе полностью. Преимущество метода в том, что частота не превышает числа узлов в графе. Недостаток метода в том, что частота зависит от выбора якорного узла. Поэтому при сравнении графов необходимо использовать одно и то же положение анкерного нода в подграфе.

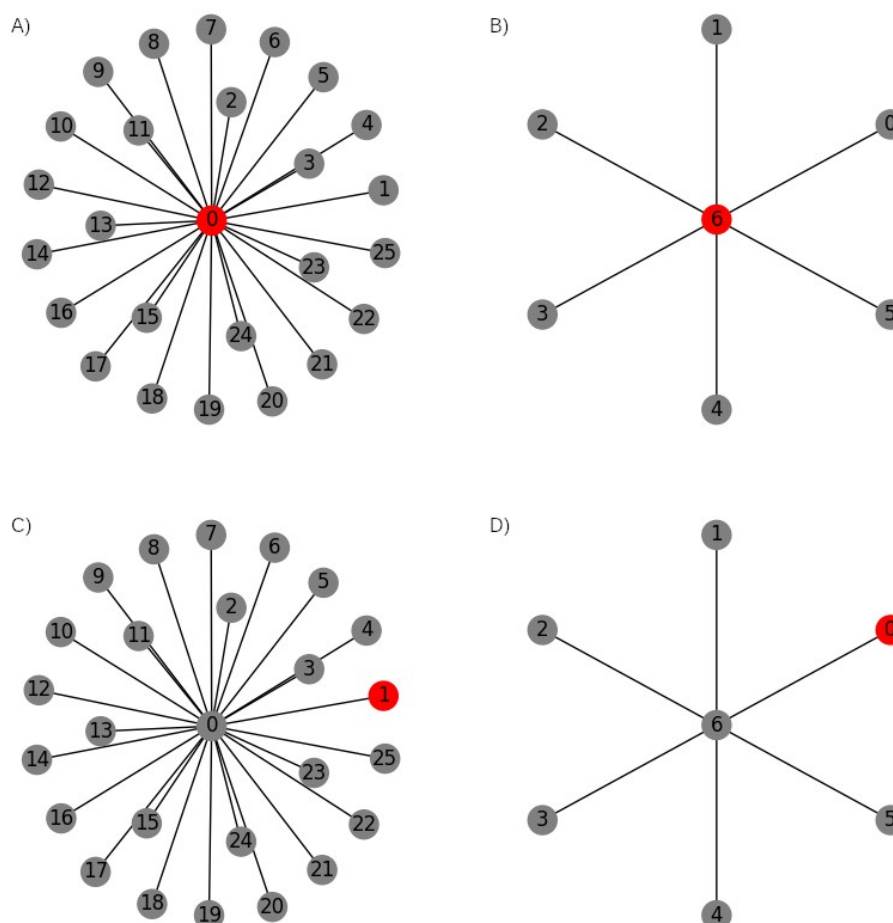


Рис.36 К расчету частоты на уровне узла с различными анкорными нодами.

A)-B) частота = 1

C)-D) частота = 25

11.4.2. Генерация случайных графов для сравнения

Чтобы понять, является ли частота подграфа значимой, её сравнивают с распределением частот появления этого подграфа в ансамбле случайных графов с теми же свойствами.

Можно использовать модели случайных графов, например модель Эрдёша-Реньи, где каждая пара узлов соединяется с фиксированной вероятностью p . Однако такие графы существенно отличаются от исходного графа - распределение степеней вершин не совпадает, и это легко может быть проверено.

Чтобы сравнение было корректным, случайные графы должны сохранять ключевые свойства исходного графа. Одно из них - распределение степеней вершин. Поэтому чаще используют случайные графы, сохраняющие распределение степеней по нодам. Существует два основных способа построения таких случайных графов.

Способ 1. Генерация всех ребер с сохранением степеней вершин.

- Берём исходный граф;
- Размыкаем все ребра в графе, оставляя только «кочерыжки» от них (степени);
- Случайно соединяем кочерыжки между собой, сохраняя тем самым степени.

Этот метод требует генерации нового графа и полной генерации новых ребер.

Способ 2. Тасование рёбер.

- Выбираем два случайных ребра: (A-B) и (C-D) и удаляем их из графа;
- Меняем их концы и получаем новые ребра (A-D) и (C-B) в граф;
- Повторяем операцию много раз (≈ 100 раз достаточно).

При обмене нужно избегать образования самопетель (типа A-A) и дублирования рёбер (если граф простой). Эти процедуры сохраняют степени каждого узла, но делают структуру графа случайной.

11.4.3. Поиск значимых подграфов методами проверки гипотез

Когда в графе обнаруживается характерный подграф, он может служить признаком (“мотивом”) определённого свойства системы (например, биологической активности молекулы). Чтобы определить, является ли частота его появления статистически значимой, используется стандартный статистический подход:

1. Считается частота появления данного подграфа в исходном графе.
2. Сравнивается с распределением частот этого же подграфа в случайных графах (сгенерированных, например, методом тасования рёбер).
3. Вычисляется p-value попадания наблюдаемой частоты в доверительный интервал случайного распределения или считается аналогичная статистика.

Если значение выходит за пределы 95% доверительного интервала, считается, что подграф встречается значимо чаще, чем в случайном случае, и, следовательно, он не случаен - он характерен для данного графа.

Чтобы количественно оценить значимость, можно использовать Z-статистику:

$$Z = \frac{N_{real} - \mu_{rand}}{\sigma_{rand}}$$

где N_{real} - число вхождений подграфа в исходном графе, μ_{rand} - среднее число вхождений по ансамблю случайных графов, σ_{rand} - стандартное отклонение числа вхождений по ансамблю случайных графов.

Статистическая интерпретация:

- Если $|Z| \leq 1.96$ → подграф случаен (входит в 95% доверительный интервал).
- Если $|Z| > 1.96$ → подграф значим (встречается/не встречается не случайно).

При большом количестве исследуемых подграфов можно также строить профиль значимости (significance profile) - вектор Z-оценок для множества мотивов, нормализованный по L2 или L1 норме:

$$P_i = \frac{Z_i}{\sqrt{\sum_j Z_j^2}}$$

суммирование идет по исследуемым подграфам.

Этот максимумы и минимумы в профиле показывают, какие мотивы наиболее выражены в графе.

Алгоритм 38. Оценка значимости заданного графлета в графе

```
import numpy as np
import networkx as nx
import pyfglt.fglt as fg
def switch(g0):
    edges=np.array(g0.edges())
    g1=nx.Graph()
```

```

for itt in range(100):
    s1=np.random.choice(range(edges.shape[0]))
    s2=np.random.choice(range(edges.shape[0]))
    p=edges[s1,1]
    q=edges[s2,1]
    edges[s1,1]=q
    edges[s2,1]=p
for i in range(edges.shape[0]):
    g1.add_edge(edges[i,0],edges[i,1])
return g1
g0=nx.karate_club_graph()
F0=fg.compute(g0)
gltarr=[]
for k in range(100):
    g0=nx.karate_club_graph()
    edges=np.array(g0.edges())
    g1=switch(g0)
    F1=fg.compute(g1)
    gltarr.append(F1.sum(axis=0)[4])
gltarr=np.array(gltarr)
m1=gltarr.mean()
m0=F0.sum(axis=0)[4]
Zsc=(m0-m1)/gltarr.std()
print('Zscore ',Zsc)

```

Zscore 6.265031821483471

11.4.4.Извлечение наиболее частых подграфов заданного размера

Задача поиска наиболее частых подграфов размера k в графе относится к области анализа структуры графов, и применяется например, для обнаружения микросообществ, в задачах биоинформатики или анализа социальных сетей. Нет универсального быстрого алгоритма для поиска всех подграфов размера k в произвольном графе. Выбор метода зависит от размера графа, значения k , типа подграфа и требуемой точности.

К основным методам можно отнести:

- Перебор всех возможных подмножеств из k вершин и проверка, образуют ли они полный подграф (клик) или другой заданный тип подграфа — это базовый, но вычислительно затратный метод. Для больших графов и значений k (например, $k > 10$) он неприменим из-за экспоненциального роста времени.
- Алгоритмы поиска клик (clique detection), такие как алгоритм Брона–Кербоша, позволяют эффективно находить все клики в графе. Алгоритм является рекурсивным.
- Алгоритмы приближённого поиска (например, на основе жадного выбора или методов поиска в глубину) могут использоваться для больших графов, но не гарантируют нахождение всех подграфов. Они подходят для случаев, когда требуется быстро оценить наиболее часто встречающиеся структуры.
- Алгоритмы сравнения графовых эмбедингов, чтобы сравнивать подграфы по вхождению в граф, используемые для жадного поиска подграфов.

Рассмотрим задачу нахождения k -нодовых подграфов, которые встречаются в графе наиболее часто, с использованием графовых эмбедингов. Одно из известных решений задачи - жадный алгоритм поиска через якорные узлы и неотрицательный эмбединг.

Алгоритм заключается в следующем.

1. Обучаем на графах неотрицательный эмбединг;
2. Выбираем небольшие начальные подграфы (например графлеты);

3. К каждому подграфу добавляем одного соседа из графа, выбираем уникальные, входящие в исходный граф, вхождение оцениваем согласно неотрицательному эмбедингу;
4. Из всех вариантов выбираем подграф с наибольшим числом вхождений в полный граф.
5. Повторяем шаги 3,4 пока не достигнут необходимый размер подграфа

12.Рекомендательные системы

Рекомендательные системы - популярная задача машинного обучения. Задача формулируется следующим образом. Есть множество пользователей и множество товаров. Одни пользователи выбирают одни товары, другие пользователи - другие товары. Нужно каждому пользователю предсказать, какие товары ему могут понравиться.

Задачу можно решать в рамках графовых нейронных сетей, предсказывая понравившиеся пользователю товары, как ноды товаров, близкие (соединенные ребром) к ноду пользователя. Граф, на котором решается задача состоит из двух типов нод — ноды пользователей и ноды товаров, каждый своими свойствами, причем отсутствуют непосредственные связи между различными нодами пользователей. Также отсутствуют непосредственные связи между различными нодами товаров.

Граф, который можно разделить на две части без внутренних связей внутри этих частей, называется двудольным. Двудольные графы часто применяются в рекомендательных системах. Фактически, это модель рекомендательной системы. Когда решается задача на уровне ребра (определение, существует ли ребро между двумя узлами), то это ребро можно провести только от одной доли к другой (от U к V) - других вариантов нет. Соответственно, ребро от U к V - это и есть рекомендация (например, что этот пользователь хотел бы купить определенный товар). Узлы типа U - это пользователи. Узлы типа V - это товары.

Если решается задача на уровне узла (определение характеристик узла - пользователя или объекта), то она трансформируется в вопрос: какими общими свойствами обладают пользователи, выбравшие конкретный товар (как их классифицировать), или какими свойствами обладают товары, которые выбрало много пользователей (как их классифицировать).

Такую задачу можно описать в терминах работы с двудольным графом. У двудольных графов существует понятие проекции. Это означает, что граф можно спроецировать на одну из долей. Процесс проекции заключается в полном удалении одной из долей (например, доли V) и соединении узлов оставшейся доли (например, доли U) рёбрами. Новое ребро между двумя узлами проводится, если в исходном графе существовал путь между ними через удалённый узел из другой доли. Например, проекция на долю U (пользователей). Если в исходном графе от пользователя U_1 к пользователю U_3 можно было "добежать" через объект V_A (путь $U_1 \rightarrow V_A \leftarrow U_3$), то после удаления доли объектов V , пользователи U_1 и U_3 будут соединены прямым ребром $U_1|U_3$. Аналогично, если пользователи U_1 и U_2 были связаны через объект V_B (путь $U_1 \rightarrow V_B \leftarrow U_2$), то после проекции они также окажутся соединены одним ребром $U_1|U_2$. Таким образом получается проекция исходного двудольного графа на одну из его долей.

То же самое можно сделать для проекции графа на долю V . Вы удаляете узлы доли U , и если два ребра замыкаются через один узел доли U , вы заменяете их одним ребром. Таким образом появляется проекция на долю V .

Эти проекции показывают, насколько сильно связаны элементы внутри соответствующей доли. Становится понятным, что пользователи U_1, U_3, U_2 достаточно близки, потому что они выбирают один и тот же товар. Это позволяет анализировать близость узлов по каким-то характеристикам.

Главная проблема рекомендательных систем: графы рекомендаций могут быть огромны - миллионы пользователей, миллионы товаров, миллиарды рёбер. Прямой перебор всех товаров для каждого пользователя вычислительно невозможен. Из-за масштаба (миллионы пользователей и товаров) эффективные рекомендательные системы строятся в два этапа.

Этап 1. Генерация (фильтрация) кандидатов (candidate generation). Обычно на этом этапе быстро отбираются тысячи товаров, которые потенциально могут быть интересны пользователю. Обычно генерация кандидатов производится разными методами, а результаты объединяются. Требования к этапу генерации: высокая скорость, допустимы ложные срабатывания. Желательно, чтобы алгоритм был достаточно быстрым, чтобы можно было обработать весь набор вариантов товаров. Примерами таких фильтров может быть выбор наиболее популярных товаров, товары близкие к уже купленным товарам, товары которые выбирали близкие пользователи или пользователи с близкими профилями.

Этап 2. Ранжирование кандидатов (ranking). На этом этапе машинные модели ранжируют кандидатов по релевантности, которая оценивает близость между заданным пользователем и каждым товаром-кандидатом. Необходимые требования к алгоритму - высокая точность алгоритма, при этом можно тратить больше ресурсов. На этом этапе выбираются лучшие варианты (например, топ-10) товаров для каждого пользователя - это и будут рекомендации. На этом этапе можно использовать алгоритмы для ранжирования товаров-кандидатов на основе графовых нейросетевых моделей.

12.1. Модели ранжирования на основе эмбедингов

Задача поиска рекомендаций товаров для пользователя может рассматриваться, как задача прогноза ребра. В рамках графового машинного обучения эта задача сводится к поиску близких нод, в свою очередь сводящихся к поиску нод с эмбедингами, близкими по некоторой метрике. Каждому пользователю и каждому товару сопоставляется вектор в общем пространстве:

$$Z_{user}, Z_{item} \in R^d$$

Сходство нод измеряется функцией над эмбедингами, например, скалярным произведением или евклидовым расстоянием:

$$score(user, item) = Z_{user} Z_{item}^T$$

Чем выше score (или меньше евклидово расстояние), тем выше вероятность рекомендации.

Целью обучения обычно является максимизация Recall@K - доли релевантных товаров среди K рекомендованных.

$$Recall@K = (\text{число релевантных товаров в топ-K}) / K$$

Чем выше эта метрика, тем выше вероятность, что товар понравится пользователю. Для нормализации рекомендаций часто используется Softmax от скалярного произведения, но из-за размера графа его считают не по всем товарам, а только по негативным примерам (используют негативный сэмплинг):

$$P = \frac{\exp(Z_{user} Z_{item}^T)}{\exp(Z_{user} Z_{item}^T) + \sum_{j \in NegativeItems} \exp(Z_u Z_j^T)}$$

12.2. Дифференцируемые потери

Основная проблема обучения рекомендательных систем в том, что Recall@K - недифференцируемая функция, поэтому её нельзя использовать напрямую как функцию потерь для обучения нейросетей. Поэтому вместо нее применяют дифференцируемые потери — бинарную кросс-энтропию или байесовские персонализированные ранговые потери.

12.2.1. Бинарная кросс-энтропия

Бинарная кросс-энтропия (Binary Cross-Entropy Loss, BCE) имеет вид:

$$BCE = - \sum_{(u,i) \in PosPairs} \log(\sigma(\text{score}(u,i))) - \sum_{(u,i') \in NegPairs} \log(1 - \sigma(\text{score}(u,i')))$$

здесь PosPairs - множество позитивных пар (пользователь u взаимодействовал с товаром i); NegPairs - множество негативных пар (нет взаимодействия); σ - сигмоида на выходе модели, аппроксимирующая вероятность рекомендации данного товара.

Недостатком использования бинарной кросс-энтропии является то, что негативные и позитивные отклики суммируются. Это приводит к усредненному пониманию рекомендаций — для модели неважно отношение выходов модели для заданного пользователя на положительных и отрицательных примерах, для нее важнее чтобы в среднем выходы на негативных примерах были менее важными, чем выходы на позитивных примерах.

Например, если товар А куплен чаще В, модель будет рекомендовать А всем, даже если конкретному пользователю больше подходит В. В результате рекомендации становятся усредненными, а не персонализированными. Таким образом, модель стремится предсказать глобальную популярность товаров, а не обеспечить персонализацию для каждого пользователя.

12.2.2. Байесовские персонализированные ранговые потери

Байесовские персонализированные ранговые потери (BPR, Bayesian Personalized Ranking Loss) имеют вид:

$$BPR = - \sum_{u \in Users} \sum_{i \in PosItems} \sum_{i' \in NegItems} \log(\sigma(\text{score}(u,i) - \text{score}(u,i')))$$

где PosItems - товары, с которыми пользователь u взаимодействовал, NegItems - товары без взаимодействия.

Преимущество таких потерь в том, что модель учится ранжировать товары для каждого отдельного пользователя, сравнивая пары «позитивный пример - негативный пример» для одного пользователя. Байесовский персонализированный ранг учитывает относительный порядок рекомендаций для каждого пользователя: для каждого пользователя позитивный товар должен быть ранжирован выше, чем негативный. Это гарантирует, что для любого пользователя позитивный товар всегда ранжируется выше негативного, независимо от других пользователей, что обеспечивает необходимую персонализацию рекомендаций.

12.3. Матричная факторизация

Самая простая модель для решения задачи поиска рекомендаций - матричная факторизация. Идея модели состоит в том, что исходные данные - это разреженная матрица взаимодействий R (пользователи $\times$ товары). В ячейке этой матрицы 0 если взаимодействия не было, и 1 - если было (или рейтинг: чем выше - тем лучше). Ее можно приблизить произведением двух матриц:

$$R \approx UV^T$$

где U - матрица эмбедингов пользователей, V - матрица эмбедингов товаров.

Прогноз рейтинга товара i с точки зрения пользователя u осуществляется исходя из линейной регрессии, где смещения зависят от пользователя и товара:

$$y_{ui} = u_u x_i^T + b_u + b_i + C$$

где u_u - эмбединг пользователя u , v_i - эмбединг товара i , b_u - смещение пользователя u , b_i - смещение товара i , C - глобальное смещение (например, средний рейтинг).

Модель учится так, чтобы прогноз y_{ui} максимально близко соответствовал реальной матрице взаимодействий r_{ui} . Можно использовать стандартные функции потерь - MAE, RMSE (норму Фробениуса), или MSE:

$$Loss = \sum_{(u,i) \in Known} (r_{ui} - y_{ui})^2$$

К преимуществам модели можно отнести простоту, интерпретируемость, и хорошая работа на исторических данных.

К недостаткам модели можно отнести то, что она не работает для новых пользователей (cold start problem), так как у них нет обученного эмбединга. Кроме того, модель не использует собственные признаки нод (возраст, жанр фильма и т. п.), модель учитывает только прямые связи (1-hop окрестность). Поэтому модель не использует информацию о том, что похожие пользователи могут любить похожие товары, даже если у текущего пользователя с ними нет прямой связи.

Алгоритм 39. Предсказание рейтингов фильмов методом матричной факторизации

```
class MatrixFactorization(MessagePassing):
    def __init__(self, num_users, num_items, embedding_dim=50):
        super().__init__(aggr='add')
        self.user_embedding = nn.Embedding(num_users, embedding_dim)
        self.item_embedding = nn.Embedding(num_items, embedding_dim)
        self.user_bias = nn.Embedding(num_users, 1)
        self.item_bias = nn.Embedding(num_items, 1)
        self.global_bias = nn.Parameter(torch.zeros(1))
    def forward(self, user_ids, item_ids):
        user_embed = self.user_embedding(user_ids)
        item_embed = self.item_embedding(item_ids)
        user_b = self.user_bias(user_ids).squeeze()
        item_b = self.item_bias(item_ids).squeeze()
        interaction = (user_embed * item_embed).sum(dim=1)
        prediction = interaction + user_b + item_b + self.global_bias
        return prediction

def train(model, train_data, val_data, epochs=50, lr=0.01, weight_decay=1e-6):
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    criterion = nn.MSELoss()
    train_users = torch.tensor(train_data['user_id'].values, dtype=torch.long)
    train_items = torch.tensor(train_data['item_id'].values, dtype=torch.long)
    train_ratings = torch.tensor(train_data['rating'].values, dtype=torch.float32)
    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        predictions = model(train_users, train_items)
        loss = criterion(predictions, train_ratings)
        loss.backward()
        optimizer.step()
    return model

model = MatrixFactorization(num_users, num_items, embedding_dim=32)
trained_model = train(model, train_df, val_df, epochs=50, lr=0.01)
```


12.4. Учёт более широких окрестностей

Матрица взаимодействий фактически представляет собой матрицу смежности, и ее факторизация в произведение матриц позволяет построить эмбединг ноды на основе ее непосредственных соседей. Таким образом, учитываемая при факторизации матрицы окрестность ограничивается одним соседом, например непосредственно товарами, которые человек покупал. Оно не учитывает более сложные комбинации, например свойства и покупки пользователей, которые покупали похожие товары — окрестность, учитываемая такой моделью, равна единице.

Чтобы преодолеть ограничение малой учитываемой окрестности, используется графовая коллаборативная фильтрация (Neural Graph Collaborative Filtering, NGCF). Идея метода - распространять эмбединги по графу на K шагов, чтобы учитывать пользователей и товары через посредников.

Например, допустим пользователи A и B похожи (оценили одни и те же фильмы одинаково), и пользователь B посмотрел фильм X и поставил 5. Тогда фильм X рекомендуется пользователю A , даже если у A с X нет прямой связи.

Одной из таких архитектур является графовая коллаборативная фильтрация (NGCF).

12.4.1. Архитектура модели NGCF

12.4.1.1. Инициализация эмбедингов

Для решения задачи рекомендаций сначала создаются начальные эмбединги для всех узлов: каждому пользователю u и товару i сопоставляется вектора $Z_u, Z_i \in R^d$. Эти начальные эмбединги не зависят от структуры графа и настраиваются так, чтобы пользователи, покупающие одинаковые товары, имели близкие эмбединги, а товары, покупаемые одним пользователем, были также близки в пространстве эмбедингов. Обычно для этого эмбединги пользователей и товаров объединяются в единую матрицу эмбедингов:

$$Z = (Z_{user} \cup Z_{item}) \in R^{(N_{users} + N_{items}) \times d}$$

и строится начальное приближение — оптимальный эмбединг нод в графе исходя из их соседства, вне зависимости от типа этих нод. Обученная таким образом матрица эмбедингов позволяет сделать первое приближение по построению рекомендаций на основе окрестностей единичного радиуса. Такой эмбединг называется поверхностным (shallow).

12.4.1.2. Распространение эмбедингов и обучение

Следующим этапом полученные эмбединги распространяются по графу с помощью механизма message passing.

На каждой итерации каждый узел собирает эмбединги своих соседей, агрегирует их (обычно через усреднение) и комбинирует с собственным эмбедингом:

$$\begin{aligned} m_v &= \sum_{s \in N(v)} H_s \\ H'_v &= (H_v \parallel m_v) \\ H_v^{new} &= ReLU(W H') \end{aligned}$$

где W - обучаемая матрица проекции. Этот процесс повторяется k раз, где k - глубина модели (количество «хопов»). При $k = 1$: пользователь «видит» только тех, кто купил те же товары. При $k = 2$: он «видит» и пользователей второго порядка, связанных через свои товары, и т.д.

Таким образом, эмбединг пользователя обогащается информацией о расширяющейся окрестности. Фактически каждая итерация близка к графовой свертке или аналогичному ей слою графовой нейронной сети, осуществляющей передачу сообщений.

В алгоритме NGCF эмбединги с выходов всех слоёв, включая начальный эмбединг нод, конкатенируются:

$$Z_v^{final} = (H_v^{(0)} \| H_v^{(1)} \| \dots \| H_v^{(k)})$$

Это позволяет построенному эмбедингу учитывать как локальные, так и глобальные паттерны в расположении связанных с нодой соседей.

После построения такого эмбединга объекты снова разделяются на ноды пользователей u и ноды товаров i . Прогноз рекомендаций строится как скалярное произведение эмбединга пользователя и эмбединга товара:

$$y_{ui} = Z_u^{final} \cdot Z_i^{final}$$

Полученная сеть обучается с одной из функций потерь, это может быть BCE, BPR, или даже MSE:

$$MSE = \frac{1}{|Pairs|} \sum_{u,i \in Pairs} (r_{ui} - y_{ui})^2$$

Следует заметить, что исходный граф направленный (рёбра идут только от пользователей к товарам), и для распространения сообщений он обязательно преобразуется в ненаправленный: для каждого ребра $(u \rightarrow i)$ добавляется обратное ребро $(i \rightarrow u)$. Это позволяет сообщениям распространяться в обоих направлениях: от пользователя к товарам, и от товаров к пользователям. Без этого преобразования распространение сообщений остановится на первом шаге.

NGCF - это простая, но мощная архитектура. Модель учитывает высокоуровневые зависимости через многошаговое распространение информации, что делает рекомендации более персонализированными по сравнению с классической матричной факторизацией.

Алгоритм 40. Реализация модели NGCF для прогноза рейтингов фильмов

```
class NGCFLayer(MessagePassing):
    def __init__(self, embedding_dim, dropout=0.1):
        super().__init__(aggr='add')
        self.embedding_dim = embedding_dim
        self.W1 = nn.Linear(embedding_dim, embedding_dim)
        self.W2 = nn.Linear(embedding_dim, embedding_dim)
        self.dropout = nn.Dropout(dropout)
        self.leaky_relu = nn.LeakyReLU(0.2)
    def forward(self, x, edge_index):
        row, col = edge_index
        deg = degree(col, x.size(0), dtype=x.dtype)
        deg_inv_sqrt = deg.pow(-0.5)
        deg_inv_sqrt[deg_inv_sqrt == float('inf')] = 0
        norm = deg_inv_sqrt[row] * deg_inv_sqrt[col]
        out = self.propagate(edge_index, x=x, norm=norm)
        return self.dropout(self.leaky_relu(out))
    def message(self, x_j, norm):
        messages = self.W1(x_j) + self.W2(x_j)
        return norm.view(-1, 1) * messages

class NGCF(nn.Module):
    def __init__(self, num_users, num_items, embedding_dim=64, num_layers=3, dropout=0.1):
        super().__init__()
        self.num_users = num_users
        self.num_items = num_items
        self.embedding_dim = embedding_dim
```

```

self.num_layers = num_layers
self.user_embedding = nn.Embedding(num_users, embedding_dim)
self.item_embedding = nn.Embedding(num_items, embedding_dim)
self.layers = nn.ModuleList()
for _ in range(num_layers): self.layers.append(NGCFLayer(embedding_dim, dropout))
self.dropout = nn.Dropout(dropout)
def forward(self, edge_index, user_indices=None, item_indices=None):
    x = torch.cat([self.user_embedding.weight, self.item_embedding.weight], dim=0)
    x = self.dropout(x)
    all_embeddings = [x]
    for layer in self.layers:
        x = layer(x, edge_index)
        all_embeddings.append(x)
    final_embeddings = torch.cat(all_embeddings, dim=1)
    user_final = final_embeddings[:self.num_users]
    item_final = final_embeddings[self.num_users:]
    if user_indices is not None and item_indices is not None:
        user_embs = user_final[user_indices]
        item_embs = item_final[item_indices]
        predictions = (user_embs * item_embs).sum(dim=1)
        return predictions
    return user_final, item_final

```

12.4.2. Архитектура LightGCN

При решении задачи прогноза методом NGCF возникает серьёзная проблема: матрица эмбедингов, обучаемая на первом этапе и учитывающая только непосредственных соседей, содержит огромное число параметров $(N_u + N_i) \times d$ (где d — размерность вектора эмбединга), а графовая свёрточная сеть (GCN), учитывающая более далеких соседей, содержит немного, $d \times d$, параметров. Это говорит о том, что архитектура несбалансированна, и подавляющее большинство параметров — просто “память” (эмбединги), а не “вычисление” (сеть). Тем не менее, при расчетах рекомендаций на вычисления приходится существенная доля вычислительных затрат, и наименее информативные операции (вычисления) вычисляются наиболее долго.

Чтобы устранить эту несбалансированность между информативностью и вычислительной нагрузкой, была предложена легкая графовая свёрточная сеть (LightGCN, Light Graph Convolutional Network)[<https://arxiv.org/abs/2002.02126>]. В LightGCN убрано большое количество слоев, и в качестве базового преобразования используется многократное распространение сообщений через граф, после чего выполняется взвешанное суммирование. Анализ показал, что нелинейные преобразования и самоконкатенация в NGCF не дают существенного выигрыша.

Финальный эмбединг узла в рамках модели вычисляется как:

$$Z^{final} = \sum_{k=0}^K \alpha_k (D^{-1/2} \tilde{A} D^{-1/2})^k H^0$$

где H^0 — начальные эмбединги; $\tilde{A} = A + I$ — матрица смежности с петлями; D — диагональная матрица степеней для $\tilde{A}$; K — количество слоёв (глубина распространения); α_k — гиперпараметры важности вкладов разных окрестностей, для простоты их можно выбрать равными: $\alpha_k = 1/(K+1)$.

Таким образом, вся сеть сводится к нескольким линейным преобразованиям, а обучаемыми остаются только начальные эмбединги. Результирующая сеть получается компактной — вместо K нелинейных свёрточных слоёв — матричные умножения, поэтому сеть намного быстрее обучается. Часто LightGCN превосходит NGCF по качеству прогнозов.

Алгоритм 41. Предсказание рейтингов фильмов моделью LightGCN

```
from torch_geometric.nn import LightGCN
class LightGCNRecommender(nn.Module):
    def __init__(self, num_users, num_items, embedding_dim=64, num_layers=3):
        super().__init__()
        self.num_users = num_users
        self.num_items = num_items
        self.lightgcn = LightGCN(num_nodes=num_users + num_items,
                                embedding_dim=embedding_dim, num_layers=num_layers)
    def forward(self, edge_index):
        all_embeddings = self.lightgcn.get_embedding(edge_index)
        return all_embeddings[:self.num_users], all_embeddings[self.num_users:]
    def predict(self, user_ids, item_ids, train_edge_index):
        user_embeddings, item_embeddings = self.forward(train_edge_index)
        user_emb = user_embeddings[user_ids]
        item_emb = item_embeddings[item_ids]
        return (user_emb * item_emb).sum(dim=-1)

def train(model, train_data, val_data, epochs=50, lr=0.01):
    optimizer = optim.Adam(model.parameters(), lr=lr)
    train_ratings = torch.tensor(train_data['rating'].values, dtype=torch.float32)
    train_ratings_norm = (train_ratings - 1.0) / 4.0 # Scale 1-5 to 0-1
    train_edge_idx = torch.tensor([train_data['user_id'].values,
                                   train_data['item_id'].values + num_users], dtype=torch.long)
    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        predictions = model.predict(torch.tensor(train_data['user_id'].values, dtype=torch.long),
                                     torch.tensor(train_data['item_id'].values, dtype=torch.long), train_edge_idx)
        predictions_scaled = predictions * 4.0 + 1.0
        loss = nn.MSELoss()(predictions_scaled, train_ratings)
        loss.backward()
        optimizer.step()
    return model
```

13. Сообщества

13.1. Поиск сообществ в графе

Характерным явлением при анализе социальных сетей являются сообщества (объединения) людей. Сообщества могут возникать по интересам (люди близкой профессии, например программисты), по географическому положению (например живут в одном доме), по историческим мотивам (учились в одном классе). Люди, состоящие в одном сообществе часто характеризуются определенными предпочтениями в образовании связей-ребер: они чаще устанавливают связи с людьми, которые уже состоят в этом сообществе, чем в других сообществах. При поиске сообществ (communities) в графах часто выделяют два основных варианта формирования сообществ:

- связи сообществ через ребра, когда сообщества соединены небольшим количеством связей;

- связи сообществ через общие ноды, когда существуют общие ноды, которые одновременно входят в оба сообщества.

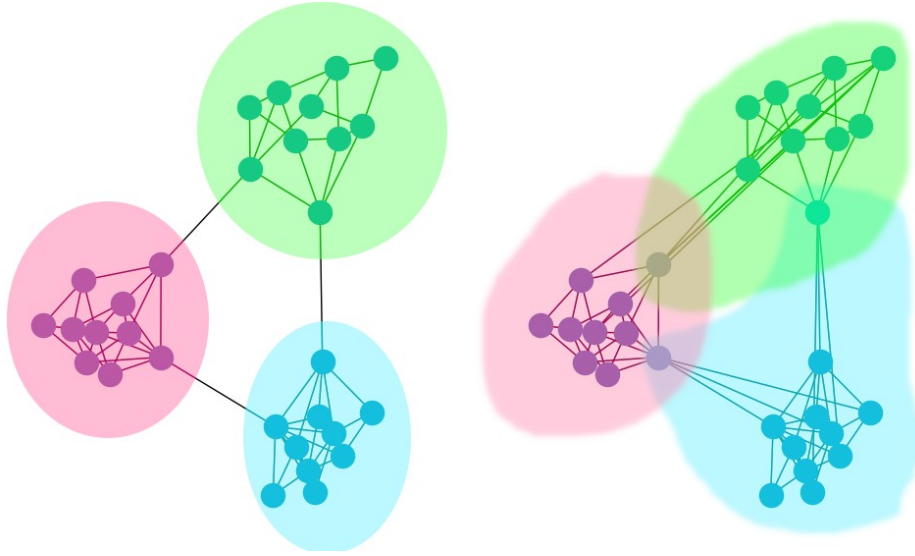


Рис.37. Связи через ребра (слева) и связи через общие ноды (справа).

Для выделения ребер, которые могут потенциально соединять различные сообщества, обычно используют понятие сильных и слабых ребер. Сила ребра между нодами u и v часто определяется мерой перекрытия ребер (edge overlap):

$$O_{u,v} = \frac{|N(u) \cap N(v) \setminus (u,v)|}{|N(u) \cup N(v) \setminus (u,v)|}$$

и определяет отношение числа общих соседей у нод к общему числу соседей у этих нод, исключая само исследуемое ребро (его ноды). Для слабых ребер эта величина близка к нулю, для сильных ребер - близка к единице. Слабые ребра соединяют ноды из разных сообществ, сильные ребра соединяют ноды внутри одного сообщества.

Другим параметром, используемым при кластеризации на сообщества, является модулярность Q , фактически определяющая плотность связей в сообществе:

$$Q = \sum_{s \in S} [|E_s| - |E_{s, \text{expected}}|]$$

здесь S — сообщества внутри данного графа, E_s - ребра в сообществе s , $E_{s, \text{expected}}$ - ожидаемое число ребер в сообществе s для числа нод в этом сообществе. Ожидаемое число ребер определяется как число ребер в случайном графе, имеющем то же самое распределение по степеням нод, что и исходное сообщество. Можно показать, что модулярность может быть вычислена непосредственно по заданному сообществу, как:

$$Q = \frac{1}{2m} \sum_{s \in S} \sum_{u \in s} \sum_{v \in s} \left(A_{u,v} - \frac{k_u k_v}{2m} \right)$$

здесь S - все сообщества графа; k_u, k_v - степени вершин нод (сумма весов в случае взвешанных нод) внутри сообщества s ; $2m$ - сумма весов всех ребер (сумма степеней всех вершин) в графе.

Большое количество различных методов разделения графа на сообщества приведено в библиотеке `cdlib`.

13.2. Поиск сообществ через слабые ребра. Алгоритм Лёвена

Алгоритм Левена для обнаружения сообществ — это жадный метод оптимизации, предназначенный для извлечения непересекающихся сообществ, созданный в Левенском университете. В алгоритме Левена чередуются две фазы.

На первой фазе узлы группируются в сообщества на основе того, как изменяется модульность графа при перемещении узла в другое сообщество. В начале все ноды размещаются в собственные уникальные однонодовые сообщества. Выбор перемещаемой из одного сообщества в другое ноды определяется жадным методом, как выбор ноды, дающей наибольшее увеличение модулярности. Перемещения нод между сообществами продолжается циклически до тех пор, пока модулярность не перестанет улучшаться. Необходимо заметить, что изменение модулярности при перестановки ноды из сообщества в сообщество рассчитывается аналитически, поэтому алгоритм достаточно быстрый.

На второй фазе создается новый граф, где каждое сообщество рассматриваются как отдельный нод. Веса ребер между такими псевдонодами определяются, как сумма весов, соединяющих эти сообщества, каждая псевдонода имеет ребро само на себя с весом, определяемым суммой весов ребер в сообществе.

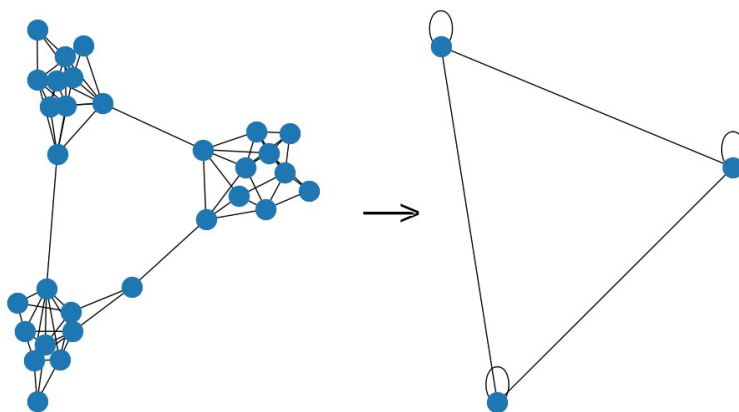


Рис.38. Иллюстрация второй фазы алгоритма Левена

Первая и вторая фазы чередуются, увеличивая размер сообществ и уменьшая их число до тех пор, пока модулярность графа не перестанет расти. Это будет соответствовать оптимальному разделению графа на сообщества с точки зрения алгоритма Левена.

Алгоритм 42. Выделение сообществ в графе Karate Club алгоритмом Левена

```
import networkx as nx
from community import community_louvain
G = nx.karate_club_graph()
partition = community_louvain.best_partition(G, random_state=42)
pos = nx.kamada_kawai_layout(G)
colors = [partition[node] for node in G.nodes()]
nx.draw(G, pos, node_color=colors, with_labels=True)
```

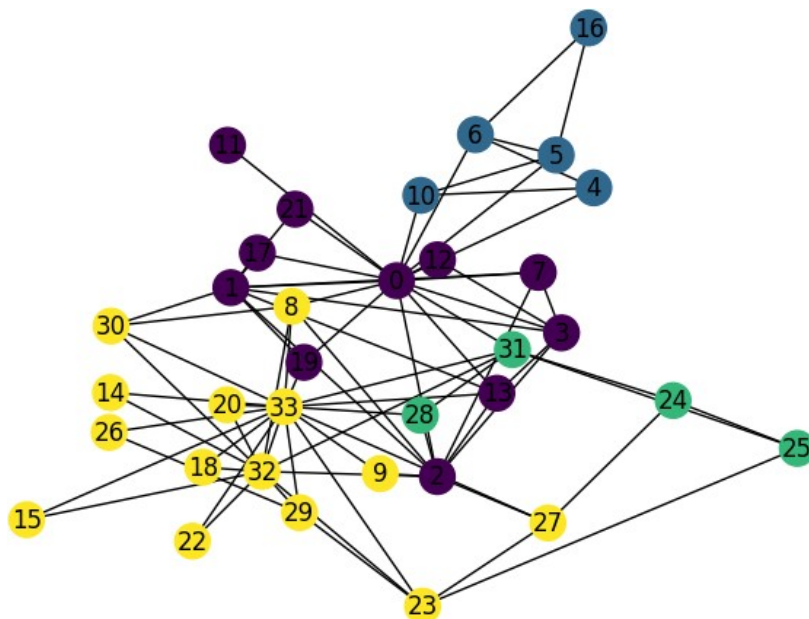


Рис.39. Результат выделения сообществ в графе Karate Club алгоритмом Левена. Разные цвета нод соответствуют различным сообществам.

13.3. Поиск сообществ через общие ноды

13.3.1. Модель графа принадлежности (AGM)

Модель графа принадлежности (AGM) предполагает, что связи между нодами возникают благодаря общему членству в сообществах, при этом вероятность связи возрастает по мере увеличения числа сообществ, к которому принадлежит эта нода.

В режиме генерации графа AGM использует двудольный граф принадлежности с двумя типами нод — членами (например, людьми) и сообществами (например, группами, организациями), — где ребра представляют членство. Полученная социальная сеть (регулярный граф) генерируется на основе общего членства в сообществах.

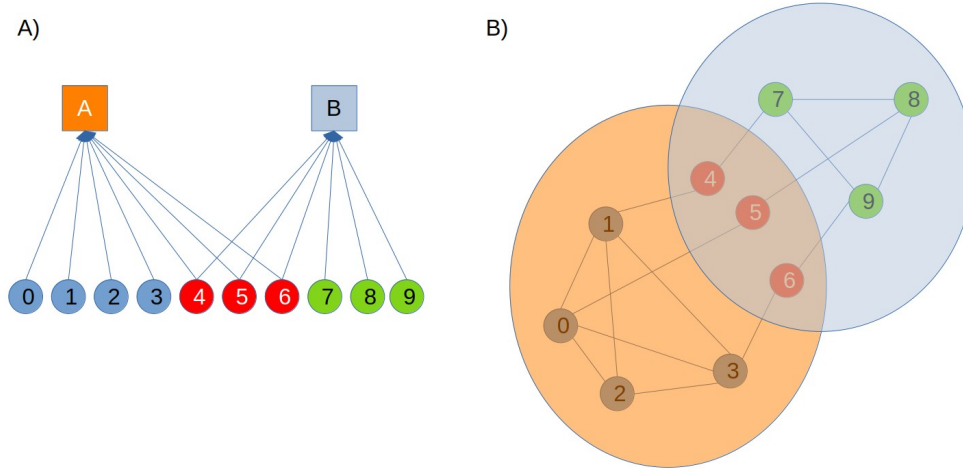


Рис.40. Принцип работы AGM в режиме генерации: А) размещение нод (0-9) на сообщества A,B; В) — сгенерированный граф.

Вероятность ребра между двумя узлами u и v моделируется следующим образом:

$$p(u, v) = 1 - \prod_{c \in M_u \cap M_v} (1 - p_c)$$

Генерация ребер между нодами происходит последовательно — сначала для нод, принадлежащих сообществу A, потом для нод, принадлежащих сообществу B и так далее. С увеличением количества сообществ, которому принадлежит нода, вероятность ее связей растет.

Для заданного размещения нод по сообществам можно посчитать вероятность ребер между нодами и сообществами $F_{u,s}$ и оптимизировать это размещение так, чтобы оно наиболее близко описывалось матрицей смежности анализируемого графа $G(A_{u,v})$:

$$P(G|F) = \prod_{u,v} [(1 - \exp(-\sum_{s \in S} F_{v,s} F_{u,s})) A_{u,v}] \cdot \prod_{u,v} [\exp(-\sum_{s \in S} F_{v,s} F_{u,s}) (1 - A_{u,v})] \rightarrow \max$$

13.3.2. Нейронное обнаружение перекрывающихся сообществ (NOCD)

Идея алгоритма [https://arxiv.org/pdf/1909.12201] состоит в том, чтобы обучить матрицу вероятностей принадлежности нод к сообществам, минимизируя функцию потерь:

$$-\log(P(G|F)) = \sum_{s \in S} \left(-\sum_{(u,v) \in E} \log(1 - \exp(-F_{u,s} F_{v,s}^T)) + \sum_{(u,v) \notin E} F_{u,s} F_{v,s}^T \right)$$

или с учетом дисбаланса существующих E и несуществующих ребер:

$$-\log(P(G|F)) = -E_{(u,v) \in E} [\log(1 - \exp(-F_{u,s} F_{v,s}^T))] + E_{(u,v) \notin E} [F_{u,s} F_{v,s}^T]$$

Модель прогноза принадлежности ноды к сообществу представляет собой двухслойную сверточную сеть:

$$F = ReLU(\tilde{A} ReLU(\tilde{A} X W_1) W_2)$$

Алгоритм 43. Реализация NOCD на датасете Cora

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
import numpy as np
from sklearn.metrics import adjusted_rand_score, confusion_matrix

dataset = Planetoid(root='./', name='Cora')
data = dataset[0]
num_classes = dataset.num_classes
num_nodes = data.num_nodes
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
node_features = data.x.to(device)

class NOCD(nn.Module):
    def __init__(self, nhid=24, nclass=7, nf=1433):
        super().__init__()
        self.gc1 = GCNConv(nf, nhid)
        self.gcout = GCNConv(nhid, nclass)
    def forward(self, x, edge_index):
        x = F.relu(self.gc1(x, edge_index))
        x = F.relu(self.gcout(x, edge_index))
        return x

def myloss(out_pos, out_neg, eps=1e-8):
    loss = -torch.sum(torch.log(1.-torch.exp(-torch.matmul(out_pos, out_pos.t())))+eps))
    loss += torch.sum(torch.matmul(out_neg, out_neg.t()))
    return loss

num_classes = 10
num_nodes=data.num_nodes
model = NOCD(nhid=64, nclass=num_classes).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.1, weight_decay=5e-4)
data = data.to(device)
node_features=data.x.to(device)
model.train()
from torch_geometric.utils import negative_sampling
neg_edge_index = negative_sampling(data.edge_index)
for epoch in range(100):
    optimizer.zero_grad()
    out_pos = model(node_features, data.edge_index)
    out_neg = model(node_features, neg_edge_index)
    loss = myloss(out_pos, out_neg)
    loss.backward()
    optimizer.step()
pred=model(node_features, data.edge_index)
labels=torch.argmax(pred,dim=1)
print(confusion_matrix(data.y,labels))
```

13.3.3.Перенос обучения: использование классификатора

По аналогии с переносом обучения, можно обучать разбиению на сообщества на одном датасете, а использовать его на другом датасете с аналогичными признаками. Например разбиение на сообщества в одной социальной сети может быть аналогичным разбиению в другой социальной сети. Алгоритм эквивалентен обучению классификации, требуется лишь чтобы признаки нод на двух этих графах были идентичными.

Рекомендуемая литература и источники к курсу

- Лекции J.Leskovec, Machine learning with graphs, 2021/2024
- M.Labonne, Hands-On Graph Neural Networks Using Python//Packt, 2023, 332pp.
- Y.Ma and J.Tang, Deep Learning on Graphs//Cambridge University Press, 2021, 400pp, https://yaoma24.github.io/dlg_book/
- L.Wu, P.Cui, J.Pei, L.Zhao, Graph Neural Networks: Foundations, Frontiers, and Applications//Springer, 2022, 689pp, <https://doi.org/10.1007/978-981-16-6054-2>
- K.Broadwater, N.Stillman, Graph Neural Networks in Action//Manning, 2025, 392pp.
- W.L. Hamilton, Graph Representation Learning// Synthesis Lectures on Artificial Intelligence and Machine Learning, 2020, Vol. 14, No. 3 , Pages 1-159.
- D.Easley,J.Kleinberg, Networks, Crowds, and Markets: Reasoning about a Highly Connected World//Cambridge University Press, 2010, 819pp,
- Z.Liu and J.Zhou, Introduction to Graph Neural Networks//Springer, 2020, 127pp, <https://doi.org/10.1007/978-3-031-01587-8>
- A.Negro, Graph-Powered Machine Learning// Manning, 2020, 467pp